EKNATHALABS · INTERVIEW PREP

Terraform Production

Platform, SRE & DevOps interview guide

Questions	**173 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/terraform_interview_guide.html**
Printed	**2026-10-21**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 **What is Terraform and how does it manage infrastructure state?** State

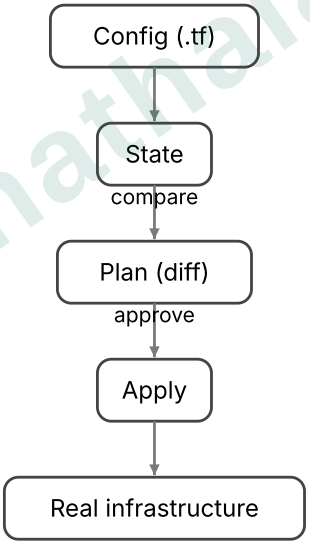
🎙 Spoken answer · ~73 s

Terraform is an infrastructure-as-code tool from HashiCorp. You describe the infrastructure you want in declarative HCL files, and Terraform works out what has to be created, changed, or destroyed to get there, across clouds and other providers through plugins. The key to how it does this is state. Terraform keeps a state file that maps each resource in your configuration to the real object in the world: its ID, attributes, and dependencies. When I run plan, Terraform reads the configuration, refreshes what it knows from the provider, compares it with state, and produces a diff. Apply then executes that plan in dependency order and updates the state. State is therefore Terraform's source of truth about what it manages. That makes it critical: it may contain sensitive values, so it must be stored securely; it must be shared by the team, so I use a remote backend such as S3 with locking; and it must never be edited casually. Commands like state list, state show, and state mv let me inspect and refactor it safely.

Likely follow-ups

- Why shouldn't you edit the state file by hand?
- What happens if state and reality disagree?

🗺 Terraform workflow



certs.eknathalabs.com

Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp that lets you define, provision, and manage infrastructure across multiple cloud providers using declarative HCL (HashiCorp Configuration Language) files.

State management: Terraform maintains a state file (`terraform.tfstate`) that maps real-world resources to your configuration. This file tracks resource metadata, dependencies, and current values — it is Terraform's source of truth.

KEY STATE CONCEPTS

```
terraform state list          # list all tracked resources
```

```
terraform state show <resource> # inspect resource attributes
```

```
terraform refresh              # sync state with real infra
```

Interview tip: Mention that state enables Terraform to calculate diffs (plan), detect drift, and avoid duplicate resource creation. Remote state (S3, GCS, Terraform Cloud) is production best practice.

📚 **Official docs**

- Terraform: State

Q2 **Terraform state file is accidentally deleted — how would you recover?** State

🎙 **Spoken answer · ~66 s**

First, I don't panic and I don't run apply, because applying against an empty state would try to create everything again. Recovery depends on the backend. If it's a remote backend like S3 with versioning, I restore the previous version of the state object; that's the best case, and it's why I always use a remote backend with versioning enabled. With Terraform Cloud, the state history gives me the same thing. If there is no backup, I rebuild state by importing the existing resources: keep the configuration, use terraform import or import blocks for each resource, and run plan until it shows no changes. It's tedious but safe. In the meantime I lock down applies so nobody makes things worse. Afterwards I fix the root cause: a remote backend with versioning and locking, restricted delete permissions on the state bucket, and regular backups. The lesson is that state is critical data and should be treated like a database.

Likely follow-ups

- How would you import 200 existing resources efficiently?
- Which permissions would you restrict on the state bucket?

This is a critical incident. The recovery approach depends on whether you had backups and whether remote state was configured.

RECOVERY STEPS

```
# 1. Check S3 versioning / GCS object versioning for previous tfstate
```

```
# 2. Restore from backup: aws s3api list-object-versions --bucket <bucket>
```

```
terraform import <resource_type>.<name> <resource_id>  # re-import each resource
```

```
terraform refresh  # reconcile after imports
```

If no backup exists, use `terraform import` to re-import each existing resource one by one, then run `terraform plan` to verify no unintended changes are proposed.

Prevention: Always enable S3 versioning + DynamoDB locking. Enable Terraform Cloud state history. Never store state locally in production.

📚 **Official docs**

- Terraform: State

certs.eknathalabs.com

Q3 **Multiple team members are working on the same Terraform code — how do you prevent conflicts?** Backend

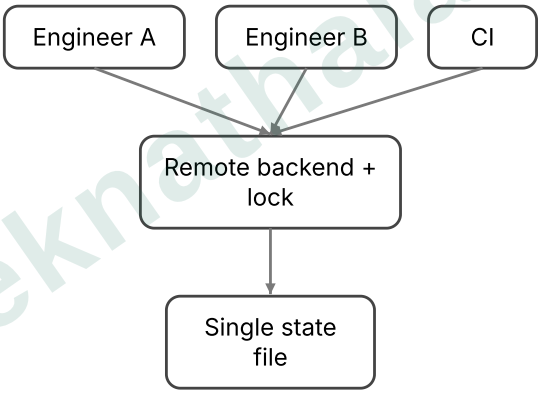
🎙 Spoken answer · ~70 s

The core mechanism is a remote backend with state locking. If state lives on someone's laptop, two people can apply at once and corrupt it or overwrite each other's changes. With an S3 backend and a DynamoDB table for locks, or the native lock support in newer versions, only one apply holds the lock at a time, and others wait or fail with a clear message. Locking prevents concurrent runs, but good team practice goes beyond it. Changes go through pull requests, and the plan is reviewed before anyone applies. Better still, I'd run Terraform from a pipeline or a tool such as Atlantis, HCP Terraform, or Spacelift, so plans are posted on the pull request and applies run from one controlled place with consistent credentials and versions. I'd split state into smaller pieces by environment and team to reduce contention, and separate environments by backend, not just workspaces. I'd pin the Terraform and provider versions, commit the lock file, and restrict who can run apply against production.

Likely follow-ups

- What happens if two applies start at the same time?
- Why run Terraform from CI instead of laptops?

🗺 Shared state



State locking is the core mechanism. Configure a remote backend with locking enabled.

```
terraform { backend "s3" { bucket = "tf-state" key = "prod.tfstate" dynamodb_table = "tf-lock" } }
```

DynamoDB provides atomic lock acquisition — only one `terraform apply` runs at a time. Other best practices: use Terraform workspaces per environment, enforce code review via pull requests before apply, and use Atlantis or Terraform Cloud for run orchestration.

Key answer points: Remote state + locking (DynamoDB/Consul), Atlantis for PR-based workflows, separate workspaces or state files per environment.

📚 Official docs

- HCP Terraform: Recommended practices
- Terraform: Remote state & state storage (team workflows)

certs.eknathalabs.com

Q4

Terraform apply failed in the middle of execution — how do you recover from a partial deployment?

State

🎙 Spoken answer · ~76 s

Terraform isn't transactional, so a failed apply can leave some resources created and others not, and it does not roll back. The good news is that state is updated for each resource as it succeeds, so recovery is mostly re-running. First I read the error to understand why it failed: a quota, a permission, an API error, or a dependency. I fix the cause, then run plan to see what's still missing versus what exists, and apply again. Terraform skips what's already created. Special cases: if a resource was actually created at the provider but isn't in state, because the failure came after creation or the process crashed, then plan will try to create it again and fail with an already-exists error, so I import it into state. If a resource is half-created and broken, I can force recreation with apply -replace on that resource. I also check that the state lock was released, and use force-unlock only if a crashed run left it. Afterwards I make sure the plan is clean, and consider why it failed so I can prevent it.

Likely follow-ups

- What do you do if a resource exists in the cloud but not in state after a failure?
- How do you tell a crashed run's lock from an active one?

Terraform is not transactional. A mid-apply failure leaves some resources created and others not.

RECOVERY PROCESS

```
terraform plan  # identify what's missing vs what exists
```

```
terraform apply  # re-run — Terraform skips already-created resources
```

```
terraform state list  # verify state matches reality
```

If a resource was created outside state (partial creation with no state entry), use `terraform import`. For resources stuck in a bad state, use `terraform taint <resource>` (pre-1.0) or `terraform apply -replace` to force recreation.

Root cause: Check provider API errors, IAM permissions, quota limits, or dependency ordering issues before re-applying.

📚 Official docs

- Terraform CLI: apply

certs.eknathalabs.com

Q5 Terraform shows drift between infrastructure and state — how do you identify and fix it?

State

🎙 Spoken answer · ~73 s

Drift is when real infrastructure no longer matches what Terraform believes, usually because someone changed something in the console or another automation modified it. To identify it, I run terraform plan, which refreshes and shows drift as changes. For a safer look, plan with refresh-only shows what has changed outside Terraform without proposing configuration changes. Then I decide which direction to reconcile. If the manual change was a mistake or an unauthorised change, I apply Terraform to revert the resource to the code. If the change was legitimate, say an emergency fix, I update the configuration to match, so the plan becomes clean, or accept it into state with an apply in refresh-only mode. For attributes managed by something else, like autoscaling counts, I use lifecycle ignore_changes. The longer-term fix is prevention and detection: restrict console changes in production with IAM and organisation policies, run scheduled plans in CI that alert when the plan is not empty, and use a drift detection tool, so drift is found quickly rather than during an unrelated deploy.

Likely follow-ups

- When would you use ignore_changes?
- How would you automate drift detection?

🗺 Drift check



Drift occurs when resources are modified outside Terraform (manual console changes, other automation).

```
terraform plan  # drift shows as "changes" in plan output
```

```
terraform refresh  # update state to match real infra (careful!)
```

```
terraform plan -refresh-only  # preview what refresh would change (TF 1.1+)
```

To fix: either update your Terraform config to match the manual changes (then apply to align), or apply Terraform to revert the manual change back to IaC-defined values. The choice depends on whether the manual change was intentional.

Production advice: Use drift detection in CI pipelines — run `terraform plan` on a schedule and alert on non-empty plans. AWS Config or Driftctl can augment this.

📚 Official docs

- Terraform: State
- Terraform CLI: plan

certs.eknathalabs.com

Q6

Terraform plan shows unexpected resource changes — how do you troubleshoot?

State

🎙 Spoken answer · ~73 s

When a plan shows changes I didn't expect, I look at the diff attribute by attribute instead of just the summary. Common causes: a provider version upgrade that changed defaults or how an attribute is represented; computed attributes that change on every run; drift from outside changes; module version changes; and values that differ by environment, such as variables or workspace. My steps: compare the old and new values in the plan output, and use terraform state show for the resource to see what state holds versus the configuration. Check the lock file and provider versions to see if something moved, since an unpinned provider is the classic culprit. Check whether the diff is a replacement, marked with a force-new attribute, which is dangerous for databases and deserves special attention. If it's still unclear, TF_LOG at debug level shows the provider calls. Fixes include pinning provider and module versions, setting lifecycle ignore_changes for attributes that are legitimately managed elsewhere, correcting the configuration, and importing or refreshing state. I never apply a plan I don't understand.

Likely follow-ups

- How do you spot a destroy-and-recreate in a plan?
- Why pin provider versions?

Unexpected plan changes are often caused by provider upgrades, computed attributes, or config drift.

DEBUGGING STEPS

```
TF_LOG=DEBUG terraform plan 2>&1 | tee debug.log
```

```
terraform state show <resource>  # compare state values with config
```

```
terraform providers lock  # check provider version pinning
```

Common causes: provider version bump changing default attribute values, computed fields (like ARNs) changing on recreation, `ignore_changes` not set for externally managed attributes, or sensitive value encoding differences.

Fix: Pin provider versions in `required_providers`. Use `lifecycle { ignore_changes = [...] }` for attributes managed externally.

📚 Official docs

- Terraform CLI: plan

certs.eknathalabs.com

Q7 **Remote backend (like S3) is not accessible — how do you debug backend issues?** Backend

🎙 Spoken answer · ~76 s

When the remote backend isn't accessible, I check from the outside in, starting with identity. Which credentials is Terraform using? aws sts get-caller-identity shows the actual identity or role. Then permissions: for an S3 backend the role needs list on the bucket, get and put on the state key, and read and write on the DynamoDB lock table if used. A bucket policy, a service control policy, or an explicit deny can block access, particularly across accounts, and KMS key permissions are a common miss when the bucket is encrypted. Then I check connectivity: if the runner is in a private subnet, does it have a VPC endpoint or NAT route to S3 and DynamoDB; are there proxy settings? I test directly with the CLI, for example listing the bucket and key, which separates a Terraform issue from an access issue. I also verify the backend configuration itself: bucket name, key, region, and that environment variables or profiles are set in CI. TF_LOG at trace level during init shows the failing call. Afterwards I make sure the pipeline role has explicit backend access.

Likely follow-ups

- What KMS permissions does an encrypted S3 backend need?
- How do you bootstrap the backend bucket itself?

```
TF_LOG=TRACE terraform init 2>&1 | grep -i backend
```

```
aws s3 ls s3://<bucket>/<key>  # verify bucket access manually
```

```
aws sts get-caller-identity  # confirm correct AWS credentials/role
```

Check: IAM permissions for the executing role (s3:GetObject, s3:PutObject, s3:ListBucket, DynamoDB read/write). Verify VPC endpoint or internet access if running in a private subnet. Check S3 bucket policy — it may block cross-account access.

CI/CD context: Ensure the pipeline IAM role has explicit backend access. Environment variables AWS_ACCESS_KEY_ID / AWS_ROLE_ARN must be set correctly in the runner.

📚 Official docs

- Terraform: Backends
- Amazon S3 User Guide

certs.eknathalabs.com

Q8 **Terraform state file is locked — how do you resolve state lock issues?** Backend

🎙 Spoken answer · ~71 s

Locking prevents two runs from writing state at the same time. With S3 it has traditionally used DynamoDB, and newer Terraform versions support native S3 locking. A lock error shows a lock ID, who holds it, and when it was created. The first thing I check is whether the lock is legitimate: is someone's apply or a CI job still running? I never force-unlock while a run is in progress, because that can corrupt state. If I confirm the process died, such as a crashed pipeline or a closed laptop session, I run terraform force-unlock with the lock ID. If the lock is stuck in the backend, I inspect it and remove the leftover entry, but only once I'm certain. After unlocking I run a plan to confirm state is consistent, because an interrupted apply may have created resources state doesn't know about. To prevent a recurrence I add timeouts in CI, use the lock-timeout flag, serialize pipeline runs per state, and split very large states so applies are short.

Likely follow-ups

- What if the interrupted apply left resources outside state?
- How does lock-timeout differ from force-unlock?

```
terraform force-unlock <LOCK_ID>  # get lock ID from error message
```

```
# For DynamoDB: aws dynamodb delete-item --table-name tf-lock --key '{"LockID": {"S": "..."}}'
```

A stale lock occurs when a previous apply was interrupted (network failure, process kill). The lock ID is shown in the error message. Always verify no other apply is actually running before force-unlocking — concurrent applies corrupt state.

Never force-unlock if you are unsure whether another apply is in progress. Check CI/CD pipeline status and team communication first.

📚 Official docs

- Terraform: State
- Terraform: State locking

certs.eknathalabs.com

Q9

Terraform resources are getting recreated unexpectedly — what could be the cause?

State

🎙 Spoken answer · ~66 s

Terraform recreates a resource when a change touches an attribute that forces replacement. The plan marks it with minus-slash-plus and says so, so I read the plan and find which attribute is responsible. Common causes: someone changed an immutable argument like a name, subnet, or availability zone; a change to a count index or for_each key, so addresses shift, for example removing an item from the middle of a count list re-indexes everything after it; a rename or move of a resource or module without a moved block, so Terraform sees a destroy plus a create; or a provider upgrade that changed defaults or schema. The fixes are for_each with stable keys instead of count, moved blocks when refactoring, lifecycle ignore_changes for attributes managed elsewhere, and pinned provider versions. For critical resources I set prevent_destroy, and when replacement is unavoidable but downtime isn't acceptable, I use create_before_destroy. And I always review the plan in CI before anything is applied.

Likely follow-ups

- When is create_before_destroy not enough?
- How do you refactor a module without recreating its resources?

Unexpected recreation (destroy + create) is usually caused by one of these:

```
# ForceNew attribute changed — check provider docs for which attrs trigger recreation
```

```
terraform plan -out=tfplan && terraform show -json tfplan | jq '.resource_changes'
```

Common causes: changing a resource name or ID (some resources require recreation on name change), provider upgrade altering default values, computed attributes not stored in state, or `taint` left from a previous operation. Use `lifecycle { prevent_destroy = true }` to guard critical resources.

Key trick: Read the plan's "replacement triggered by" message carefully — it tells you the exact attribute driving recreation.

📚 Official docs

- Terraform: lifecycle meta-argument

certs.eknathalabs.com

Q10 **Sensitive values are visible in Terraform state — how do you secure them?** Security

🎙 Spoken answer · ~79 s

Terraform state can contain secrets in plain text, such as database passwords, keys, and generated credentials, even when the variable is marked sensitive, because sensitive only hides values in CLI output, not in the state file. So the protection has to be around the state itself. I store state in a remote backend with encryption at rest, for example S3 with SSE-KMS, and encryption in transit, with versioning turned on. Access is restricted by IAM so that only the CI role and a few administrators can read it, with access logging and audit trails enabled. A state file is as sensitive as a secret store. HCP Terraform or Terraform Enterprise provides encrypted state with role-based access if I want a managed option. Then I reduce what ends up in state: don't pass secrets as variables where I can avoid it; use secrets managers such as Vault or AWS Secrets Manager and have applications fetch secrets at runtime; let the database manage its own credentials, for example IAM authentication or managed master user passwords; and use ephemeral resources and write-only attributes, where the provider supports them, in newer Terraform versions.

Likely follow-ups

- Does sensitive = true keep a value out of the state file?
- What are ephemeral values in newer Terraform?

Terraform state contains sensitive data in plaintext (passwords, keys, secrets). This is a known limitation.

MITIGATIONS

```
sensitive = true  # mark outputs/variables as sensitive (redacted in plan output)
```

```
# S3 backend: enable server-side encryption (SSE-S3 or SSE-KMS)
```

```
# Use Vault provider or AWS Secrets Manager for secrets, not tfvars
```

Restrict state file access via IAM policies — only CI/CD roles should have S3 read access to state. Enable S3 access logging. Use Terraform Cloud/Enterprise for encrypted state with RBAC. Never commit `.tfstate` or `.tfvars` files to Git.

Production pattern: Store secrets in Vault/SSM Parameter Store and reference via data sources — they never need to be in your TF config at all.

📚 Official docs

- Terraform: State
- Terraform: Sensitive data in state

certs.eknathalabs.com

Q11 Terraform modules are not working as expected — how do you debug module issues?

Modules

🎙 Spoken answer · ~74 s

For module problems I first separate configuration errors from source and version problems. After changing a module source or version, I run terraform init, with upgrade if I need to refresh versions, because Terraform caches modules. terraform validate catches syntax and reference errors, such as missing required variables or wrong output names. The usual causes are: an incorrect source path, where local paths must be relative like dot slash modules slash vpc; version constraints that conflict between modules; a required variable not passed; an output not exposed from the module, or referenced wrongly: the correct form is module dot name dot output name; and provider configuration not passed into the module correctly. terraform console lets me evaluate expressions and inspect values interactively, and plan with debug logging traces how modules are resolved. For registry modules I pin an exact or tightly bounded version so an upstream release can't change my infrastructure. I also read the module's inputs and outputs documentation, and for my own modules I keep interfaces small, with typed and validated variables, so misuse fails early.

Likely follow-ups

- Why pin registry module versions?
- How do you pass a provider into a module?

```
terraform init -upgrade  # refresh module sources
```

```
terraform validate  # syntax and reference errors
```

```
TF_LOG=DEBUG terraform plan  # trace module resolution
```

```
terraform console  # interactively evaluate expressions
```

Common issues: incorrect module source path (relative vs absolute), version constraint conflicts, missing required variables, output not exposed correctly. Use `terraform console` to test expressions interactively. Check that module outputs are referenced as `module.<name>.<output>`.

Registry modules: Pin module versions explicitly. Never use `source = "terraform-aws-modules/vpc/aws"` without `version = "X.Y.Z"`.

📚 Official docs

- Terraform: Modules

certs.eknathalabs.com

Q12 **A resource dependency is not respected — how do you handle dependency issues?** Modules

🎙 Spoken answer · ~73 s

Terraform builds a dependency graph automatically from references between resources. If resource B uses an attribute of resource A, that's an implicit dependency, and Terraform creates A first. Problems appear when a dependency exists but isn't expressed through an attribute reference. For example an IAM policy attachment must finish before an instance that relies on its permissions starts, or a resource needs a security group rule to exist first. In those cases I add an explicit depends_on. It works at resource and module level. I use it sparingly, since depends_on forces ordering and can make plans less precise and applies slower. The better fix is usually to reference an attribute that creates the real dependency. To diagnose, terraform graph, piped to Graphviz, visualises the graph, and I read the error message from the failing create. Also remember that data sources depending on unknown values are deferred until apply. Timing issues such as eventual consistency in cloud APIs, where a role isn't visible yet, can look like dependency bugs, and sometimes need retries or a wait resource.

Likely follow-ups

- Why should depends_on be used sparingly?
- How do you deal with eventual-consistency errors after creating IAM roles?

Terraform builds a dependency graph automatically from references. When implicit dependencies are insufficient, use explicit `depends_on`.

```
resource "aws_instance" "app" { depends_on = [aws_security_group.sg] }
```

```
terraform graph | dot -Tsvg > graph.svg  # visualize dependency graph
```

Use `depends_on` sparingly — it forces sequential execution and slows applies. Prefer implicit dependencies via attribute references. For module dependencies, use `depends_on` at the module block level.

Common trap: IAM policy attachment must complete before EC2 instance launch. If the IAM role exists but the policy isn't attached yet, the instance may start without permissions. Use explicit `depends_on` here.

📚 Official docs

- Terraform: depends_on meta-argument

certs.eknathalabs.com

Q13 **Terraform variables are not being picked up correctly — how do you debug variable precedence?**

Modules

🎙 Spoken answer · ~71 s

Terraform has a defined precedence for variable values, from lowest to highest. Defaults in the variable block are lowest. Then environment variables named TF_VAR_ plus the name. Then terraform.tfvars, then terraform.tfvars.json, and then any auto.tfvars files, in alphabetical order. Highest are var and var-file flags on the command line, with later ones overriding earlier ones. When a value isn't what I expected, I find out what it resolved to. terraform console and typing var dot name shows the value in use, and the plan output shows the resulting configuration. The classic mistake is a terraform.tfvars or an auto.tfvars file sitting in the working directory that silently overrides a default, or one that's different between local and CI. Another is an environment variable set in the pipeline. To avoid surprises, in CI I always pass explicit var-file flags for the environment, avoid relying on auto-loaded files, keep environment-specific values in clearly named files, validate variables with type constraints and validation blocks, and print the resolved values, never secrets, in the plan review.

Likely follow-ups

- Which wins: a TF_VAR environment variable or terraform.tfvars?
- How do you stop a wrong tfvars file being picked up?

Terraform variable precedence (highest to lowest):

```
1. -var and -var-file CLI flags (highest priority)
```

```
2. TF_VAR_name environment variables
```

```
3. terraform.tfvars or *.auto.tfvars files
```

```
4. Default values in variable blocks (lowest priority)
```

```
terraform console -> var.<name>  # check resolved value
```

```
TF_LOG=TRACE terraform plan 2>&1 | grep variable
```

Common mistake: A `terraform.tfvars` is silently overriding a `-var-file` you intended to use. Always use explicit `-var-file` flags in CI/CD and avoid relying on auto-loaded files in pipelines.

📚 Official docs

- Terraform: Input variables (precedence)

certs.eknathalabs.com

Q14 **Terraform apply takes too long — how do you optimize performance?** Performance

🎙 Spoken answer · ~74 s

When apply is slow, I first find out why before tuning. Common causes are a monolithic root module with hundreds or thousands of resources, so every plan refreshes everything; unnecessary depends_on forcing serial execution; slow provider APIs or rate limiting; many data sources evaluated at plan time; and resources that inherently take a long time, such as databases and managed clusters. Fixes: split the configuration into smaller root modules with separate state files, by layer or team, with remote state data sources for cross-references; that cuts refresh time and blast radius. Remove needless explicit dependencies. Raise parallelism from the default of ten cautiously, as it can trigger rate limits. Use targeted plans only as a diagnostic or emergency tool, not a routine workflow, as target skips dependencies and can leave state inconsistent. Disable refresh when I know it's safe, or use refresh-only separately. Tune provider settings such as retries, avoid data sources that make expensive calls, and make sure CI caches provider plugins. I'd also look at the timings in debug logs to find the slowest resources.

Likely follow-ups

- Why is -target discouraged for routine use?
- What are the risks of raising -parallelism?

```
terraform apply -parallelism=20  # default is 10, increase with caution
```

```
terraform apply -target=module.<name>  # apply only specific module
```

```
# Split monolith into smaller state files / root modules
```

Root causes: monolithic root module with hundreds of resources, unnecessary `depends_on` causing serial execution, slow provider API calls, or large number of data sources evaluated at plan time. Break large configurations into smaller independently managed state files with remote state data sources for cross-references.

Architecture fix: Separate network, compute, and application layers into distinct Terraform roots. Use `terraform_remote_state` data source to share outputs between layers.

📚 Official docs

- Terraform CLI: apply

certs.eknathalabs.com

Q15 **Infrastructure created by Terraform is not accessible after deployment — how do you troubleshoot?** Production

🎙 Spoken answer · ~75 s

If Terraform created the infrastructure successfully but it isn't reachable, the problem is almost always in the configuration values, not in Terraform itself. Terraform did what it was told. So I troubleshoot like any connectivity problem, using Terraform to look up what was actually created. terraform output and state show give me the real IPs, DNS names, ports, and security group IDs. Then I check the network path layer by layer: security group ingress and egress rules, network ACLs, route tables and internet or NAT gateways, whether the subnet is public or private and the instance has a public address, DNS records, and VPC peering or endpoints. Then identity and access: IAM roles and instance profiles. Then the service itself: is the application listening on the expected port, and do load balancer health checks pass, with the right path and port? A very common cause is a rule that exists but is attached to the wrong resource or references the wrong CIDR. Once I find the cause, I fix it in code, not the console, so state stays consistent.

Likely follow-ups

- How do you avoid fixing things in the console and creating drift?
- What would you check first for an instance that has no internet access?

This is an application/network layer issue, not a Terraform issue per se. Terraform successfully created resources but something in the config is wrong.

```
terraform output  # verify IPs, DNS names, ports from outputs
```

```
terraform state show <resource>  # check security group rules, subnet config
```

Check: security group rules (ingress/egress), NACLs, route tables, IAM roles, VPC peering, DNS resolution, health check configuration for load balancers. The resource exists in state — the problem is in its configuration values.

Pattern: Compare the Terraform-created config with a known-working manually-created resource in the same region/VPC.

📚 Official docs

- Terraform: Resource behavior & dependency troubleshooting

certs.eknathalabs.com

Q16 **Terraform workspace is not switching properly — how do you fix workspace-related issues?** Backend

🎙 Spoken answer · ~71 s

Workspaces let one configuration have multiple state files, and problems usually come from misunderstanding them. I confirm the current workspace with workspace show, list them with workspace list, and switch explicitly with workspace select. Typical issues: the workspace was created against a different backend configuration, so it isn't where I expect; the workspace name isn't used in resource naming through terraform.workspace, so two workspaces try to create the same named resource; or the state path for non-default workspaces in S3 is an env colon prefix, and permissions or a prefix mismatch blocks access. A bigger point I'd make in an interview: workspaces share the same backend and the same code, so they are not strong environment isolation. A mistake in the wrong workspace hits production, access control can't easily separate dev from prod, and the code can't differ. For production I prefer separate directories or root modules with separate backends and credentials per environment, using shared modules for reuse. Workspaces are fine for short-lived copies, like feature branches or test environments.

Likely follow-ups

- Why aren't workspaces a good isolation boundary for prod?
- How do you structure code for dev, staging, and prod instead?

```
terraform workspace list  # show all workspaces and current
```

```
terraform workspace show  # confirm active workspace
```

```
terraform workspace select prod  # explicit switch
```

Common issues: workspace state stored in wrong backend path, `terraform.workspace` variable not used in resource naming, or workspace created in different backend config. Each workspace has its own state file at `<prefix>/env:<workspace>/<key>` in S3.

Production warning: Workspaces share the same backend and code — they are NOT true environment isolation. For prod/stage/dev isolation, use separate root modules with separate backends.

📚 Official docs

- Terraform: Workspaces

certs.eknathalabs.com

Q17 **Terraform provider version conflict occurs — how do you resolve provider dependency issues?** Modules

🎙 Spoken answer · ~72 s

Provider version conflicts occur when the root module and child modules declare incompatible version constraints for the same provider, or when different team members and CI resolve different versions. I diagnose with terraform providers, which shows requirements from every module, and init with upgrade to resolve to the newest versions that satisfy all constraints. Then I resolve the conflict by adjusting constraints: modules should declare minimum versions with required_providers, while the root module decides the final version, using the pessimistic operator like tilde arrow five to allow minor updates but not major ones. The most important practice is committing the dependency lock file, .terraform.lock.hcl, to Git, so everyone and CI use exactly the same provider versions and checksums. I generate hashes for the platforms I use, such as linux amd64 for CI and darwin arm64 for laptops, with providers lock, otherwise init fails on another platform. Upgrades are deliberate: bump in a branch, review the plan for changes, and read the provider changelog for breaking changes. Reusable modules should avoid over-restrictive constraints.

Likely follow-ups

- Why should the lock file be committed?
- What does the pessimistic constraint operator mean?

```
terraform providers  # show provider requirements from all modules
```

```
terraform init -upgrade  # upgrade to latest compatible versions
```

```
# In required_providers: version = "~> 5.0"  (allow patch updates only)
```

```
terraform providers lock -platform=linux_amd64  # lock for CI platform
```

Conflicts arise when a root module and a child module require incompatible provider versions. Use `terraform providers` to identify the conflict, then adjust version constraints. Commit `.terraform.lock.hcl` to Git to pin provider versions across the team.

Key practice: Always commit `.terraform.lock.hcl`. Run `terraform init` in CI — never skip the lock file.

📚 Official docs

- Terraform: Provider requirements
- Terraform: depends_on meta-argument

certs.eknathalabs.com

Q18 **Terraform apply fails due to API rate limits — how do you handle throttling issues?** Production

🎙 Spoken answer · ~74 s

When apply fails because of API rate limits, it's usually a large apply creating many resources in parallel against a throttled cloud API. First I confirm in the error that it's throttling, such as a rate exceeded or too many requests response. Short-term fixes: reduce parallelism, for example set it to five instead of the default ten; configure the provider's retry settings, such as max_retries for AWS, which uses exponential backoff; and re-run apply, since Terraform will continue from what succeeded. The structural fix is architectural. Break a monolithic configuration into smaller modules or states applied separately or sequentially, which reduces the number of calls at once. Reduce unnecessary API calls: avoid data sources that make expensive lookups on every plan, and lower refresh load. Stagger deployments across accounts and regions, rather than running every pipeline at the same time. If limits are consistently too low for legitimate use, I can request a quota increase from the cloud provider. Splitting also gives smaller blast radius and faster plans, so rate limits are a good prompt to refactor.

Likely follow-ups

- What does exponential backoff do for throttling?
- How does splitting state reduce rate limiting?

```
terraform apply -parallelism=5  # reduce concurrent API calls
```

```
# Add retry logic at provider level (some providers support max_retries)
```

AWS, GCP, and Azure all have API rate limits. Hitting them during large applies is common. Reduce parallelism, stagger deployments, split into smaller modules, or add exponential backoff via retry configuration in the provider block. For AWS: `provider "aws" { max_retries = 25 }`.

Architecture fix: Breaking a monolithic config into smaller modules deployed sequentially is both a rate limit mitigation and a blast-radius reduction strategy.

📚 Official docs

- RFC 6585: HTTP 429 Too Many Requests
- Terraform CLI: apply
- Microsoft: Cloud design pattern – Throttling

certs.eknathalabs.com

Q19 **Terraform cannot authenticate to cloud provider — how do you debug authentication failures?**

Backend

🎙 Spoken answer · ~73 s

When Terraform can't authenticate, I find out which identity it's actually using and compare it with what I expect. For AWS that's aws sts get-caller-identity; for GCP I check application default credentials; for Azure az account show. Then I follow the provider's credential chain, in order: explicit provider block settings, environment variables, shared credentials files and profiles, and then instance profiles, IRSA in EKS, or the metadata server. A common cause is an environment variable overriding a profile, expired temporary credentials or session tokens, a missing role assumption, or a mismatched region or project. In CI, I check that the pipeline assumed the right role, through OIDC federation, and that the role's trust policy allows this repository and branch, and has the needed permissions. For GCP I prefer Workload Identity Federation over downloaded service account keys, and on Azure managed identities or federated credentials. TF_LOG at debug level shows the authentication calls, but I'm careful that logs may expose sensitive data. Good practice is to avoid long-lived keys entirely, using short-lived credentials from federation.

Likely follow-ups

- Why prefer OIDC federation to stored access keys in CI?
- In what order does the AWS provider look for credentials?

```
aws sts get-caller-identity  # verify AWS credentials
```

```
gcloud auth application-default print-access-token  # GCP ADC check
```

```
TF_LOG=DEBUG terraform plan 2>&1 | grep -i auth
```

Check credential chain: environment variables → shared credentials file → IAM instance profile → IRSA (in EKS). In CI/CD, ensure the pipeline has assumed the correct role with sufficient permissions. For GCP, use Workload Identity Federation instead of service account keys.

GCP context: For GCP, Application Default Credentials (ADC) flow — ensure GOOGLE_APPLICATION_CREDENTIALS or Workload Identity is configured. Never use service account keys in CI.

📚 Official docs

- AWS provider: authentication and configuration

certs.eknathalabs.com

Q20 Terraform plan works locally but fails in CI/CD pipeline — how do you troubleshoot?

CI/CD

🎙 Spoken answer · ~76 s

When plan works locally but fails in CI, the cause is almost always an environment difference. I compare systematically. Terraform binary version: pin it with a version file or tfenv so CI and laptops match. Provider versions: the lock file must be committed, otherwise CI resolves different providers. Authentication: the identity in CI may have different permissions than mine, or credentials may not be set. Environment variables and variable files: TF_VAR values or tfvars files present locally but missing in CI, or auto-loaded files differing. Working directory and workspace: running in the wrong directory or workspace. Network: CI runners may not reach the backend, providers registry, private modules, or the cloud APIs, due to firewalls or private subnets. Also file paths and line endings, and modules sourced from private Git needing credentials. I capture the debug log in CI, compare the first failing step with my local run, and reproduce locally by using the same container image and variables the pipeline uses. The long-term fix is to run Terraform only from CI, in a pinned container, so local and CI never diverge.

Likely follow-ups

- How would you make local runs match CI exactly?
- What network issues commonly break Terraform in CI?

Environment differences between local and CI are the root cause 95% of the time.

```
# Check: Terraform version mismatch (use tfenv or .terraform-version file)
```

```
# Check: Provider versions (lock file not committed)
```

```
# Check: Environment variables (AWS creds, TF_VAR_*, workspace)
```

```
TF_LOG=DEBUG terraform plan > /tmp/ci-debug.log 2>&1
```

Common differences: Terraform binary version, provider lock file missing/different, missing environment variables for authentication, different working directory, or network access restrictions (S3 backend, provider API endpoints) from the CI runner.

Fix: Pin Terraform version in CI with a `.terraform-version` file and use `tfenv`. Commit `.terraform.lock.hcl`. Mirror the exact CI environment locally with Docker.

📚 Official docs

- Terraform CLI: plan

certs.eknathalabs.com

Q21 **Resources are manually modified outside Terraform — how do you detect and fix configuration drift?**

Production

🎙 Spoken answer · ~75 s

Drift from manual changes is inevitable in larger organisations, so I need both detection and a policy for what to do. To detect it, I run terraform plan with refresh-only to preview what changed outside Terraform without altering infrastructure, and I schedule regular plans in CI that fail or alert when the plan isn't empty. Tools like driftctl or the cloud provider's own config services can also find unmanaged resources, not just drifted ones. To fix it, I decide per case whether to reconcile to code or absorb the change. If the manual change was wrong, I apply Terraform to put it back. If it was legitimate, such as an emergency change, I update the code to match and the plan goes clean, or accept the new values into state. Ignoring is possible with lifecycle ignore_changes for attributes legitimately managed elsewhere. Prevention matters most: restrict console write access in production with IAM, service control policies, or organisation policies, give people read-only roles by default with a break-glass process, and make Terraform changes easy enough that nobody needs the console.

Likely follow-ups

- What's the difference between detecting drift and unmanaged resources?
- How would you handle an emergency console change during an incident?

```
terraform plan -refresh-only  # preview drift without applying changes
```

```
# Schedule: run terraform plan in CI, fail build on non-empty plan
```

```
# Tool: driftctl scan  # open-source drift detection
```

Drift is inevitable in large orgs. The strategic response: decide whether to reconcile to IaC or absorb the change into config. Use `terraform refresh` to absorb, or `terraform apply` to revert. Prevent future drift with SCPs (AWS), Organization Policies (GCP), or Azure Policy to restrict console-level changes in production.

Process fix: Require all changes go through IaC PR process. Use Atlantis to enforce this — no manual console access in prod.

📚 Official docs

- Terraform CLI: plan
- Infrastructure drift (Terraform: plan -refresh-only)

certs.eknathalabs.com

Q22 **Terraform destroy fails due to dependencies — how do you resolve it safely?** Production

🎙 Spoken answer · ~72 s

Destroy failures usually happen because something outside Terraform depends on a Terraform-managed resource, for example an S3 bucket that still contains objects, a VPC with network interfaces created by another service, a security group still attached to something, or resources with deletion protection. I start by reading the error, which normally names the blocking dependency. Then I remove the blocker safely: empty the bucket, detach or delete the manual resource, or turn off deletion protection through code and apply, and retry destroy. If one resource is genuinely stuck and I want Terraform to forget it rather than delete it, terraform state rm detaches it from state, but then it's orphaned in the cloud and I must clean it up manually, so I use that with care. Targeted destroy of dependents first can help untangle ordering, but target should be an exception. In production, I never force-delete anything without confirmation: I review the destroy plan carefully, check backups for stateful resources, and use prevent_destroy on critical resources so accidents fail at plan time.

Likely follow-ups

- When is terraform state rm the right tool?
- How does prevent_destroy protect critical resources?

```
terraform destroy -target=<dependent_resource>  # destroy dependents first
```

```
terraform state rm <resource>  # remove from state without destroying
```

Dependency-related destroy failures happen when resources outside Terraform depend on Terraform-managed resources (e.g., an S3 bucket with objects, a VPC with manually-created ENIs). Manually remove the blocking dependency, then retry destroy. Or use `terraform state rm` to detach resources from state before destroying the parent.

Never force-delete production resources without confirming with stakeholders. Destroy is irreversible.

📚 Official docs

- Terraform CLI: destroy

certs.eknathalabs.com

Q23 Remote state corruption occurs — how do you recover state safely?

State

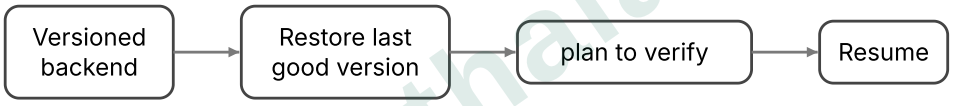
🎙 Spoken answer · ~70 s

If remote state is corrupted, I act calmly and carefully, because mistakes can orphan or destroy real resources. First, stop everything: lock or take the backend offline so no one runs apply against bad state. Second, preserve the current corrupted state as evidence. Third, restore a known-good version. With S3 versioning enabled, which is non-negotiable for production state, I list object versions and retrieve the last good one; HCP Terraform keeps state versions with a rollback option. Fourth, validate the restored state with terraform show and then run plan, without applying, to confirm that it shows no unintended changes. Differences between restored state and reality show up as a plan: resources created after the backup would appear as unmanaged, and I would import them. Fifth, investigate the cause: concurrent writes without locking, a manual edit, a failed upload, or a tool bug. Prevention is layered: versioning, optional MFA delete, locking, restricted write access, and never editing state by hand. I'd also keep periodic independent backups of state.

Likely follow-ups

- What would the plan look like if the restored state is older than reality?
- How do you protect the state bucket itself from deletion?

🗺 State recovery




```
# 1. Immediately lock or take the backend offline
```

```
# 2. Restore from S3 versioned backup
```

```
aws s3api get-object --bucket tf-state --key prod.tfstate --version-id <id>
restored.tfstate
```

```
# 3. Validate restored state
```

```
terraform show restored.tfstate
```

```
# 4. Run plan before apply to confirm no unintended changes
```

Prevention: S3 versioning is non-negotiable for production state backends. Enable MFA delete for extra protection. Terraform Cloud provides automatic state versioning with rollback UI.

📚 Official docs

- Terraform: State
- Terraform: Backends

certs.eknathalabs.com

Q24 **Multiple environments (dev, stage, prod) need isolation — how do you design Terraform structure?** Modules

🎙 Spoken answer · ~70 s

The goal is isolated state and a small blast radius per environment. I prefer a separate state per environment, and often per component, rather than workspaces alone, because workspaces share the same backend configuration and code, which makes it easy to apply to the wrong environment. My structure is reusable, versioned modules and thin environment directories for dev, staging, and prod, each with its own backend key and variables, calling the same modules at pinned versions. Separate cloud accounts or subscriptions per environment, with separate credentials and roles, give real isolation: the dev pipeline simply can't touch prod. Production changes go through pull request plans, required approvals, and a protected pipeline, while dev can be looser. I also split state by layer, such as networking, platform, and applications, so a bad apply in one doesn't risk everything. Promotion works by bumping module versions through the environments. The trade-off is some duplication in the environment folders, but that explicitness is a feature, and tools like Terragrunt can reduce it.

Likely follow-ups

- When are workspaces actually appropriate?
- How do you stop a pipeline applying to the wrong environment?

The recommended pattern is separate root modules per environment, each with their own backend state.

```
infra/ ├── modules/ │   ├── vpc/ │   └── eks/ ├── environments/ │   ├── dev/ │   ├── stage/ │   └── prod/
```

Each environment directory has its own `backend.tf`, `main.tf`, and `terraform.tfvars`. Shared infrastructure is defined in reusable modules. This gives complete blast-radius isolation — a broken plan in dev cannot touch prod state. Workspaces alone are insufficient for true environment isolation.

Alternative: Terragrunt for DRY environment configs — it generates backend configs dynamically and reduces boilerplate across environments.

📚 Start here (official docs)

- Terraform documentation

certs.eknathalabs.com

Q25 **Terraform outputs are not available after deployment — how do you debug?** Production

🎙 Spoken answer · ~76 s

If an output isn't available, I check where in the chain it breaks. terraform output lists the outputs in the current root module's state, and output with a name prints one. Common causes: the output is defined inside a child module but not re-exported from the root module, since only root module outputs are visible to the CLI and to other states; the value depends on a computed attribute that hasn't been applied yet, so the output appears only after apply; the state hasn't been refreshed or the apply didn't complete; or the output block is in a file that isn't being loaded, or in the wrong directory. For cross-state use, other configurations read it through the terraform_remote_state data source, and that fails if the output name is wrong, the backend path is wrong, or the reader lacks access to the state. terraform state show on the resource confirms the attribute exists. A note on secrets: outputs holding sensitive data should be marked sensitive, which hides them from CLI output, but they're still stored in state in plain text, so protect the state.

Likely follow-ups

- Why can't the CLI see an output from a child module?
- How does another configuration consume an output?

```
terraform output  # list all outputs in current state
```

```
terraform output <name>  # get specific output value
```

```
terraform state show <resource>  # verify attribute exists in state
```

Common causes: output defined in a module but not re-exported in root module, resource not yet applied (output depends on computed value), or state file not refreshed. For cross-state outputs, verify the `terraform_remote_state` data source path and backend access permissions.

Tip: Mark outputs as `sensitive = true` if they contain secrets — they are redacted in CLI but still accessible via `terraform output -json` for pipeline use.

📚 Official docs

- Terraform: Output values
- Terraform: Resource behavior & dependency troubleshooting

certs.eknathalabs.com

Q26 **Infrastructure deployment fails due to incorrect IAM permissions — how do you troubleshoot?** Production

🎙 Spoken answer · ~76 s

For IAM permission failures during apply, the error message usually names the exact action that was denied and the resource, so I start there. Debug logging can show the failing API call. To confirm, I can use the policy simulator, such as aws iam simulate-principal-policy for the role and action. Remember it might not be a missing allow: an explicit deny, a permission boundary, a service control policy, a resource policy, or a KMS key policy can block it as well. To build the right policy, one approach is to start with a broad managed policy in a sandbox, run the apply, capture the actions actually used, from CloudTrail logs, and use IAM Access Analyzer to generate a least-privilege policy from that activity, then apply the scoped-down policy and test again. Another is to iterate on AccessDenied errors. For production pipelines, I keep separate roles for plan and apply, with plan read-only, and scope apply to the resources the configuration manages. On GCP the equivalent is checking the project IAM policy and the service account's roles, and on Azure the role assignments.

Likely follow-ups

- Why use separate roles for plan and apply?
- What can block an action even when the IAM policy allows it?

```
TF_LOG=DEBUG terraform apply 2>&1 | grep -i "access denied|not authorized"
```

```
aws iam simulate-principal-policy --policy-source-arn <role_arn> --action-names <action>
```

The error message usually contains the exact missing permission. Build the IAM policy incrementally: start with a broad managed policy, run apply, capture all AccessDenied errors, then scope down to least privilege. Use AWS IAM Access Analyzer to generate least-privilege policies from CloudTrail logs.

GCP equivalent: `gcloud projects get-iam-policy` and Policy Analyzer. For a GCP-focused role, start with roles/editor scoped to specific project, then tighten.

📚 Official docs

- Terraform: Resource behavior & dependency troubleshooting

certs.eknathalabs.com

Q27 Terraform cannot find required modules — how do you fix module source issues?

Modules

🎙 Spoken answer · ~76 s

When Terraform can't find a module, I first remember that module sources are resolved at init, so any change to a source or version needs init again, with upgrade to force re-download. Then I check the source type. A local path must start with dot slash or dot dot slash, for example dot slash modules slash vpc, and be relative to the calling module. Registry modules need network access to the registry, or a private registry mirror, and credentials for private registries configured in the CLI configuration file or environment. Git sources need working SSH keys or HTTPS tokens in the environment where Terraform runs, and the ref or tag must exist; for subdirectories use the double slash syntax. S3 and GCS sources need access to the bucket. In CI, the runner needs those credentials and network path, and I make sure the module cache is not stale. Also check for typos in the module name, version constraints that match nothing, and a subdirectory path that doesn't contain Terraform files. For reliability I pin module versions, and cache or vendor critical modules.

Likely follow-ups

- What does the double slash mean in a module source?
- How do you authenticate to a private module registry in CI?

```
terraform init  # always required after changing module sources
```

```
# For private registry: configure credentials in .terraformrc
```

```
# For Git sources: ensure SSH key or token is configured
```

```
terraform init -get=true -upgrade  # force re-download modules
```

Module sources can be local paths, Terraform Registry, GitHub, S3, or GCS. Local paths must be relative (`./modules/vpc`). Registry modules need internet access or a private registry mirror. Git sources need SSH keys or HTTPS tokens configured in the environment.

CI/CD: Use `-get=true` in `terraform init` in pipelines. Cache the `.terraform` directory between pipeline runs to speed up init.

📚 Official docs

- Terraform: Modules

certs.eknathalabs.com

Q28 Changes are applied to the wrong environment — how do you prevent such mistakes?

CI/CD

🎙 Spoken answer · ~74 s

Applying to the wrong environment is a serious risk, so prevention should be architectural rather than relying on people being careful. The strongest control is separate state backends, and ideally separate cloud accounts or projects with separate credentials, for each environment. Then an apply in the wrong place simply can't touch production, because the credentials or backend don't permit it. In CI I require an explicit var-file for the environment and add a verification step that checks that the target account, region, or workspace matches the expected environment before plan. Production applies need protected branches, required approvers, and a manual approval gate, using a tool such as Atlantis, HCP Terraform, or the pipeline's environment protections. I also tag every resource with an environment tag, and show the account and environment prominently in plan output. I avoid long-lived admin credentials on laptops, so people can't accidentally run against production, and use read-only roles by default. With workspaces, I add safeguards, but I still prefer separate directories or backends because workspaces make it too easy to select the wrong one.

Likely follow-ups

- Why are separate accounts safer than separate workspaces?
- What checks would you add to a CI pipeline to confirm the target environment?

```
# Use separate backend configs — accidentally applying to wrong backend is immediately obvious
```

```
# Require explicit -var-file=environments/prod.tfvars in CI
```

```
# Atlantis: restrict prod applies to protected branches + required approvers
```

Prevention is architectural: separate state backends per environment eliminate cross-environment contamination entirely. Add workspace/environment verification in CI with an explicit check step. Require manual approval gates before production applies. Tag all resources with `environment = terraform.workspace` for auditability.

Process control: Production applies should require 2-person approval, be logged, and be triggered only from protected branches. Never allow direct terraform apply from developer laptops to prod.

📚 Official docs

- HCP Terraform: Recommended practices

certs.eknathalabs.com

Q29 **Terraform rollback is required after a failed deployment — how do you handle rollback strategy?** Production

🎙 Spoken answer · ~72 s

Terraform has no built-in rollback command, so I plan for rollback deliberately. There are a few approaches. The simplest is reverting the Git commit and applying the earlier code; Terraform computes what's needed to move back. Another is restoring a previous version of the state file from backend versioning, and then applying, though that's riskier because state must match reality, so I use it only in special cases. Targeted apply can do a selective rollback of a specific resource. But for stateful resources, such as databases and storage, a rollback may be impossible or dangerous: data can't be un-migrated, and recreating loses data. So often the real strategy is forward-fix: fix the problem in code and apply forward. For stateless, immutable infrastructure, I use blue-green or create-before-destroy, so the old version still exists to switch back to. I reduce risk beforehand: small, reviewed changes, plan reviewed before apply, prevent_destroy on critical resources, backups and snapshots before risky changes, and staged rollout through environments. And I consider the rollback path before applying anything risky.

Likely follow-ups

- Why is rolling back a database change harder than rolling back compute?
- How does create_before_destroy help?

Terraform does not have a built-in rollback command. Rollback must be planned explicitly.

```
# Option 1: Revert git commit, run terraform apply (previous code state)
```

```
# Option 2: Restore previous tfstate from S3 versioning, then apply
```

```
terraform apply -target=<specific_resource>  # selective rollback
```

For stateful resources (databases, storage), rollback may be impossible or dangerous. The real strategy is forward-fix: fix the issue in code and apply forward. For immutable infrastructure (EC2, containers), blue-green or canary patterns allow instant rollback at the infrastructure level without Terraform state changes.

Design principle: Build rollback into your infrastructure design (blue-green, feature flags) rather than relying on Terraform to roll back. Terraform manages desired state, not deployment history.

📚 Official docs

- Kubernetes: Deployments

certs.eknathalabs.com

Q30 **How do you troubleshoot a complete infrastructure deployment failure using Terraform?** Production

🎙 Spoken answer · ~74 s

A complete deployment failure needs a structured, incident-style approach rather than random retries. First, validate: terraform validate removes syntax and reference errors, and fmt and init confirm modules and providers load. Second, understand the current situation: terraform state list shows what exists, and plan shows what's still to do. Third, isolate: find the first failing resource, since later errors are often cascading, and a targeted plan can focus on it as a diagnostic. Fourth, categorise the failure: configuration, permission, quota or capacity, dependency or ordering, an API or provider issue, or a conflicting existing resource. The error text and debug logs usually tell me. Fifth, fix the root cause, not the symptom: raise the quota, add the permission, correct the configuration, or pin a different provider version. Sixth, apply incrementally and verify, re-running until the plan is clean, and then confirming the infrastructure actually works. I also check the state lock and that nothing is half-created or untracked. Afterwards, I write up what happened and add checks, like policy tests or quota monitoring, to catch it earlier.

Likely follow-ups

- How do you tell the root-cause error from cascading errors?
- Which failures are quota related and how do you spot them?

A complete failure requires a structured incident response approach.

TRIAGE SEQUENCE

```
TF_LOG=DEBUG terraform plan > debug.log 2>&1  # full trace
```

```
terraform validate  # eliminate config syntax errors first
```

```
terraform state list  # what exists vs what should exist
```

```
terraform plan -target=<first_failing_resource>  # isolate the failure
```

Systematic approach: (1) isolate which resource is failing, (2) identify whether it's a config, permission, quota, or API issue, (3) fix the root cause — not the symptom, (4) apply incrementally using `-target`, (5) verify full plan is clean before full apply.

Post-incident: Document root cause, add a plan validation step to CI, add resource-specific tests using Terratest, and improve IAM policies proactively. Share the runbook with the team.

📚 Official docs

- Terraform: Resource behavior & dependency troubleshooting

certs.eknathalabs.com

Q31 Explain terraform state mv, rm, and import — when do you use each in production?

State

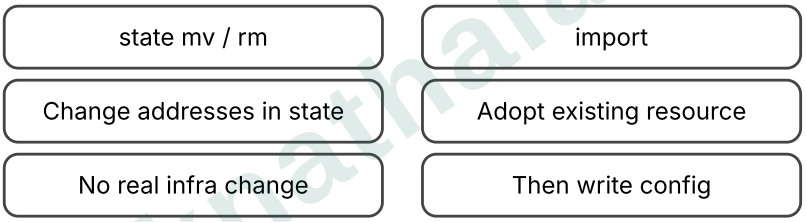
🎙 Spoken answer · ~72 s

These are the three ways I change the relationship between code and state without changing real infrastructure. state mv changes the address of something in state. I use it when refactoring, for example renaming a resource or moving it into or out of a module, so Terraform doesn't destroy and recreate it. state rm removes a resource from state without destroying it, useful when I want Terraform to stop managing something or to hand it to another state. import goes the other way: it brings an existing real resource under management by attaching it to a resource address in state, and you still need the matching configuration. In production I take a state backup first, run a plan afterwards expecting no changes, and avoid doing any of this ad hoc. Modern Terraform offers moved and import blocks, which do the same job declaratively in code, are reviewable in pull requests, and run through the normal plan and apply, so I prefer them over CLI commands because they're auditable and repeatable across environments.

Likely follow-ups

- What can go wrong with state rm?
- How do moved blocks behave across modules you consume but do not own?

🗺 State commands



These are surgical state operations for reconciling state with reality without destroying/recreating resources. Each addresses a different mismatch.

THE THREE OPERATIONS

```
terraform state mv aws_instance.old aws_instance.new   # rename/move in state, no infra change
```

```
terraform state mv 'module.a.x' 'module.b.x'           # move a resource between modules
```

```
terraform state rm aws_instance.legacy                 # forget a resource (does NOT destroy it)
```

```
terraform import aws_instance.web i-0abc123            # bring an existing resource under management
```

state mv is for refactors: you renamed a resource or moved it into a module, and want Terraform to keep managing the same real resource instead of destroying and recreating it. **state rm** tells Terraform to stop tracking a resource without touching the real infrastructure — useful when handing ownership to another stack or removing something you will manage manually. **import** is the reverse: adopt an existing resource into state.

certs.eknathalabs.com

Modern note: for refactors, prefer `moved {}` blocks (Terraform 1.1+) over imperative `state mv` — they are declarative, code-reviewed, and run automatically during apply, so the refactor is reproducible for everyone rather than a manual one-off.

📚 **Official docs**

- Terraform: State
- Terraform: Refactoring (moved blocks)
- Terraform CLI: state mv

Q32 **What are moved {} and import {} blocks, and why are they better than CLI commands?** State

🎙 **Spoken answer · ~76 s**

Moved and import blocks are declarative ways to perform state operations inside the configuration, rather than as manual CLI commands. A moved block, available since Terraform one point one, says that a resource formerly at one address is now at another, for example when I rename a resource or move it into a module. Terraform then updates state during the next apply instead of destroying and recreating it. An import block, available since one point five, says to import an existing real resource with a given ID into a given address, and with plan -generate-config-out Terraform can even scaffold the configuration for it. They are better than the CLI commands for several reasons. They are in code, so they go through pull request review and the plan shows exactly what will happen. Every teammate and every CI run applies the same change automatically, rather than somebody having to remember to run state mv on every copy of state. They're repeatable across environments, and self-documenting. Once applied, I can leave moved blocks for a while to help consumers of a module, then remove them.

Likely follow-ups

- What happens to consumers of a module if you rename a resource without a moved block?
- How does generate-config-out help with imports?

These declarative blocks (Terraform 1.1+ for moved, 1.5+ for import) encode state operations in code instead of as manual CLI steps, making refactors and adoptions reproducible and reviewable.

DECLARATIVE REFACTOR & IMPORT

```
moved {
  from = aws_instance.old_name
  to   = aws_instance.new_name
}
import {
  to = aws_instance.web
  id = "i-0abc123def"
```

certs.eknathalabs.com

```
}
```

```
terraform plan -generate-config-out=generated.tf   # scaffold config for imported resource
```

A `moved` block means every teammate and CI run applies the rename automatically — nobody has to remember to run `state mv` on their machine. An `import` block makes adopting existing infra part of the plan/apply cycle, and `-generate-config-out` can scaffold the resource HCL for you. Both turn fragile, undocumented CLI surgery into version-controlled, auditable changes.

Why it matters: imperative `state mv` only runs where someone types it — a classic source of "works on my machine, drift on yours." Declarative blocks make state changes a reviewed commit, which is the IaC philosophy applied to state itself.

📚 **Official docs**

- Terraform: Refactoring (moved blocks)

Q33 **Compare S3+DynamoDB, Terraform Cloud, and Consul as state backends. How do you choose?** Backend

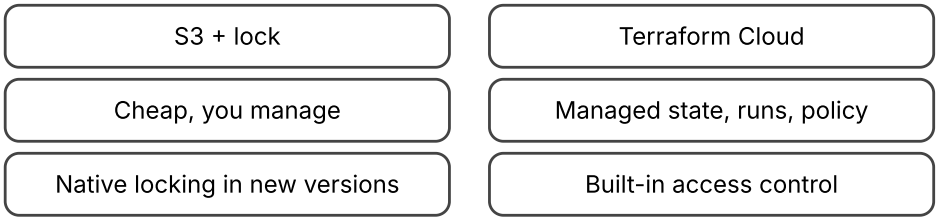
🎙 **Spoken answer · ~73 s**

The choice is about operational ownership, locking, collaboration features, and cost. S3 with DynamoDB for locking, or the native S3 locking in newer versions, is the common do-it-yourself option: it is cheap and AWS-native, I own the configuration and security, and it's simple and widely understood, but I must build the surrounding workflow myself. HCP Terraform or Terraform Enterprise is a managed service that provides state storage, locking, versioning, run orchestration, remote execution, access control, and policy checks with Sentinel or OPA, at a licensing cost. Consul is a key-value store that supports state with locking; it makes sense if I already run Consul, but otherwise it's extra operational burden. There are also cloud-native equivalents with built-in locking, like GCS and Azure Blob storage. My default for a small or medium team on AWS is S3 with locking, encryption, and versioning, plus CI-based runs. I'd choose HCP Terraform or a similar platform when I need governance, policy, many teams, and a managed workflow. Whatever I choose, I need encryption, versioning, locking, and restricted access.

Likely follow-ups

- What features does a managed platform add beyond state storage?
- How do you secure the S3 bucket that holds state?

🗺 **State backends**



cers.eknathalabs.com

The choice trades off operational ownership, locking, collaboration features, and cost. All provide remote state; they differ in what else they give you.

BACKEND COMPARISON

```
S3 + DynamoDB : you own it; S3 stores state, DynamoDB provides locking; cheap, AWS-native
```

```
Terraform Cloud/Enterprise : managed state + locking + run orchestration + policy
(Sentinel/OPA)
```

```
Consul        : KV store with locking; good if you already run Consul; more ops overhead
```

```
GCS / azurerm : cloud-native equivalents with built-in locking
```

S3+DynamoDB is the pragmatic default on AWS — cheap, simple, you control encryption and access via IAM, locking via DynamoDB. **Terraform Cloud** adds the things teams eventually want: remote runs, a run queue, VCS integration, policy-as-code, and a state UI — at a cost and with vendor coupling. **Consul** only makes sense if you already operate it. Note: as of Terraform 1.10+, S3 backend supports native locking via a lockfile, reducing the need for DynamoDB.

Decision framing: start with cloud-native (S3+DynamoDB/GCS) for simplicity and cost; graduate to Terraform Cloud/Enterprise when you need run orchestration, policy enforcement, and audit at team scale. State the trade-off (ownership/cost vs managed features) rather than naming a single winner.

📚 **Official docs**

- Terraform: State
- Terraform: Backends
- HCP Terraform documentation

Q34 **How do you structure state files across environments and teams to limit blast radius?** State

🎙 **Spoken answer · ~72 s**

A single monolithic state is a blast-radius and contention problem: any apply can affect everything, locks block everyone, and plans get slow. The remedy is splitting state along boundaries of change frequency and ownership. Common strategies are: per environment, with dev, staging, and prod in separate states and ideally separate accounts; per layer, such as networking, data, compute, and application, because networking changes rarely and apps change often; and per team or service, so each owning team manages its own state and backend. Cross-state references use the terraform_remote_state data source, which reads outputs read-only, or better, looking up resources via data sources or a parameter store, which avoids tight coupling to another team's state. The costs are more states to manage, ordering between layers, and the need for good conventions and automation. To manage that, I use consistent naming of state keys, shared modules, and an orchestration tool such as Terragrunt or a pipeline that applies layers in order. I also restrict permissions so each team can modify only its own state.

Likely follow-ups

- What are the downsides of terraform_remote_state coupling?
- How do you order applies across layered states?

certs.eknathalabs.com

Monolithic state is a blast-radius and contention problem: one apply can affect everything and locks block everyone. The fix is splitting state along boundaries of change and ownership.

STATE SEGMENTATION STRATEGIES

```
Per environment : separate state for dev / staging / prod (separate keys/backends)
```

```
Per layer       : networking / data / compute / app in separate states
```

```
Per team/service: each owning team has its own state + backend
```

```
# Cross-state references via remote state data source (read-only):
```

```
data "terraform_remote_state" "network" { backend = "s3" config = { ... } }
```

Split state by **rate of change and ownership**: slow-moving foundational layers (networking, IAM) in their own state, fast-moving app layers in another. This limits blast radius (an app apply cannot accidentally touch networking), reduces lock contention, and speeds up plans. Layers share data through the `terraform_remote_state` data source (read-only outputs) or, better, through provider data sources / SSM parameters to avoid tight state coupling.

Trade-off: more state files mean more orchestration (ordering between layers) and more remote-state plumbing. Over-splitting creates a dependency web. Split where ownership or change-rate genuinely differs — not arbitrarily.

📚 **Official docs**

- Terraform: State
- AWS: Disaster recovery & availability concepts (Fault isolation boundaries)
- AWS Builders' Library: Workload isolation using shuffle sharding

---

Q35 **A state file became corrupted or has stale entries. How do you safely repair it?** Backend

🎙 **Spoken answer · ~80 s**

State repair is delicate, because a mistake can orphan or destroy real resources, so the principles are back up first and make the smallest targeted change. Step one: terraform state pull to a backup file, and if the backend is versioned, note the earlier good versions. Step two: understand what is wrong, such as a stale entry for a resource that no longer exists, a duplicate, or a resource that exists but isn't tracked. Then use targeted operations. To remove a stale or duplicate entry, terraform state rm on that address, which only forgets it and doesn't delete anything. To re-attach a resource that exists in the real world but is missing from state, terraform import with the address and ID, or an import block. To rename or move, state mv or a moved block. If the state is badly damaged, restore a known-good version from backend versioning, and use state push only with great care, since it overwrites remote state. After each repair I run plan and check that it shows no unexpected create or destroy actions, and I do this under a lock, so nobody applies in the meantime.

Likely follow-ups

- What is the difference between state rm and destroying a resource?
- When would you use state push, and why is it risky?

certs.eknathalabs.com

State repair is delicate — mistakes can orphan or destroy real resources. Always back up first, then use targeted operations.

REPAIR PROCEDURE

```
terraform state pull > backup.tfstate        # ALWAYS back up first
```

```
aws s3api list-object-versions --bucket tf-state  # find a prior good version (if versioned)
```

```
terraform state rm <bad_address>             # remove a stale/duplicate entry
```

```
terraform import <address> <real_id>         # re-attach a resource that exists but is mistracked
```

```
terraform state push backup.tfstate          # restore a known-good version (last resort)
```

```
terraform plan                               # verify NO unexpected changes before applying
```

Pull and back up the current state first — every repair is reversible if you have the prior state. For stale entries, `state rm` the bad address; for resources that exist but are tracked wrong, `import` them back correctly. If corruption is broad, restore a versioned prior state with `state push`. The non-negotiable final step is `terraform plan` showing no surprising changes before any apply.

Never hand-edit the raw tfstate JSON in production unless there is genuinely no alternative — it is easy to break references and serial numbers. Prefer `state rm`/`import`/versioned restore. And confirm state versioning + locking are enabled so this is recoverable next time.

📚 **Official docs**

- Terraform: State

certs.eknathalabs.com

Q36 **Explain Terraform workspaces. When are they appropriate vs separate state files/dirs?** State

🎙 Spoken answer · ~73 s

Workspaces let one Terraform configuration keep several independent state files under the same backend, so I can switch between, say, dev, staging, and prod with terraform workspace select, and reference the current one with terraform.workspace. They're convenient but commonly misused for environment separation. The important limitation is that workspaces share the same code and the same backend and credentials, and only the state differs. So a mistake in the wrong workspace hits production, access control can't separate dev from prod, and environments can't have different code or provider versions. They're appropriate for short-lived, structurally identical copies: a feature-branch environment, a test stack, or per-tenant copies of the same thing, where I can name resources with the workspace. For real environments, especially production, I prefer separate directories or root modules, each with its own backend and credentials, ideally in separate cloud accounts, calling shared modules. That gives hard isolation, different configurations per environment, and an obvious signal about where you're applying. In an interview I'd say workspaces are a tool for copies, not a security boundary.

Likely follow-ups

- Why are workspaces not an access control boundary?
- When would you still use workspaces?

Workspaces let one configuration maintain multiple independent states (e.g. dev/staging/prod) under the same backend key prefix. They are convenient but commonly misused for environment separation.

WORKSPACE MECHANICS

```
terraform workspace new staging
```

```
terraform workspace select prod
```

```
terraform workspace list
```

```
# reference the active workspace in config:
```

```
locals { env = terraform.workspace }
```

Workspaces share the same code and backend, switching only the state. They suit **ephemeral or near-identical** variations — per-developer sandboxes, short-lived test environments. They are a poor fit for prod-vs-dev when those environments diverge (different sizes, regions, accounts) or need different access controls, because the shared code and single backend make isolation weak and a mistake can hit the wrong env.

Best practice for real environments: separate directories/state with explicit backends (or a tool like Terragrunt) gives true isolation — distinct access, distinct blast radius, divergent config. Reserve CLI workspaces for ephemeral or developer-scratch use. State this distinction; it is a common interview discriminator.

📚 Official docs

- Terraform: State
- Terraform: Workspaces

certs.eknathalabs.com

Q37 **How does Terraform state locking actually work, and how do you handle a stuck lock?** Backend

🎙 Spoken answer · ~79 s

State locking prevents two Terraform runs from writing state at the same time, which could corrupt it. The mechanism depends on the backend. With S3 and DynamoDB, Terraform writes a lock item with a LockID to the DynamoDB table before it starts an operation that can write state, and deletes it afterwards. Newer versions of the S3 backend can lock with a lock file in the bucket itself. Other backends have their own mechanism. If another run tries to start, it fails or waits with an error showing who holds the lock and since when. A stuck lock appears when a run crashes, loses its connection, or is killed, leaving the lock orphaned. Handling it: first confirm that no apply is actually running, by checking the CI job or asking the person named in the lock info. If it really is orphaned, use terraform force-unlock with the lock ID from the error. I can also inspect or delete the DynamoDB item directly. I never force-unlock casually, because breaking a live lock allows concurrent writes. To limit the problem, I run Terraform in CI with sane timeouts and cancellation handling.

Likely follow-ups

- How do you tell if a lock is stale or an apply is still running?
- What does the -lock-timeout flag do?

🗺 State locking



Locking prevents concurrent applies from corrupting state by serializing write access. The mechanism is backend-specific; a stuck lock blocks everyone until cleared.

LOCKING & UNLOCK

```
# DynamoDB: a lock item (LockID) is written before apply, deleted after
```

```
# A crashed/killed apply can leave the lock orphaned:
```

```
terraform force-unlock <LOCK_ID>            # remove a stuck lock (get ID from the error)
```

```
# Inspect the DynamoDB lock item directly if needed:
```

```
aws dynamodb get-item --table-name tf-lock --key '{"LockID":{"S":"..."}}'
```

Before a state-writing operation, Terraform acquires an exclusive lock (a DynamoDB conditional put, a Consul session, etc.). If an apply crashes or is killed, the lock can be orphaned, and the next run errors with a lock ID and who/when held it. `force-unlock` with that ID clears it.

Before force-unlocking, confirm no apply is actually running — force-unlocking an in-progress operation lets two applies run concurrently and can corrupt state. Check with the holder (the error shows who and when) first. Only force-unlock genuinely orphaned locks.

certs.eknathalabs.com

📚 **Official docs**

- Terraform: State
- Terraform: State locking
- Terraform CLI: force-unlock

Q38 **What is terraform refresh / -refresh-only, and why was refresh-during-plan behavior changed?** State

🎙 **Spoken answer · ~77 s**

Refresh is the step where Terraform reads the real attributes of the resources it manages and reconciles state with reality. The old standalone terraform refresh command updated state directly with no review, which was risky: if credentials were wrong or a resource was temporarily missing, state could be changed in a harmful way, for example dropping resources it believed were gone. So since Terraform zero point fifteen point four, the refresh-only mode was added to make drift reconciliation an explicit, reviewable step. plan with refresh-only shows the drift between real infrastructure and state without proposing configuration changes. apply with refresh-only updates state to match reality, with no infrastructure changes, after I approve. Normal plan and apply still refresh as part of their operation. And plan with refresh equals false skips the refresh, which makes plans much faster on large configurations but is risky if drift exists, so I use it deliberately, for example during quick iteration. The standalone refresh command is deprecated. The practical lesson: when I want to accept manual changes into state, I use apply refresh-only, and review what it will change.

Likely follow-ups

- Why was the standalone refresh command considered dangerous?
- When is it safe to use -refresh=false?

Refresh reconciles state with the real world by reading current resource attributes. Terraform 0.15.4+ added `-refresh-only` to make drift reconciliation an explicit, reviewable step.

REFRESH OPERATIONS

```
terraform plan -refresh-only      # show drift (real vs state) WITHOUT proposing config changes
```

```
terraform apply -refresh-only     # update state to match reality, no infra changes
```

```
terraform plan -refresh=false     # skip refresh (faster plans; risky if drift exists)
```

```
# (the standalone 'terraform refresh' command is deprecated in favor of -refresh-only)
```

Plain `terraform plan` refreshes implicitly, then diffs against config. `-refresh-only` separates concerns: it shows you what drifted in reality and lets you accept that into state without conflating it with config-driven changes. This matters because an automatic refresh can mask or surprise you with out-of-band changes; making it explicit gives a clean, reviewable "reconcile drift" step distinct from "apply my code changes."

certs.eknathalabs.com

Performance note: on large states, refresh is the slow part of plan. `-refresh=false` speeds CI plans but only safe when you trust there is no drift — pair it with periodic `-refresh-only` drift checks. Mentioning this trade-off shows operational maturity.

📚 **Official docs**

- Terraform CLI: plan

Q39 **How do you migrate state from one backend to another (e.g. local to S3) with zero downtime?** Backend

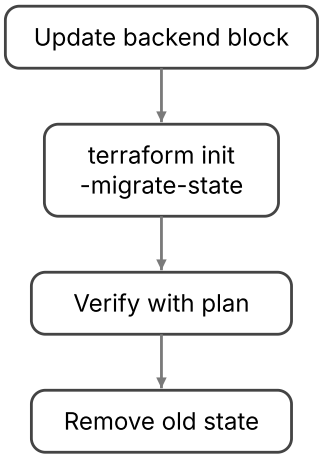
🎙 **Spoken answer · ~73 s**

Terraform automates backend migration, but I do it carefully so state is never lost. First I take a backup: terraform state pull into a file, and I make sure nobody else is running Terraform, by pausing pipelines or taking the lock. Then I change the backend block in the configuration, for example from local to S3 with the bucket, key, region, and encryption and locking settings. I run terraform init with migrate-state. Terraform detects the backend change and asks whether to copy the existing state to the new backend; I confirm. Afterwards I verify: terraform state list shows all resources, and a plan shows no changes, which proves the migrated state matches reality. I check that the old state is retained or archived until I'm confident, and that the new backend has versioning, encryption, locking, and restricted access. Then I update CI and teammates, who need to re-initialise with the new backend, and remove the old state files from developer machines. There's no infrastructure downtime, because migration only moves the state file, not the resources.

Likely follow-ups

- How do you verify the migration was successful?
- What should the new backend have enabled before you migrate?

🗺 **Backend migration**



Terraform automates backend migration when you change the backend block and re-init. The key is doing it carefully so no state is lost in transit.

MIGRATION STEPS

```
terraform state pull > pre-migration-backup.tfstate   # back up first
```

certs.eknathalabs.com

```
# 1. Add/modify the backend block in terraform { backend "s3" { ... } }
```

```
terraform init -migrate-state                          # Terraform copies state to the new
backend
```

```
#    it prompts to confirm copying existing state
```

```
terraform state list                                   # verify all resources present post-
migration
```

```
terraform plan                                         # confirm no changes proposed
```

When you change the backend configuration and run `terraform init -migrate-state`, Terraform detects the change and offers to copy the existing state to the new backend. Always pull a manual backup first. After migration, verify with `state list` and a clean `plan`. There is no real downtime because state migration does not touch live infrastructure — it only relocates the tracking data.

Coordinate the team: everyone must re-init against the new backend, and you should ensure no apply is mid-flight during migration. Commit the backend change and announce it so stale local backends do not cause split state.

> 📚 **Official docs**
>
> - Terraform: Backends
> - Terraform: S3 backend
> - Terraform: Locals

Q40 **What is sensitive data exposure in state, and how do you protect it?** State

> 🎙 **Spoken answer · ~73 s**
>
> Terraform state stores resource attributes as plain JSON, including secrets such as generated passwords, private keys, and connection strings. Anyone who can read the state can read those secrets, and marking a variable or output as sensitive only hides it from CLI output and plan logs, not from the state file. So I protect state in layers. Encrypt it at rest and in transit: S3 with SSE-KMS, GCS with customer-managed keys, or HCP Terraform's encrypted state. Restrict access with least-privilege IAM, so only the CI role and a few administrators can read the bucket and lock table, with access logging and versioning on. Treat the state bucket as a secret store, including its backups and any local copies. Reduce what gets into state: don't put secrets in variables if possible, fetch them at runtime from Vault or a secrets manager, let services manage their own credentials, use managed identity where possible, and in newer Terraform use ephemeral values and write-only attributes so secrets aren't persisted. And never commit state or tfvars containing secrets to Git.
>
> **Likely follow-ups**
>
> - Why doesn't sensitive = true protect the state file?
> - How would you audit who accessed the state bucket?

State stores resource attributes in plaintext JSON — including secrets like generated passwords, private keys, and connection strings. Anyone with state access can read them.

certs.eknathalabs.com

PROTECTION LAYERS

```
# 1. Encrypt state at rest + in transit:
#    S3 SSE-KMS, GCS CMEK, Terraform Cloud encrypted state
# 2. Restrict state access via IAM (least privilege on the bucket/table)
# 3. Mark variables/outputs sensitive to keep them out of logs:
variable "db_password" { type = string sensitive = true }
output "endpoint" { value = ... sensitive = true }
# 4. Don't generate secrets in TF at all — reference a secrets manager:
data "aws_secretsmanager_secret_version" "db" { secret_id = "prod/db" }
```

Defense in depth: encrypt state at rest (KMS) and in transit, lock down access with least-privilege IAM, and mark sensitive variables/outputs so they are redacted from CLI/CI logs (note: `sensitive` hides them from output but they are **still plaintext in state**). The strongest measure is architectural — do not have Terraform generate or hold secrets; reference a secrets manager (Vault, AWS Secrets Manager) so the secret never lands in state.

> **Critical misconception:** `sensitive = true` does NOT encrypt state — it only suppresses log output. The secret is still readable in the raw tfstate. Anyone who can read your state bucket can read every secret in it. Treat state access as equivalent to secret access.

> 📚 **Official docs**
>
> - Terraform: Sensitive data in state

certs.eknathalabs.com

Q41 **How would you design state for a multi-account, multi-region AWS organization?** Backend

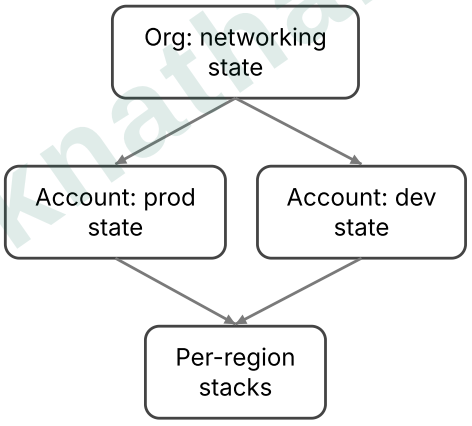
🎙 Spoken answer · ~75 s

For a multi-account, multi-region AWS organisation, state design should mirror account boundaries, blast radius, and ownership. I give each account its own state, stored in a central, locked-down state account or in each account, with a bucket per account or per environment, and cross-account access through assumed roles. Within an account I split states into layers by rate of change: identity and organisation baseline, networking, shared platform services, and then per-service application stacks. Slow, critical layers change rarely and are tightly controlled; fast layers change often and are owned by application teams. State keys follow a convention such as account, region, layer, and service dot tfstate, so they're predictable and easy to apply permissions to. For multi-region, I keep a state per region for regional resources, with a separate global layer for things like IAM and Route 53. Cross-references use outputs published to SSM Parameter Store or read via the remote state data source, with read-only access, minimising coupling. A pipeline or Terragrunt orchestrates ordering, and a landing zone approach, such as Control Tower or account factory, bootstraps new accounts consistently.

Likely follow-ups

- How do you let one team read another team's outputs safely?
- How do you bootstrap state backends for new accounts?

🗺 Multi-account state



At org scale, state design must reflect account boundaries, blast-radius isolation, and clear ownership, while enabling controlled cross-references.

DESIGN APPROACH

```
Per-account state backend (state lives in or is keyed per account)
```

```
Layered: org/identity (slow) | networking | platform | per-service app (fast)
```

```
Naming convention for state keys: <account>/<region>/<layer>/<service>.tfstate
```

```
Cross-account reads via assumed roles + terraform_remote_state or SSM/Parameter Store
```

```
Centralized backend bucket with strict IAM, or backend-per-account for isolation
```

certs.eknathalabs.com

Mirror the org structure: separate state per account and per layer, with a consistent key naming scheme so any state is findable. Foundational layers (identity, networking) change rarely and many things depend on them — isolate and protect them. Cross-account/cross-layer data flows through assumed-role remote-state reads or, preferably, a shared parameter store to avoid brittle state-to-state coupling. Access to each state is governed by least-privilege IAM matching who owns that layer.

Tooling: Terragrunt or Terraform Cloud workspaces help manage the proliferation of backends/state keys with DRY backend config and dependency ordering. Mention that the goal is isolation + clear ownership + controlled cross-references, and that over-centralizing state recreates the blast-radius problem.

📚 **Official docs**

- Terraform: Provider configuration (aliases)
- HCP Terraform: Recommended practices

Q42

Explain dependency ordering: how does Terraform build its graph and where do depends_on and lifecycle fit?

State

🎙 **Spoken answer · ~73 s**

Terraform builds a directed acyclic graph from the references in the configuration. If resource B refers to an attribute of resource A, there's an edge from A to B, so A is created first. Terraform then walks the graph, running independent branches in parallel and respecting the order for dependent ones; on destroy it walks it in reverse. Most ordering should be implicit, through references, because that expresses the real relationship. Where a dependency exists but can't be inferred, for example an IAM policy attachment that must exist before an application using its permissions starts, I add explicit depends_on. I use it sparingly because it can make plans less precise and slows applies by forcing serialisation. The lifecycle block adds further controls: create_before_destroy changes the replacement order to build the new one first, prevent_destroy blocks accidental deletion of critical resources, ignore_changes ignores attributes managed elsewhere, and replace_triggered_by forces replacement when something else changes. To debug ordering, terraform graph shows the DAG, and I look for unexpected edges or chains that serialise work that should be parallel.

Likely follow-ups

- What does create_before_destroy do to the graph order?
- When is depends_on unavoidable?

Terraform builds a directed acyclic graph (DAG) from resource references and walks it to parallelize independent work and order dependent work. Most ordering is implicit via references.

GRAPH & ORDERING CONTROLS

```
# Implicit dependency (preferred): referencing an attribute creates an edge
```

```
subnet_id = aws_subnet.main.id   # this resource now depends on the subnet
```

```
# Explicit, when there is a hidden dependency Terraform can't infer:
```

```
depends_on = [aws_iam_role_policy.app]
```

certs.eknathalabs.com

```
# Lifecycle controls ordering/replacement behavior:
```

```
lifecycle { create_before_destroy = true }
```

```
lifecycle { prevent_destroy = true }
```

```
lifecycle { ignore_changes = [tags] }
```

```
lifecycle { replace_triggered_by = [aws_instance.web.id] }
```

Prefer **implicit** dependencies — referencing `aws_subnet.main.id` automatically orders creation. Use `depends_on` only for dependencies Terraform cannot see (e.g. an IAM policy that must exist before an app starts, with no attribute reference). `lifecycle` tunes behavior: `create_before_destroy` avoids downtime on replacement, `prevent_destroy` guards critical resources, `ignore_changes` stops fighting out-of-band edits, and `replace_triggered_by` forces replacement when a related resource changes.

Interview depth: emphasize implicit-over-explicit (overusing `depends_on` serializes the graph and slows applies), and that `create_before_destroy` is the key to zero-downtime resource replacement. Visualize with `terraform graph | dot -Tpng`.

📚 **Official docs**

- Terraform: lifecycle meta-argument
- Terraform: depends_on meta-argument

Q43 **What is a partial backend configuration and why use it?** Backend

🎙 **Spoken answer · ~73 s**

A partial backend configuration means leaving some backend settings out of the code and supplying them at init time. The backend block is evaluated before variables, so it can't use variables, which makes it awkward to vary per environment. With partial configuration, the code has a minimal block, for example just the backend type, and I pass the rest with init using backend-config, either as key-value pairs or as a file such as prod.s3.tfbackend. The benefits: environment-specific settings like bucket, key, and region can differ per environment without duplicating the code or editing it; sensitive details such as credentials or access keys stay out of the repository, coming from the CI environment instead; and one codebase can safely target different backends. It also helps prevent applying to the wrong environment when the backend file is chosen explicitly by the pipeline. I'd still keep non-secret backend files in version control per environment for transparency, and have CI select the right one. Changing backend settings later requires re-running init, and migrate-state if state needs to move.

Likely follow-ups

- Why can't the backend block use variables?
- How do you keep backend credentials out of the repository?

Partial backend config omits some backend settings from code and supplies them at init time — useful for keeping environment-specific or sensitive backend details out of the repo.

certs.eknathalabs.com

PARTIAL CONFIG PATTERN

```
# In code: backend block with minimal/empty settings
```

```
terraform { backend "s3" {} }
```

```
# Supply the rest at init via a file or flags:
```

```
terraform init -backend-config=prod.s3.tfbackend
```

```
terraform init -backend-config="bucket=tf-state-prod" -backend-config="key=app.tfstate"
```

```
# prod.s3.tfbackend:  bucket=... key=... region=... dynamodb_table=...
```

Backend blocks cannot use variables, so partial configuration is the sanctioned way to parameterize them. You keep a minimal backend block in code and inject the bucket/key/region per environment at `init` time via `-backend-config` files or flags. This lets the same code target different backends (dev/staging/prod) and keeps account-specific identifiers out of version control.

Common use: CI pipelines pass the right `.tfbackend` file per environment, so one codebase initializes against the correct state per deploy target. Terragrunt automates this pattern (generating backend config) at scale.

📚 **Official docs**

- Terraform: Backends

Q44 **How do you handle a resource that must be replaced with zero downtime?** State

🎙 **Spoken answer · ~74 s**

By default, when a change forces replacement, Terraform destroys the old resource and then creates the new one, which causes an outage for anything that depends on it. The lifecycle setting create_before_destroy reverses this: Terraform builds the replacement first, waits for it to be created, repoints dependents, and only then destroys the old one. For a web instance behind a load balancer, that means the new instance is healthy and registered before the old one is removed. There are things to watch. Both resources exist at the same time, so names or unique values must not clash, which is why I use name_prefix or random suffixes. Quotas and costs briefly double. The setting propagates to dependencies, so resources that the replaced one depends on may also need it. And for stateful resources, create-before-destroy doesn't solve data migration. For forced replacement of a specific resource, I use terraform apply with the replace option, the modern replacement for taint. For true zero-downtime at the service level I combine it with load balancer health checks, autoscaling group instance refresh, or blue-green.

Likely follow-ups

- Why do names need to be unique with create_before_destroy?
- What replaced terraform taint?

By default Terraform destroys-then-creates on replacement, causing an outage. `create_before_destroy` inverts this so the new resource is healthy before the old is removed.

certs.eknathalabs.com

ZERO-DOWNTIME REPLACEMENT

```
resource "aws_instance" "web" {
  # ...
  lifecycle { create_before_destroy = true }
}
# Force a replacement explicitly when needed:
terraform apply -replace=aws_instance.web   # replaces the 'tainted' way (modern syntax)
```

`create_before_destroy = true` tells Terraform to provision the replacement first, cut over (e.g. via a load balancer or DNS that references the new resource), and only then destroy the old one — eliminating the gap. It requires that the resource can have two instances briefly (no hard uniqueness conflicts on names/IPs; handle those with naming that allows overlap). For intentional replacement, `-replace=` is the modern alternative to the deprecated `terraform taint`.

Watch for conflicts: create_before_destroy fails if the new resource collides with the old on a unique attribute (a fixed name, EIP, etc.). Design names to allow two to coexist momentarily, or the create step errors and you get neither smooth cutover nor the benefit.

📚 **Official docs**

- Terraform: lifecycle meta-argument

Q45 **Explain terraform_remote_state vs data sources vs SSM for sharing values between configurations.** Backend

🎙 **Spoken answer · ~76 s**

There are three ways for one configuration to consume values produced by another, and they differ in coupling and access. The terraform_remote_state data source reads the outputs of another state file. It's easy, but it couples me to that configuration's backend and structure, requires read access to the whole state file, which might contain secrets, and the producer must expose outputs explicitly. A provider data source reads the real thing directly from the cloud API, for example looking up a VPC by tag. It doesn't depend on another team's Terraform at all, only on the resource existing, so it's less coupled and works even if the resource wasn't created by Terraform, but lookups need unique, stable identifiers. Parameter stores like SSM Parameter Store or Secrets Manager are a third option: the producer writes values to the store, and consumers read them with data sources. That decouples completely from Terraform, supports fine-grained access control, and works for non-Terraform consumers too. My preference at scale is data sources or a parameter store for loose coupling, and remote state for simple cases inside one team.

Likely follow-ups

- What is the security downside of terraform_remote_state?
- Why might SSM parameters be better for cross-team sharing?

certs.eknathalabs.com

Three ways one Terraform config consumes values produced by another. They differ in coupling, access control, and how tightly they bind to Terraform.

THE OPTIONS

```
# 1. Remote state data source (reads another state's OUTPUTS):
```

```
data "terraform_remote_state" "net" { backend="s3" config={...} }
```

```
subnet = data.terraform_remote_state.net.outputs.subnet_id
```

```
# 2. Provider data source (reads from the cloud API directly):
```

```
data "aws_vpc" "main" { tags = { Name = "prod" } }
```

```
# 3. Parameter store / SSM (decoupled, any consumer):
```

```
data "aws_ssm_parameter" "vpc_id" { name = "/network/vpc_id" }
```

Remote state is convenient but creates tight coupling — the consumer needs read access to the producer's entire state (which contains secrets), and breaks if the producer renames an output. **Provider data sources** query the cloud directly (e.g. find a VPC by tag), decoupling from state entirely but depending on stable tags/naming. **SSM/Parameter Store** is the loosest coupling: the producer publishes values, any consumer (Terraform or not) reads them, with independent IAM.

Recommendation to state: prefer provider data sources or SSM over `terraform_remote_state` for cross-stack values — remote state forces granting full state read (a security and coupling cost). Reserve remote state for tightly-related layers you control end to end.

📚 **Official docs**

- Terraform: The terraform_remote_state data source

certs.eknathalabs.com

Q46 **What happens during terraform init, plan, and apply internally — walk through the lifecycle.** State

🎙 Spoken answer · ~72 s

Each phase has a distinct job, and knowing them explains where things fail. Init prepares the working directory: it downloads providers and modules, configures the backend, creates the dot terraform directory, and writes or checks the dependency lock file. It must be re-run when providers, modules, or backend settings change. Plan is the preview: Terraform loads the configuration, reads state, refreshes the real resource attributes from the providers, builds the dependency graph, compares desired and actual to produce a diff, and shows the proposed create, update, and destroy actions. It doesn't change anything, and can save the plan to a file. Apply executes: it acquires the state lock, walks the graph in dependency order with parallelism, calling the provider for each create, update, or delete, writes state after each resource succeeds, and releases the lock. Supporting commands include validate for static checks, fmt for formatting, and output and show for inspection. This explains, for example, why a failed apply leaves partial state, why plan needs credentials, and why changing a backend needs init.

Likely follow-ups

- Why does plan need cloud credentials?
- What does Terraform write to the state during a failed apply?

Understanding the phases explains where things fail and why certain operations need re-init. Each phase has a distinct job.

THE LIFECYCLE

```
init  : download providers + modules, configure backend, build .terraform/ + lock file
```

```
plan  : refresh state (read real infra) → build graph → diff config vs state → propose
actions
```

```
apply : acquire lock → execute graph (parallelized) → update state after each resource →
release lock
```

```
# Supporting:
```

```
terraform validate   # static config/syntax/type checks (no API calls)
```

```
terraform fmt        # canonical formatting
```

init sets up the working directory: it downloads provider plugins and modules, configures the backend, and writes the dependency lock file — required after adding providers/modules or changing the backend. **plan** refreshes state from the real world, builds the dependency DAG, and computes the diff into a set of create/update/delete/replace actions. **apply** locks state, walks the graph executing actions in parallel where independent, and writes state incrementally so a mid-run failure still records what succeeded.

Why apply writes state incrementally: if apply fails halfway, the resources already created are in state — so re-running continues rather than recreating. This is why a partial-apply recovery is just “fix the cause and re-apply,” not “start over.”

certs.eknathalabs.com

📚 **Official docs**

- Terraform: lifecycle meta-argument

Q47 **How do you safely run Terraform in CI/CD, and what are the key guardrails?** Backend

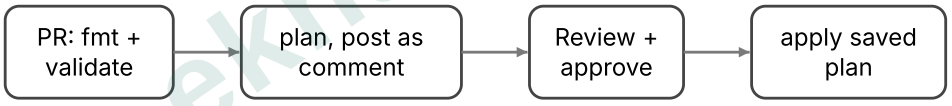
🎙 **Spoken answer · ~74 s**

Automated Terraform needs to be safe by construction. The key guardrails are these. Separate plan from apply: on a pull request, run fmt check, validate, and plan, and post the plan as a comment for review. Apply only after merge, and apply the saved plan file, not a freshly computed one, so what runs is exactly what was reviewed. Use short-lived credentials via OIDC federation rather than stored cloud keys, with separate roles for plan and apply scoped per environment. State locking plus serialised pipeline runs ensure one apply at a time per state. Policy as code: scan with tfsec, Checkov, or Trivy, and enforce rules with OPA or Sentinel against the plan JSON, blocking things like public buckets. Approval gates for production, with protected branches and required reviewers. Pinned Terraform and provider versions, and a committed lock file. Drift detection on a schedule. And protect the pipeline itself: restricted access to modify workflows, since whoever controls them controls the credentials. Finally, I run Terraform only from CI for shared environments, so there are no surprise local applies.

Likely follow-ups

- Why apply a saved plan rather than recompute at apply time?
- How would you enforce policy on the plan?

🗺 **Safe CI/CD**



Automated Terraform must be safe by construction: plan and apply are separated, credentials are short-lived, and applies are gated and serialized.

CI/CD GUARDRAILS

```
Plan on PR: terraform plan -out=tfplan  → post plan to the PR for review
```

```
Apply on merge: terraform apply tfplan   # apply the SAVED plan, not a fresh one
```

```
OIDC for cloud creds (no long-lived secrets in CI)
```

```
State locking ensures only one apply at a time; serialize via environments/queues
```

```
Policy as code: OPA/Conftest or Sentinel gates before apply
```

```
Required manual approval for prod applies
```

certs.eknathalabs.com

The core pattern is **plan/apply separation**: generate and save a plan (`-out`) on PR for human review, then apply *that exact saved plan* on merge so what was reviewed is what runs (no drift between review and apply). Use OIDC for ephemeral cloud credentials, rely on state locking plus environment serialization to prevent concurrent applies, gate with policy-as-code (OPA/Sentinel) for compliance, and require manual approval for production.

Tools that productize this: Atlantis (PR-driven plan/apply with locking comments), Terraform Cloud/Enterprise (run queue + Sentinel), Spacelift, env0. Mentioning plan/apply separation + saved-plan apply + OIDC is the strongest signal of safe automation.

📚 **Official docs**

- AWS Control Tower

Q48 **A teammate ran terraform apply locally and it diverged from CI-managed state. How do you reconcile?** State

🎙 **Spoken answer · ~78 s**

A local apply bypasses review and can leave reality and CI's expectations out of sync, so the goal is to get reality, state, and code back into agreement. First, don't panic and don't apply anything else. I check what happened: if the teammate used the same remote backend, state was updated, so state matches reality but the code in Git may not match. I run terraform plan with refresh-only to see what diverged, and a normal plan to see what CI would change now. Then decide: if the local change was desired, codify it in a pull request so the code matches, and the plan goes clean through CI. If it was a mistake, revert it by applying the reviewed code from the pipeline. If they applied with local or different state, I compare it with the remote state, using state pull to inspect, and reconcile carefully, using import or state operations as needed. Then prevent a repeat: remove write credentials from laptops, restrict the backend to the CI role, require pull requests, and explain that local plans are fine but applies go through the pipeline.

Likely follow-ups

- What if the teammate applied against a different state file?
- How do you stop people applying from laptops?

Local applies bypass review and can create drift between what CI expects and what exists. Reconciliation means getting reality, state, and code back in agreement.

RECONCILIATION

```
terraform plan -refresh-only      # detect what reality/state diverged from code
```

```
terraform plan                     # see what CI would now change
```

```
# If the local change is desired: codify it, PR it, let CI apply
```

```
# If undesired: revert via apply from the reviewed code
```

```
terraform state pull > current.tfstate   # inspect what state believes
```

certs.eknathalabs.com

First diagnose with `-refresh-only` (what drifted) and a normal `plan` (what CI wants to change). If the local change was legitimate, the fix is process: write it into code, open a PR, and let CI apply it so state and code agree. If it was a mistake, apply from the reviewed mainline code to revert reality. The state is shared (remote), so once code and reality align, CI is consistent again.

Root cause is process, not Git: local applies against shared prod state are the problem. The durable fix is removing humans' ability to apply to prod directly — restrict apply to CI with OIDC, lock down state-write IAM, and require PR-driven applies. Reconciling once without fixing the process guarantees recurrence.

📚 **Official docs**

- Terraform: State
- Terraform CLI: apply

Q49 **How do you design a reusable Terraform module? What makes a module good vs bad?** Modules

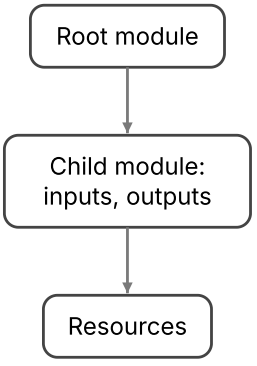
🎙 **Spoken answer · ~79 s**

A good module is a well-bounded abstraction with a clear interface and sensible defaults. The principles I follow: a single responsibility, one cohesive thing such as a VPC or a service, rather than a kitchen sink; a clear interface with typed variables, descriptions, validation, and defaults that are safe; explicit outputs that expose only what callers need; no hardcoded provider configuration, which should come from the caller; and no hidden assumptions about the environment, naming, or account. It should be composable, so modules can be combined, and versioned, with semantic versions and a changelog. It needs documentation and examples, and tests. A bad module is either too thin, a wrapper that merely renames a resource's arguments and adds no value, or too thick, trying to handle every use case with dozens of variables and conditionals, which makes it hard to understand and change. Another warning sign is a module that reaches out to look up things it should be passed. I also avoid deep nesting of modules calling modules, and I prefer flat composition in a root module. And I design for backward compatibility, using moved blocks when refactoring.

Likely follow-ups

- How many variables is too many for a module?
- How do you evolve a module without breaking consumers?

🗺 **Module design**



certs.eknathalabs.com

A good module is a well-bounded abstraction with a clear interface, sensible defaults, and minimal assumptions about its caller. A bad module either does too little (a thin wrapper) or too much (a kitchen sink).

MODULE DESIGN PRINCIPLES

```
Clear interface: typed variables with descriptions, validation, sensible defaults
```

```
Explicit outputs: expose what callers need, nothing internal
```

```
Single responsibility: a module does one cohesive thing (a VPC, an ECS service)
```

```
No hardcoded provider/region/account assumptions; pass them in
```

```
Versioned + documented: README with examples, version constraints
```

Treat a module like a function: well-typed inputs (with `validation` blocks and defaults), explicit outputs, and one clear responsibility. Avoid leaking implementation details or hardcoding environment specifics — callers supply those. A module that wraps a single resource adds indirection without value; a module that provisions an entire environment is too coupled to reuse. The sweet spot is a cohesive, composable unit (a VPC, a service, a database).

Interview signal: “modules are functions for infrastructure” — inputs/outputs/single-responsibility — plus versioning and validation. Mention the registry pattern (root module composes child modules) and that over-abstraction is as harmful as none.

📚 **Official docs**

- Terraform: Modules

Q50 **Explain count vs for_each for resource iteration. When does each cause problems?** Modules

🎙 **Spoken answer · ~78 s**

Both create multiple instances of a resource, but they index differently, and that matters for state stability. Count uses integer positions: resource dot web index zero, one, two. For_each uses a map or set with stable keys: resource web with a key like api or worker. The problem with count is that if the list changes, removing an element from the middle shifts the indexes of everything after it, so Terraform sees those resources as changed, and may destroy and recreate resources that didn't really change. For_each keys are stable, so removing one item affects only that item. So I prefer for_each for collections of distinct things, and use count mainly for simple on or off toggles, using zero or one. For_each has its own gotchas: the keys must be known at plan time, so you can't use values that are only known after apply, such as an ID of another resource that doesn't exist yet, and it requires a map or a set of strings, so lists need to be converted with toset. Also changing a key means a destroy and create, so moved blocks help renaming.

Likely follow-ups

- Why does removing a middle list item with count cause trouble?
- What does 'known only after apply' mean for for_each?

certs.eknathalabs.com

Both create multiple instances of a resource, but they index differently — count by integer position, for_each by a stable key — and that difference has major consequences for state stability.

COUNT VS FOR_EACH

```
# count: integer-indexed (aws_instance.web[0], [1], [2])
```

```
resource "aws_instance" "web" { count = 3 }
```

```
# for_each: key-indexed (aws_instance.web["api"], ["worker"])
```

```
resource "aws_instance" "web" { for_each = toset(["api","worker"]) }
```

```
resource "aws_instance" "web" { for_each = var.instances_map }
```

The trap with **count**: removing an item from the middle of the list shifts every subsequent index, so Terraform destroys and recreates everything after it (index 1 becomes 0, etc.). **for_each** keys instances by a stable identifier, so adding/removing one entry only affects that entry. Use `count` only for truly identical, position-independent instances or simple on/off toggles (`count = var.enabled ? 1 : 0`); use `for_each` whenever instances have distinct identities.

Classic production incident: a list with `count` where someone removes a middle element → mass recreation of unrelated resources. Default to `for_each` with a map/set keyed by a stable name. This is one of the most-asked Terraform discriminators.

📚 **Official docs**

- Terraform: for_each

Q51 **How do you version and distribute internal modules across many teams?** Modules

🎙 **Spoken answer · ~75 s**

Modules need versioning so consumers get reproducible, opt-in upgrades instead of surprise changes. The distribution options range from simple to scalable. Git sources with a pinned tag or commit, for example a ref of v3.2.1, are easy to start with, and need credentials management. A private registry, such as HCP Terraform's private registry, Artifactory, or a self-hosted one, is best at scale: it supports semantic versions and version constraints like tilde arrow three point two, discoverability, documentation, and access control. Whatever the mechanism, I always pin versions and never float on a branch, because a change to main would silently change infrastructure for every consumer. I use semantic versioning: patch for fixes, minor for backward-compatible features, and major for breaking changes, with a changelog, and automate releases from tags in CI. Consumers upgrade deliberately by bumping the version and reviewing the plan; I can use Renovate or Dependabot to propose updates. Each module has tests, examples, and documentation, an owner, and a deprecation policy, and a monorepo or one repo per module are both workable depending on team size.

Likely follow-ups

- Why is floating on a branch dangerous?
- How do you notify consumers of breaking changes?

certs.eknathalabs.com

Modules need versioning so consumers get reproducible, opt-in upgrades. Distribution options range from Git refs to a private registry.

VERSIONING & SOURCES

```
# Private registry (best at scale): semantic versions + version constraints
```

```
module "vpc" { source="app.terraform.io/org/vpc/aws" version="~> 3.2" }
```

```
# Git with a pinned tag/ref:
```

```
module "vpc" { source="git::https://git/org/modules.git//vpc?ref=v3.2.1" }
```

```
# Always PIN the version; never float on a branch:
```

```
version = "3.2.1"   # exact   or   "~> 3.2"  (>=3.2.0, <3.3.0)
```

Publish modules to a **private registry** (Terraform Cloud/Enterprise or an OSS one) with semantic versioning so consumers pin via `version` constraints and upgrade deliberately. If using Git sources, always pin to a tag (`?ref=v3.2.1`), never a branch — floating on `main` means a module change can silently break every consumer's next apply. SemVer signals intent: patch = safe, minor = additive, major = breaking.

> **Never use an unpinned module source in production.** `source=".../vpc"` without `?ref` or `version` pulls latest — an upstream change then alters infra on your next apply with no review. Pin everything; treat module upgrades as reviewed PRs.

> 📚 **Official docs**
>
> - Terraform: Publishing modules (registry & versioning)

certs.eknathalabs.com

Q52 **Terraform plan/apply is very slow on a large configuration. How do you diagnose and speed it up?** Performance

🎙 Spoken answer · ~76 s

Slowness comes from refresh time with many resources, a large monolithic state, slow provider APIs, or excessive data source calls, so I diagnose before tuning. I find the time sink using debug logs with durations, or by timing plan with and without refresh. Then I attack the dominant cause. Skipping refresh with refresh equals false is a big win during iteration when I trust there's no drift, but I don't rely on it in the pipeline's final plan. Splitting the configuration into smaller states by layer, team, or environment cuts refresh time, lock contention, and blast radius; this is the structural fix. Scoping with target helps while iterating but isn't a routine tool. Raising parallelism from ten can speed up applies if the provider's API allows it, but may trigger throttling. Reducing data sources that call the API on every plan, and caching values in parameter stores or locals, helps too. Also, make sure providers are cached in CI, avoid unnecessary depends_on that serialise work, and keep the number of resources per state manageable. Some resources, like databases, are just slow to create.

Likely follow-ups

- How do you find which resources take longest during an apply?
- Why is -refresh=false risky in a pipeline?

Slowness usually comes from refresh time (many resources, slow provider APIs), an oversized monolithic state, or excessive provider calls. Diagnose first, then attack the dominant cause.

SPEED-UP TECHNIQUES

```
terraform plan -refresh=false          # skip refresh if you trust no drift (big win)
```

```
terraform plan -target=module.x        # scope to a subset while iterating
```

```
terraform apply -parallelism=20        # raise concurrency (default 10) if API allows
```

```
TF_LOG=trace ... 2>&1 | grep duration   # find slow provider calls
```

```
# Structural fixes:
```

```
Split monolithic state into layers/services (smaller graphs, faster refresh)
```

```
Reduce data sources that hit slow APIs on every plan
```

The biggest lever is usually **state size**: a monolith with thousands of resources refreshes slowly because every plan reads every resource's live state. Splitting into layered/service states shrinks each graph dramatically. Tactically, `-refresh=false` skips the slow refresh (safe only when drift is unlikely), `-target` scopes work during iteration, and raising `-parallelism` helps if the provider API tolerates it. Excessive `data` sources that call slow APIs on every plan are a common hidden cost.

Diagnose before optimizing: `TF_LOG=trace` reveals where time goes (refresh vs specific slow API calls). The durable fix for genuinely large infra is state segmentation, not flags — say that to show architectural thinking.

certs.eknathalabs.com

📚 **Official docs**

- Terraform CLI: plan

Q53 **What are dynamic blocks and when should you avoid them?** Modules

🎙 **Spoken answer · ~77 s**

Dynamic blocks generate repeated nested blocks, such as multiple ingress rules in a security group, from a collection, avoiding copy-paste. You write dynamic with the block name, provide a for_each, and a content block that uses the iterator's value. They're the right tool when a resource has a repeatable nested block and the number or content of blocks is driven by data, for instance a variable list of ingress rules or multiple tags in a certain nested structure. I'd avoid them when the number of blocks is small and fixed, because writing them explicitly is clearer. Overuse makes configuration harder to read, since someone has to mentally expand the loops, and it makes debugging plan output harder. Nested dynamic blocks are especially hard to follow. Another trap is trying to use dynamic blocks to build everything generically, creating an abstraction that hides the actual resource arguments and is hard to extend. Also note that dynamic blocks only generate nested blocks, not resource arguments, and for_each values must be known at plan time. My rule: use them when they remove real duplication, and keep them shallow.

Likely follow-ups

- Can a dynamic block generate resource arguments?
- Why avoid deeply nested dynamic blocks?

Dynamic blocks generate repeated nested configuration blocks (e.g. multiple ingress rules) from a collection, avoiding copy-paste. They are powerful but hurt readability when overused.

DYNAMIC BLOCK USAGE

```
dynamic "ingress" {
  for_each = var.ingress_rules
  content {
    from_port = ingress.value.from
    to_port   = ingress.value.to
    protocol  = ingress.value.protocol
  }
}
```

certs.eknathalabs.com

Use dynamic blocks when a resource has a repeatable nested block whose count/content is data-driven — security group rules, IAM policy statements, load balancer listeners. They keep config DRY. But they obscure the resulting configuration (you cannot read the actual rules at a glance), complicate diffs, and make modules harder to understand. If the nesting is fixed or small, write the blocks out literally for clarity.

Readability cost: heavily nested or conditional dynamic blocks become write-only code. Prefer explicit blocks when the set is small/static; reserve dynamic blocks for genuinely variable, data-driven repetition. Over-engineering modules with dynamic-everything is a real anti-pattern interviewers probe for.

📚 **Official docs**

- Kubernetes: Ingress

Q54 **Explain variable validation, type constraints, and how to build a robust module interface.** Modules

🎙 **Spoken answer · ~76 s**

A robust module rejects bad input early, with a clear message, instead of failing deep in a provider call. Two tools help. Type constraints state exactly what a variable accepts: string, number, bool, lists, maps, sets, and structured object types with named attributes, such as a map of objects with cidr and az. I use precise types instead of any, and optional attributes with defaults for objects, so callers can omit fields. Validation blocks add rules beyond the type, using a condition expression and an error message, for example checking that an instance type matches an allowed family, a CIDR is valid, or a name matches a pattern. Preconditions and postconditions on resources and outputs can check assumptions about the environment and results. Beyond that, a robust interface has descriptions on every variable and output, sensible and safe defaults, sensitive marking for secrets, and a small, stable set of inputs, with the complexity kept inside. I document it with examples, and test it using the native test framework. The result is a module that is hard to misuse and easy to consume.

Likely follow-ups

- What is the difference between a validation block and a precondition?
- Why avoid the any type in variables?

A robust module rejects bad input early with clear errors rather than failing deep in a provider call. Type constraints and validation blocks are the tools.

TYPED, VALIDATED INPUTS

```
variable "instance_type" {
  type        = string
  description = "EC2 instance type"
  default     = "t3.micro"
  validation {
    condition     = can(regex("^t3\\.", var.instance_type))
```

certs.eknathalabs.com

```
    error_message = "Only t3 family is allowed."
  }
}
variable "subnets" { type = map(object({ cidr = string, az = string })) }
```

Use precise **type constraints** (`map(object({...}))`, `list(string)`) so callers get a clear type error instead of a confusing downstream failure. Add `validation` blocks to enforce business rules (allowed values, naming patterns, ranges) with human-readable `error_message`s. Provide sensible `default`s for optional inputs and mark secrets `sensitive`. This turns the module interface into a contract that fails fast and self-documents.

Newer features to mention: `optional()` attributes in object types (with defaults) let you build flexible-but-typed inputs, and custom `validation` with `can()`/`regex()` catches misuse at plan time. Fast, clear failure is a hallmark of a production-grade module.

📚 **Official docs**

- Terraform: Custom conditions & variable validation

Q55 **How does Terraform parallelism work, and when would you tune it?** Performance

🎙 **Spoken answer · ~74 s**

Terraform walks the resource dependency graph and processes independent resources concurrently, up to a limit that defaults to ten. The graph determines what can run in parallel, because dependent resources must wait, and the parallelism flag caps how many operations run at once. I tune it in two directions. Raising it, for example to twenty or more, can speed up applies on large configurations with many independent resources, provided the provider API can handle the load. Lowering it, down to five or even one, helps when the provider throttles requests, causing rate limit errors, and setting it to one serialises everything, which is a useful debugging step when I suspect race conditions or ordering problems that aren't expressed in the graph. It affects plan and apply, including the refresh phase. The trade-off is speed versus API pressure and the risk of eventual-consistency failures. Raising parallelism doesn't help if the graph is mostly a serial chain, caused by dependencies or unnecessary depends_on, so the better fix for slow applies is often splitting the configuration or removing artificial dependencies.

Likely follow-ups

- Does raising parallelism help a configuration with a long dependency chain?
- When would you set parallelism to 1?

Terraform walks the resource graph concurrently, processing independent resources in parallel up to a limit (default 10). Tuning trades speed against provider/API rate limits.

PARALLELISM CONTROL

```
terraform apply -parallelism=20    # raise from default 10
```

certs.eknathalabs.com

```
terraform apply -parallelism=1     # serialize (debugging ordering/races)
```

```
# The graph dictates what CAN run in parallel; -parallelism caps how many at once
```

The DAG determines which resources are independent; `-parallelism` caps how many of those run simultaneously. Raising it speeds up large applies **if** the provider API can handle the concurrency. Lowering it (even to 1) helps when you hit API rate limits/throttling, or when debugging suspected ordering issues. It does not change ordering — dependent resources still wait for their prerequisites.

The trap: cranking parallelism on a provider with strict rate limits causes throttling errors and slower (or failed) applies, not faster ones. Tune based on the provider's limits; sometimes *lowering* parallelism fixes intermittent apply failures caused by API throttling.

📚 **Official docs**

- Terraform CLI: apply

Q56 **How do you compose modules (root calling child modules) and pass data between them?** Modules

🎙 **Spoken answer · ~75 s**

Composition is how Terraform scales: a thin root module wires together child modules and passes the outputs of one as inputs to another. For example, a network module takes a CIDR and outputs the VPC ID and private subnet IDs; the app module takes those as inputs, so in the root I write subnet_ids equals module dot network dot private_subnet_ids. Terraform infers the dependency from that reference and orders the modules correctly. The root can also re-export child outputs so they're visible to the CLI and to other configurations. The guidance is that the root module should mostly be wiring: calling modules, passing values, configuring providers and the backend, and not containing lots of resources and logic itself. Child modules should be independent and not reach into each other; they communicate only through variables and outputs. I keep the hierarchy shallow, preferring flat composition to deeply nested modules, because deep nesting makes data flow hard to follow. Providers are configured in the root and passed down. This structure makes modules reusable, testable on their own, and easy to reason about.

Likely follow-ups

- Why prefer flat composition to deeply nested modules?
- How does a root module re-export a child's output?

Composition is the heart of scalable Terraform: a thin root module wires together child modules, passing one module's outputs as another's inputs.

COMPOSITION PATTERN

```
module "network" { source = "./modules/network"  cidr = var.cidr }
```

```
module "app" {
```

```
  source     = "./modules/app"
```

certs.eknathalabs.com

```
  subnet_ids = module.network.private_subnet_ids   # output → input wiring
```

```
  vpc_id     = module.network.vpc_id
```

```
}
```

```
output "app_url" { value = module.app.url }         # re-export child output
```

The root module should be mostly **wiring**: instantiate child modules and connect them by feeding outputs into inputs (`module.network.vpc_id → module.app.vpc_id`). This creates the dependency graph automatically and keeps each child focused and reusable. The root re-exports the outputs callers care about. Keep business logic in children; keep the root declarative and readable.

Anti-pattern to avoid: deeply nested module trees (modules calling modules calling modules) become hard to trace and slow to plan. Favor a flat-ish composition — a root composing a handful of well-bounded children — over deep hierarchies.

📚 **Official docs**

- Terraform: Module composition

Q57 **What is the dependency lock file (.terraform.lock.hcl) and how do you manage it?** Performance

🎙 **Spoken answer · ~73 s**

The dependency lock file, .terraform.lock.hcl, pins the exact provider versions and their checksums that Terraform selected, so every run and every teammate uses identical providers. That gives reproducibility, and also supply-chain integrity, because the checksums verify that downloaded provider binaries are what was originally recorded. It's created and updated by terraform init. I commit it to version control. To update providers within the version constraints I run init with upgrade, and review the resulting diff to the lock file in a pull request. Because checksums are platform specific, I generate hashes for all platforms my team and CI use with terraform providers lock and the platform flags, such as linux amd64 and darwin arm64, otherwise init may fail on a different machine than the one that created the lock. Constraints in required_providers say what's acceptable, the lock file records what was chosen. The lock file doesn't cover modules, which I pin with exact versions in the module block. Without a committed lock file, CI may silently pick a newer provider than the one tested locally.

Likely follow-ups

- What is the difference between a version constraint and the lock file?
- Why generate hashes for multiple platforms?

The lock file pins exact provider versions and their checksums so every run and every teammate uses identical providers — reproducibility and supply-chain integrity.

LOCK FILE MANAGEMENT

```
terraform init                      # creates/updates .terraform.lock.hcl
```

certs.eknathalabs.com

```
terraform providers lock -platform=linux_amd64 -platform=darwin_arm64  # multi-platform
hashes
```

```
terraform init -upgrade            # update providers within constraints + refresh lock
```

```
# COMMIT .terraform.lock.hcl to version control
```

`.terraform.lock.hcl` records the selected provider versions and cryptographic checksums. **Commit it** so CI and every developer resolve to the exact same providers — otherwise a teammate could silently pull a newer provider with behavior changes. Add multi-platform hashes with `terraform providers lock` when developers and CI run on different OS/arch (e.g. Mac ARM laptops, Linux CI), or init fails on a missing checksum. Upgrade deliberately with `-upgrade` and review the diff.

Supply-chain angle: the checksums defend against a tampered/substituted provider binary. Committing the lock file is both a reproducibility and a security control — frame it that way. Treat lock-file bumps as reviewable PRs like any dependency upgrade.

📚 **Official docs**

- Terraform: Dependency lock file

Q58 **How do you handle conditional resource creation and optional features in a module?** Modules

🎙 **Spoken answer · ~76 s**

Modules often need to turn resources on or off or change behaviour based on inputs. The idiomatic tools are these. The count toggle: set count to a conditional expression, one when the feature is enabled and zero otherwise, for example for an optional NAT gateway; other resources then reference it with an index or one function and need to handle the case where it doesn't exist. For_each over a conditional collection, using an empty set when disabled, which also handles multiple optional items with stable keys. Optional attributes in object types, with defaults, so callers can provide only what they need. Ternary expressions and locals to compute values. And dynamic blocks for optional nested blocks. Two cautions: conditions can only depend on values known at plan time, so I can't toggle based on an attribute that is only known after apply, and too many conditionals make a module hard to understand and test. If a module has dozens of boolean toggles, it's a sign it is doing too much and should be split. I use preconditions to validate valid combinations of options.

Likely follow-ups

- Why can't count depend on a resource attribute known only after apply?
- When do too many toggles signal that a module should be split?

Modules often need to toggle resources on/off or vary behavior by input. The idiomatic tools are count-as-toggle, for_each over conditional collections, and optional object attributes.

CONDITIONAL PATTERNS

```
# On/off toggle with count:
```

```
resource "aws_nat_gateway" "this" { count = var.enable_nat ? 1 : 0 }
```

certs.eknathalabs.com

```
# Conditional set membership with for_each:
```

```
for_each = var.create ? toset(var.names) : toset([])
```

```
# Optional attributes with defaults (object type):
```

```
variable "cfg" { type = object({ size = optional(string, "small") }) }
```

```
# Reference a count=0/1 resource safely:
```

```
value = var.enable_nat ? aws_nat_gateway.this[0].id : null
```

Use `count = condition ? 1 : 0` for a simple on/off resource, and `for_each` over a conditionally-empty collection for variable sets. The `optional()` modifier in object types gives flexible inputs with defaults. Be careful referencing conditional resources: a `count`-gated resource must be indexed (`[0]`) and guarded so you do not dereference it when it does not exist.

The footgun: referencing `aws_nat_gateway.this[0].id` when `count` is 0 errors. Always guard with the same condition (`var.enable_nat ? ...[0].id : null`). Overusing conditionals also makes modules hard to reason about — sometimes two clear modules beat one conditional-laden one.

📚 **Official docs**

- Terraform: Conditional expressions

Q59 **How do you reduce the number of provider/API calls and avoid hitting rate limits?** Performance

🎙 **Spoken answer · ~77 s**

The volume of provider API calls is driven by the number of resources refreshed, the data sources queried, and the parallelism, so reducing those cuts both time and throttling. Techniques: minimise data sources that query the API on every plan, and where the values are stable, cache them in locals, parameters, or variables passed from a parent. Split state so each plan refreshes fewer resources; a plan only refreshes what's in its state. Lower parallelism when the provider throttles. Use refresh equals false for quick iterative plans when drift is unlikely, and target during development to avoid refreshing the whole graph, though neither is a routine pipeline setting. Configure provider retries with exponential backoff, such as max_retries on AWS. Avoid many tiny resources where one resource can do the job, and avoid patterns that generate large numbers of calls such as per-item lookups in loops. Stagger pipelines so many don't hit the same account's API limits at once. If limits remain a problem for legitimate scale, request quota increases. Every data source is an API call on every plan, which is easy to forget.

Likely follow-ups

- Why does every data source cost an API call on every plan?
- How does splitting state reduce API calls?

Excessive data sources, oversized state, and high parallelism drive API call volume. Reducing calls speeds plans and avoids throttling.

REDUCING API PRESSURE

certs.eknathalabs.com

```
Minimize data sources that query on EVERY plan (cache values in SSM/locals instead)
```

```
Split state so each plan refreshes fewer resources
```

```
Lower -parallelism when the provider throttles
```

```
Use -refresh=false for fast iterative plans (when drift is unlikely)
```

```
-target during iteration to avoid refreshing the whole graph
```

Every `data` source and every managed resource is refreshed/queried on plan. The big reductions come from **fewer resources per state** (segmentation) and **fewer expensive data sources** (publish stable values to SSM/Parameter Store once rather than querying the API on every plan). When you do hit limits, lowering parallelism and using `-refresh=false`/`-target` for iteration reduces call bursts.

Provider-specific: some providers support request caching or have known rate limits; check provider docs and cloud quotas. Persistent throttling on a huge config is a signal to segment state, not just to retry harder.

📚 **Official docs**

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling

Q60 **Explain locals, variables, and outputs — and the right use of each.** Modules

🎙 **Spoken answer · ~78 s**

These are the three data primitives of a Terraform configuration, and each has a distinct role. A variable is an input to a module or root configuration: the knobs a caller turns, set through the command line, tfvars files, environment variables, or defaults, and typed and validated. A local is a computed or intermediate value within a module, used to avoid repetition, such as a name prefix built from project and environment, or a merged tag map. It isn't an input and can't be set by the caller. An output exposes a value to the caller of a module, or to the CLI and other configurations from the root module. Confusing the roles leads to messy code: don't use variables for things that should be derived, because that lets callers set inconsistent values; use locals for derived values and for keeping expressions readable; and expose outputs deliberately, only what consumers need, because outputs form the module's public interface. Mark sensitive variables and outputs, remembering this only hides them in CLI output. I keep variables in variables.tf, outputs in outputs.tf, and use locals sparingly, giving them meaningful names.

Likely follow-ups

- When should a value be a local rather than a variable?
- Do outputs get stored in state?

These are the three data primitives of a configuration. Confusing their roles leads to messy, hard-to-maintain code.

ROLES

```
variable : INPUT to a module/config (set by caller, CLI, tfvars, env)
```

certs.eknathalabs.com

```
local    : computed/intermediate value, DRY within a module (not an input)
```

```
output   : value EXPOSED to the caller or other configs
```

```
locals { name_prefix = "${var.project}-${var.env}" }
```

```
output "vpc_id" { value = aws_vpc.main.id }
```

Variables are the module's inputs — the knobs callers turn. **Locals** are internal computed values used to avoid repetition and to name things consistently (e.g. a derived name prefix or a merged tag map); they are not settable from outside. **Outputs** publish values for other modules or the root to consume. A clean module reads inputs (variables), derives internals (locals), provisions resources, and publishes results (outputs).

Common smell: using a variable with a default as a "constant" that callers should never set — that should be a `local`. And overusing locals for trivial one-off values adds indirection. Match the primitive to the role: caller-facing → variable/output, internal → local.

📚 **Official docs**

- Terraform: Locals
- Terraform: Output values

Q61 **What is -target and why is it considered a code smell in routine use?** Performance

🎙 **Spoken answer · ~79 s**

The target option limits a plan or apply to specific resources and whatever they depend on. It's legitimate in a few cases: recovery, such as applying just the resource that failed mid-run; iteration speed on a very large configuration while developing; and working around a bug or a temporary problem. But it's a code smell when used routinely, for several reasons. It skips the rest of the graph, so other changes are not applied, and the infrastructure can end up in a state the configuration doesn't describe. Dependencies may be partially applied, with unexpected results. It hides drift and makes plans less trustworthy, because a clean targeted plan says nothing about the rest. It encourages workflows where people avoid fixing the underlying problem, such as a slow monolithic state or a broken dependency. Terraform itself warns that target is for exceptional use. If I find myself needing it regularly, the root cause is usually that the configuration is too big and should be split into smaller states, or the pipeline is too slow. After any targeted apply, I run a full plan to confirm the overall state is consistent.

Likely follow-ups

- What risks does -target create for dependent resources?
- What would you do instead if plans are too slow?

`-target` limits an operation to specific resources and their dependencies. It is a legitimate recovery/iteration tool but a warning sign when used routinely.

TARGETED OPERATIONS

certs.eknathalabs.com

```
terraform apply -target=aws_instance.web        # apply only this + its deps
```

```
terraform plan -target=module.network           # scope to a module
```

```
terraform apply -replace=aws_instance.web        # targeted replacement
```

Targeting is appropriate for **recovery** (apply just the resource that failed mid-run) and **iteration speed** on a huge config while developing. But routine reliance on `-target` means your normal plan/apply is unmanageable — usually a sign the state is too big (should be split) or the config has hidden coupling. Targeting also skips parts of the graph, so it can apply an inconsistent partial change that a full apply would have ordered differently.

Why it is a smell: a healthy config can run a full `plan`/`apply` routinely. Constant targeting hides a structural problem (monolithic state, slow refresh). The fix is state segmentation, not normalizing `-target`. HashiCorp explicitly warns it is for exceptional use.

📚 **Official docs**

- Terraform CLI: apply

Q62 **How do you test Terraform modules? Walk through the testing pyramid.** Modules

🎙 **Spoken answer · ~77 s**

I test Terraform modules in layers, from cheap and fast to slower and more realistic. At the base, static checks: terraform fmt, validate, and linters like TFLint, plus security scanners such as Checkov, tfsec, or Trivy, which catch misconfiguration without creating anything. Next, plan-level tests using the native test framework available since Terraform one point six: test files with run blocks and assert conditions that check the planned values, such as tags, names, or that a resource is or isn't created. These can run with mocked providers, so they're fast and free. Then integration tests that actually apply the module in a sandbox account, assert on the real result, and destroy it, either with the native framework using command apply, or with Terratest in Go, which can also make HTTP calls to verify the infrastructure works. These are slower and cost money, so I run them less often, on module changes or nightly, with unique names and reliable cleanup. On top, policy tests with OPA or Sentinel against the plan. I run the cheap layers on every pull request and the expensive ones before release.

Likely follow-ups

- What can the native test framework verify without creating resources?
- How do you make sure integration tests clean up after failures?

Terraform testing spans static checks, unit-ish plan assertions, and integration tests that actually provision. A layered approach catches different failure classes cheaply.

TESTING LAYERS

certs.eknathalabs.com

```
Static : terraform validate, fmt, tflint, checkov/tfsec (security)
```

```
Native test framework (1.6+): tests in .tftest.hcl with assert blocks
```

```
  run "plan" { command = plan  assert { condition=...  error_message=... } }
```

```
Integration : Terratest (Go) — apply to a real sandbox, assert, destroy
```

```
Policy : OPA/Conftest or Sentinel against the plan JSON
```

Bottom of the pyramid (cheap, fast, run on every PR): `validate`, `fmt`, `tflint`, and security scanners (checkov/tfsec). The middle: Terraform's **native test framework** (1.6+) with `.tftest.hcl` files asserting on plan/apply outputs without leaving Terraform. The top (slow, expensive, fewer): **Terratest** spins up real resources in a sandbox account, asserts they work, and tears them down — catching issues only visible against real APIs. Policy-as-code gates compliance against the plan.

Modern emphasis: the native `terraform test` framework (1.6+) reduced the need to reach straight for Terratest — mention it. Pyramid logic applies: many cheap static/plan tests, few expensive apply-and-verify tests.

📚 **Official docs**

- Terraform: Modules

Q63 **How do you handle Terraform with thousands of resources without applies becoming unmanageable?** Performance

🎙 **Spoken answer · ~74 s**

At thousands of resources, a single monolithic state makes plans slow, applies risky, locks painful, and the blast radius huge. The answer is architectural decomposition plus orchestration. I split state by layer, such as networking, data, compute, and applications, and by service or team, so each plan is small and each team owns its state. Cross-layer references go through outputs published to a parameter store or data sources, not by sharing one giant state. Ordering between states is handled by Terragrunt dependencies or by pipeline stages that apply layers in sequence. Each state gets its own CI plan and apply, so runtime and blast radius stay bounded, and independent states can run in parallel. I use the refresh equals false option strategically on large states in pipelines where scheduled drift detection covers refresh separately. Modules provide reuse, and I standardise conventions for naming and structure. I also watch provider API limits and consider quotas. Finally, strong guardrails matter more at scale: policy as code, code owners, and drift detection, because manual oversight doesn't scale to thousands of resources.

Likely follow-ups

- How do you order applies across dozens of states?
- What is the downside of too many small states?

At thousands of resources, monolithic state makes plans slow, applies risky, and locks painful. The answer is architectural decomposition plus orchestration tooling.

SCALING STRATEGIES

certs.eknathalabs.com

```
Decompose state by layer (network/data/compute) and by service/team
```

```
Cross-layer references via SSM/data sources, not giant shared state
```

```
Orchestrate ordering with Terragrunt (dependencies) or pipelines
```

```
Per-state CI plan/apply so blast radius and runtime stay bounded
```

```
Use -refresh=false strategically + periodic drift checks per state
```

The core move is **state segmentation**: many small, independently-applied states instead of one monolith. Each state plans fast, has bounded blast radius, and can be owned by one team. Cross-state data flows through loosely-coupled mechanisms (SSM, provider data sources). Orchestration tools (Terragrunt for DRY config + dependency ordering, or a pipeline that sequences states) manage the inter-state ordering that segmentation introduces.

Trade-off to name: segmentation buys speed, isolation, and ownership at the cost of orchestration complexity (ordering, more backends, more remote-state plumbing). The skill is segmenting along genuine ownership/change-rate boundaries, not arbitrarily — that judgment is the staff-level signal.

📚 **Start here (official docs)**

- Terraform documentation

Q64 **What are provider aliases and how do you manage multi-region or multi-account providers in modules?** Modules

🎙 **Spoken answer · ~78 s**

Provider aliases let one configuration use multiple instances of the same provider, for example two AWS regions or two accounts. I declare the default provider and then additional ones with an alias and different settings, such as region eu-west-1. A resource selects an aliased provider with the provider argument, like provider equals aws dot eu. Modules need explicit handling. By default a module inherits the default provider from its caller, but to use an aliased provider I pass it in the module block with a providers map, for instance aws equals aws dot eu. If the module needs more than one provider, such as a source and a replica region, it declares them with configuration_aliases in required_providers and the caller maps each one. Good practice is to configure providers only in the root module, not inside reusable modules, because a module with its own provider block can't be removed cleanly. For multi-account setups, an alias with assume_role selects the target account. Aliases are fixed at plan time, so I can't loop over regions with for_each on providers; I'd need to generate configuration or use separate states.

Likely follow-ups

- Why shouldn't reusable modules contain provider blocks?
- Can you for_each over providers?

Provider aliases let one configuration use multiple instances of the same provider — different regions or accounts — and pass the right one to each module/resource.

certs.eknathalabs.com

ALIASED PROVIDERS

```
provider "aws" { region = "us-east-1" }                # default
```

```
provider "aws" { alias = "eu"  region = "eu-west-1" }
```

```
resource "aws_s3_bucket" "eu" { provider = aws.eu }
```

```
# Passing aliased providers INTO a module:
```

```
module "replica" { source="./m"  providers = { aws = aws.eu } }
```

```
# Module declares it needs a provider via required_providers/configuration_aliases
```

Define multiple provider blocks with `alias`, then assign `provider = aws.eu` on resources or pass providers into modules via the `providers` argument. This is how you build multi-region replication or multi-account deployments in one config. Modules that need a specific aliased provider declare it (via `configuration_aliases`) so callers must supply it explicitly.

Footgun: a module should generally NOT define its own provider blocks — that breaks reusability and complicates destroy. Pass providers in from the root via `providers = {}`. Self-contained provider config inside modules is a known anti-pattern that causes “provider configuration not present” errors on removal.

📚 **Official docs**

- Terraform: Provider configuration (aliases)
- HCP Terraform: Recommended practices

Q65 **Explain the difference between terraform.tfvars, *.auto.tfvars, env vars, and -var precedence.**

Modules

🎙 **Spoken answer · ~71 s**

Terraform takes variable values from several sources, with a defined precedence, and mixing them up causes the question of why a value is being ignored. From lowest to highest: the default in the variable block; then environment variables named TF_VAR_ plus the variable name; then terraform.tfvars; then terraform.tfvars.json; then any files ending in auto.tfvars or auto.tfvars.json, processed in alphabetical order; and finally the var and var-file flags on the command line, which are the highest and, when several are given, the later ones win. The practical traps: a terraform.tfvars file sitting in the directory silently applies automatically, and can override what you expected; auto.tfvars files are also loaded without being named; and environment variables can differ between a laptop and CI. When debugging, terraform console and evaluating var dot name shows the resolved value. For predictability in pipelines, I pass explicit var-file flags for the environment, avoid relying on auto-loaded files, and keep secrets out of files, using environment variables or a secrets manager. Also, validation blocks catch bad values early.

Likely follow-ups

- Which source wins between TF_VAR and terraform.tfvars?
- How do you debug which value a variable resolved to?

certs.eknathalabs.com

Terraform sources variable values from several places with a defined precedence. Knowing the order prevents “why is my value being ignored?” confusion.

PRECEDENCE (LATER OVERRIDES EARLIER)

```
1. Environment variables (TF_VAR_name)
```

```
2. terraform.tfvars
```

```
3. terraform.tfvars.json
```

```
4. *.auto.tfvars / *.auto.tfvars.json (alphabetical order)
```

```
5. -var and -var-file on the CLI (highest precedence)
```

Values are applied in increasing precedence: env vars (`TF_VAR_`) first, then `terraform.tfvars`, then auto-loaded `*.auto.tfvars` (alphabetical), and finally explicit `-var`/`-var-file` flags win over everything. Variable **defaults** apply only when no source sets the value. `*.auto.tfvars` is loaded automatically (handy for environment-specific files); a plain `secret.tfvars` requires explicit `-var-file`.

Practical use: commit non-secret `*.auto.tfvars` per environment for auto-loading; pass secrets via `TF_VAR_` env vars or a `-var-file` that is not in the repo. Knowing the precedence order is a common quick-fire interview check.

📚 **Official docs**

- Terraform: Input variables (precedence)

Q66 **What is terraform graph and how do you use it to debug or optimize a configuration?** Performance

🎙 **Spoken answer · ~80 s**

Terraform graph outputs the dependency graph in DOT format, which I render with Graphviz into an image. It shows how resources, data sources, and modules depend on each other, and I can ask for the graph for a particular operation type. I use it for debugging and optimisation. For debugging order problems, it shows why Terraform creates things in a certain sequence or why it won't proceed, and I can find unexpected edges, hidden coupling, and cycles when Terraform reports a cycle error. For optimisation, it reveals long dependency chains that serialise the apply, and resources that should be independent but are linked, perhaps by an unnecessary depends_on or a broad module-level reference, which cost time because parallelism is limited by the graph. For large configurations the raw graph is unreadable, so I focus on a module or a subset, or use it as a prompt for splitting the configuration. It also helps explain a design to teammates. The graph determines what can run in parallel; the parallelism flag only caps how many at once. It shows what Terraform knows, so a missing edge reveals a dependency I need to express.

Likely follow-ups

- How do you use the graph to find a dependency cycle?
- What does a long chain in the graph tell you?

certs.eknathalabs.com

`terraform graph` emits the dependency DAG in DOT format, letting you visualize and analyze resource relationships — useful for debugging ordering and spotting unnecessary coupling.

GRAPH INSPECTION

```
terraform graph | dot -Tpng > graph.png      # render the dependency graph
```

```
terraform graph -type=plan                    # graph for a specific operation
```

```
# Look for: unexpected edges (hidden coupling), long dependency chains,
```

```
#           resources that should be parallel but are serialized
```

The graph reveals why Terraform orders things the way it does. Debugging uses: spotting an **unexpected dependency** (often an overzealous `depends_on` serializing work that could be parallel), confirming a resource depends on what you think, or understanding why a change ripples. Optimization uses: long serial chains or excessive cross-edges indicate coupling you can break (e.g. removing unnecessary `depends_on` or splitting state) to increase parallelism and reduce blast radius.

When it earns its keep: on a confusing "why is this being recreated / why does apply order surprise me" problem, visualizing the graph often makes the hidden edge obvious. Pair with removing needless `depends_on` to restore parallelism.

📚 **Official docs**

- Terraform CLI: graph

Q67 **How do you structure a Terraform CI/CD pipeline with plan and apply stages?** CI/CD

🎙 **Spoken answer · ~74 s**

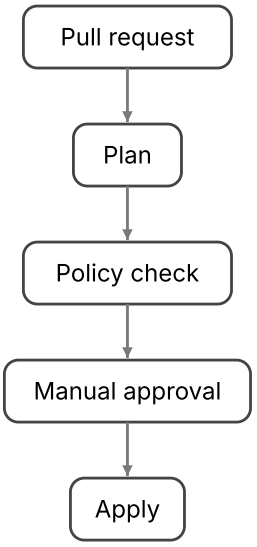
A safe Terraform pipeline separates a read-only plan stage, which runs on pull requests, from a mutating apply stage, which is gated on merge. On a pull request: format check, validate, lint and security scan, then plan with the output saved to a plan file, and the plan posted as a comment so reviewers see the concrete diff, not just the code diff. Tools such as Atlantis, HCP Terraform, or custom jobs do this. On merge to the main branch, the apply stage runs, using the exact saved plan so that what's applied is what was reviewed, with approval gates for production. Around it: remote state with locking, short-lived OIDC credentials with separate plan and apply roles, serialised runs per state, pinned versions with a committed lock file, and policy as code evaluated on the plan. Environments are promoted in order, dev, then staging, then production, each with its own state and credentials. After apply, I can run smoke tests and notify. A scheduled job runs plan to detect drift. This gives reviewable, repeatable, auditable infrastructure changes.

Likely follow-ups

- Why should the apply use the saved plan?
- How do you promote the same change across environments?

certs.eknathalabs.com

🗺 **Plan / apply pipeline**



A safe Terraform pipeline separates **plan** (read-only, on PR) from **apply** (mutating, gated on merge), so changes are reviewed as a concrete diff before touching infrastructure.

PIPELINE STAGES

```
terraform fmt -check         # fail on unformatted code
```

```
terraform validate           # syntax + internal consistency
```

```
terraform plan -out=tfplan   # on PR: produce a saved plan artifact
```

```
# post the plan as a PR comment for review (Atlantis / tf-cloud / custom)
```

```
terraform apply tfplan       # on merge: apply the EXACT reviewed plan
```

Saving the plan to a file (`-out=tfplan`) and applying that exact artifact is the key safety property: it guarantees you apply what was reviewed, not a freshly-recomputed plan that may have drifted. The PR shows the diff; merge triggers apply of the approved plan.

Interview tip: emphasize plan-on-PR / apply-on-merge with a saved plan artifact, plus required approvals for production. Atlantis and Terraform Cloud/Enterprise provide this orchestration out of the box; many teams also build it on GitHub Actions/GitLab CI.

📚 **Official docs**

- Terraform CLI: plan (saved plan files)

certs.eknathalabs.com

Q68 Why should you apply a saved plan file rather than re-running plan during apply?

CI/CD

🎙 Spoken answer · ~78 s

A bare terraform apply recomputes the plan at apply time against the current state and infrastructure, which may differ from what the reviewer approved. Between review and apply, someone may have changed a resource manually, another pipeline may have changed shared state, or the cloud's data could have changed, so the apply could do something nobody reviewed. Applying a saved plan file eliminates that gap. I run plan with the out option to capture the plan, review it, and then apply that file. Terraform applies exactly those actions, and if the state has changed since the plan was made, it refuses with a stale plan error instead of proceeding, which is the safe behaviour. That is why pipelines should produce the plan artifact in the review stage and consume it in the apply stage, rather than running auto-approve apply that recomputes. A few details: the plan file can contain sensitive values, so I store it securely with short retention; it's tied to the same Terraform and provider versions and the same working directory contents; and in CI I pass it between jobs as an encrypted artifact.

Likely follow-ups

- What happens if state changed between plan and apply of a saved plan?
- Why is a plan file sensitive?

Running a bare `terraform apply` recomputes the plan at apply time against current state — which may differ from what the reviewer saw. Applying a saved plan eliminates that gap.

```
terraform plan -out=tfplan    # capture the reviewed plan
```

```
terraform apply tfplan        # apply EXACTLY that, no recompute
```

```
# vs the unsafe pattern:
```

```
terraform apply -auto-approve # recomputes; could apply unexpected changes
```

Between review and apply, someone might change a resource manually (drift), another pipeline might alter shared state, or a data source might return new values. A bare apply would silently incorporate those into a new plan. Applying the saved plan file makes the operation deterministic and auditable: the bytes you reviewed are the bytes you apply.

Production rule: never `-auto-approve` a freshly-computed plan in prod. Either apply a saved, reviewed plan, or require a human to inspect the apply-time plan before confirming.

📚 Official docs

- Terraform CLI: plan (saved plan files)

certs.eknathalabs.com

Q69 **How do you manage Terraform across multiple environments (dev/staging/prod) in CI/CD?** CI/CD

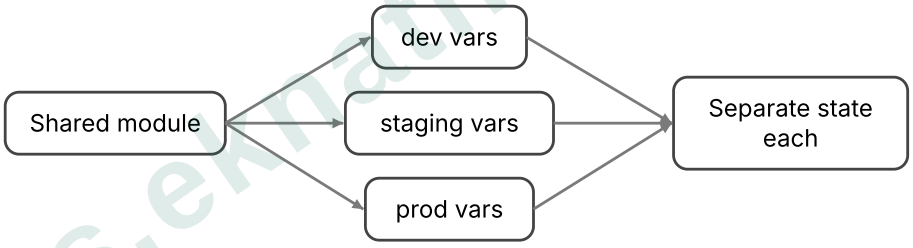
🎙 Spoken answer · ~72 s

There are two dominant patterns: directory per environment, and workspaces. Directory per environment is generally preferred for production. Each environment has its own folder, such as environments dev, staging, and prod, with its own backend configuration, its own variable values, and a main file that calls shared modules from a common modules directory. That gives hard isolation: separate state, separate credentials or accounts, different versions if needed, and a clear signal in the path of where you're working. It does mean a bit of repeated wiring, which is acceptable, and shared modules keep the logic in one place. Workspaces keep one codebase and switch state per environment. They're easy, but the code, backend, and credentials are shared, so isolation is weak and a mistake in the wrong workspace hits production. In CI/CD I run a pipeline per environment folder, promote changes from dev to staging to production, apply the same module versions, use separate OIDC roles per environment, and require manual approval for production. Tools like Terragrunt can reduce duplication across environments.

Likely follow-ups

- How do you reduce duplication across environment directories?
- How do you promote a change from staging to prod?

🗺 Multi-environment



The two dominant patterns are **directory-per-environment** and **workspaces**. Directory-per-environment is generally preferred for production isolation.

DIRECTORY-PER-ENVIRONMENT (RECOMMENDED)

```
environments/dev/main.tf      # own backend key, own tfvars
```

```
environments/staging/main.tf
```

```
environments/prod/main.tf
```

```
# shared logic lives in ../../modules/, called by each env
```

WORKSPACES (USE WITH CARE)

```
terraform workspace new staging
```

```
terraform workspace select prod
```

certs.eknathalabs.com

```
# same backend key prefix; differentiate via terraform.workspace
```

Directory-per-environment gives each env its own state file, backend config, and variable values — strong isolation, and a blast-radius boundary (a bad dev apply cannot touch prod state). Workspaces share one configuration and backend, which is lighter but risks cross-environment mistakes and makes per-env backend differences awkward. For production, most teams choose separate directories (or separate root modules) and wire the env into the pipeline.

Interview tip: state the trade-off explicitly — workspaces are convenient for ephemeral/similar environments, but directory-per-environment is safer for prod because of isolated state and independent backends. Avoid using workspaces as the *only* separation between dev and prod.

📚 **Official docs**

- Terraform: Backends

Q70 **How do you run Terraform plan automatically on pull requests and surface the diff?** CI/CD

🎙 **Spoken answer · ~78 s**

Plan on pull request turns infrastructure changes into reviewable diffs, the same way code review works for application code. In CI, the job runs init without input, then plan with no color and an output file, then terraform show to render the plan as readable text, and posts that text as a pull request comment using the platform API or a marketplace action. Tools like Atlantis and HCP Terraform do this natively: they detect the pull request, run plan, and comment the diff, and Atlantis also supports an apply command in the comment, with locking per directory so two pull requests don't plan the same state at once. Considerations: the plan job needs read-only credentials, ideally a dedicated plan role; the plan output may contain sensitive values, so I rely on sensitive marking and avoid leaking secrets in comments; large plans exceed comment size limits, so I summarise and link to the full output; and for security, plans from forked pull requests shouldn't run with secrets. I also add policy checks and cost estimation to the comment, so reviewers see risk and cost, not just the diff.

Likely follow-ups

- Why shouldn't plans from forked pull requests run with credentials?
- How do you stop two pull requests planning the same state at once?

Automating plan-on-PR turns infrastructure changes into reviewable diffs, the same way code review works for application code.

```
# In CI (GitHub Actions example flow):
```

```
terraform init -input=false
```

```
terraform plan -no-color -out=tfplan
```

```
terraform show -no-color tfplan > plan.txt   # human-readable
```

```
# post plan.txt as a PR comment via the platform API or a tool
```

certs.eknathalabs.com

Tools like **Atlantis** and **Terraform Cloud** do this natively: they detect the PR, run plan, and comment the diff directly on the PR, then let an approver trigger apply with a comment (e.g. `atlantis apply`). On bare CI you replicate it: run plan, render with `terraform show`, and post it back. The reviewer evaluates the concrete resource changes, not just the HCL.

Interview tip: the value is that reviewers see the *effect* (resources to add/change/destroy), not just the source change. Highlight that a "+/- destroy" in a plan comment is exactly the signal that catches dangerous changes before merge.

📚 **Official docs**

- Terraform CLI: plan

Q71 **A Terraform apply failed halfway through in CI. What is the state, and how do you recover?** CI/CD

🎙 **Spoken answer · ~75 s**

Terraform applies resources incrementally and records each success in state as it goes, so a mid-apply failure leaves a partial apply: some resources created and tracked, others not. State therefore reflects what succeeded. In CI I first read the error to understand the cause: a quota, a permission, a dependency, a provider bug, or a timeout. I fix that cause. Then I run plan, which shows what's left, since resources already in state are not recreated, and apply again; apply is idempotent and continues from where it stopped. I confirm with state list what's tracked. Special cases: if a resource was created at the provider but didn't make it into state, for example the CI job was killed, then plan wants to create it and fails with already exists, so I import it. If the job died and left the state lock behind, I verify nothing is running and force-unlock. If a resource is half-built and unhealthy, I use apply with replace. Then I check the final plan is clean. Prevention: reasonable timeouts, not cancelling applies abruptly, and storing logs.

Likely follow-ups

- What if the CI job is cancelled mid-apply?
- How do you handle a resource that exists but isn't in state?

Terraform applies resources incrementally and records each success in state as it goes. A mid-apply failure leaves a **partial** apply: some resources created and tracked, others not.

RECOVERY

```
terraform plan              # shows what's left to do (already-created resources are in state)
```

```
# fix the cause of the failure (quota, permission, dependency, provider bug)
```

```
terraform apply             # continues from where it stopped; idempotent
```

```
terraform state list        # confirm what's tracked
```

```
# if a resource was created but NOT recorded (rare), import it:
```

certs.eknathalabs.com

```
terraform import <addr> <id>
```

Because Terraform writes state as each resource completes, re-running `plan` after fixing the root cause shows only the remaining work — it will not recreate what already succeeded. The dangerous edge case is a resource that was created in the cloud but whose state write failed (e.g. lock/network issue at the wrong moment): then plan wants to create it again and you may hit "already exists." Resolve by importing it into state.

Locking matters here: ensure state locking (DynamoDB/GCS) is enabled so a failed CI run does not leave a dangling lock blocking the retry. `terraform force-unlock <lock-id>` clears a stuck lock — use only after confirming no apply is actually running.

📚 **Official docs**

- Terraform: State
- Terraform CLI: apply

Q72 **How do you handle Terraform provider and module version pinning in a team?** CI/CD

🎙 **Spoken answer · ~73 s**

Unpinned versions cause works-on-my-machine drift and surprise breakages when a new provider or module is released. So I pin deterministically at several levels. In the terraform block, required_version constrains the Terraform CLI, for example tilde arrow one point seven, and required_providers pins each provider with a version constraint such as tilde arrow five point forty, allowing patch and minor updates within a limit. Modules are pinned to exact versions or tight constraints in the module block, never floating on a branch. The lock file, .terraform.lock.hcl, is committed so everyone and CI use identical provider versions and checksums, generated for all platforms in use. Upgrades are deliberate: a dedicated branch bumps versions, the plan is reviewed for unexpected changes, the provider changelog is read for breaking changes, and it is tested in a lower environment first. Automation tools like Renovate or Dependabot can open update pull requests. I also pin the Terraform version in CI, through a version file or container image, so local and CI match. The aim is that infrastructure changes only when someone chooses.

Likely follow-ups

- What does the tilde-arrow constraint allow?
- How do you roll out a major provider upgrade safely?

Unpinned versions cause "works on my machine" drift and surprise breakages when a new provider/module release ships. Pin deterministically and commit the lock file.

```
terraform {
```

```
  required_version = "~> 1.7"
```

```
  required_providers {
```

```
    aws = { source = "hashicorp/aws", version = "~> 5.40" }
```

certs.eknathalabs.com

```
  }
```

```
}
```

```
module "vpc" { source = "..."  version = "5.8.1" }   # pin modules exactly
```

```
# Commit .terraform.lock.hcl — it pins provider checksums across the team
```

Use `required_version` and `required_providers` with sensible constraints (`~>` allows patch/minor but not breaking majors), pin registry **modules to exact versions**, and — critically — commit `.terraform.lock.hcl` so every developer and CI run resolves the same provider checksums. Upgrade deliberately via `terraform init -upgrade` in a reviewed PR.

Interview tip: the lock file (`.terraform.lock.hcl`) is the detail that separates juniors from seniors here — it guarantees reproducible provider versions and verifies checksums (supply-chain integrity). Always commit it.

📚 **Official docs**

- Terraform: Modules

Q73 **How do you keep secrets out of Terraform code and state?** Security

🎙 **Spoken answer · ~75 s**

There are two distinct problems: secrets in code and secrets in state. For code, never hardcode them or commit tfvars files containing them. Source them at runtime: reference a secrets manager through a data source, such as AWS Secrets Manager, mark the variables as sensitive, and pass values through environment variables, the Vault provider, or the CI system's secret store. Run secret scanning, such as gitleaks or pre-commit hooks, to catch accidents. For state, Terraform stores resource attributes, including some secrets, in plain text, and sensitive only hides them from CLI output. So I protect the state: encrypted remote backend with KMS, least-privilege access, versioning and audit logging. Better, avoid putting secrets in state at all. Let services generate and manage their own credentials, for example RDS managed master user passwords stored in Secrets Manager, IAM database authentication, or workload identity. In newer Terraform versions, ephemeral resources and write-only arguments keep secrets out of state and plan files. For anything that must exist, create the secret container in Terraform but populate the value outside it. And rotate credentials regularly.

Likely follow-ups

- How do write-only attributes keep secrets out of state?
- How can an RDS password avoid ever appearing in your Terraform state?

Two distinct problems: secrets in **code** (committed to Git) and secrets in **state** (Terraform stores resource attributes, including some sensitive values, in plaintext state).

OUT OF CODE

```
# Never hardcode. Source secrets at runtime:
```

```
data "aws_secretsmanager_secret_version" "db" { secret_id = "prod/db" }
```

certs.eknathalabs.com

```
variable "db_password" { sensitive = true }   # mark sensitive
```

```
# pass via TF_VAR_db_password env, Vault provider, or a secrets manager data source
```

OUT OF (OR PROTECTED IN) STATE

```
# State CAN contain secrets in plaintext — so protect the backend:
```

```
# - encrypt state at rest (SSE-KMS on S3), restrict bucket access tightly
```

```
# - use a backend that encrypts (Terraform Cloud, or KMS-encrypted S3)
```

Mark variables and outputs `sensitive = true` so they are redacted in plan/CLI output. But understand that **sensitive does not encrypt state** — the value is still in the state file in plaintext. The real protection for state is a locked-down, encrypted backend with strict IAM. The cleanest pattern is to reference secrets from a secrets manager at apply time rather than passing them through Terraform at all.

Critical misconception: `sensitive = true` only hides values in CLI output — it does NOT encrypt them in state. Anyone with read access to the state file sees the plaintext. Lock down and encrypt the backend, and prefer external secret references.

📚 **Official docs**

- Terraform: State

Q74 **How do you scan Terraform code for security and compliance issues?** Security

🎙 **Spoken answer · ~75 s**

I scan Terraform code with static analysis in CI, as a required gate, so misconfigurations are caught before they are applied. Tools such as tfsec, now part of Trivy, Checkov, and Terrascan check for problems like open security groups, unencrypted storage, public buckets, missing logging, and overly permissive IAM, mapped to benchmarks like CIS. For policy enforcement beyond generic rules, I use policy as code: Sentinel in HCP Terraform, or OPA and Conftest evaluated against the plan rendered as JSON with terraform show, so I can enforce organisation-specific rules such as required tags, allowed regions, approved instance types, or no public ingress. Scanning the plan, not just the source, matters because it sees the real values after variables and modules are resolved. Complementary controls: pre-commit hooks for fast feedback, secret scanning, module and provider version checks, and drift detection plus cloud-side guardrails like service control policies and AWS Config, since scanning code doesn't catch changes made outside Terraform. I tune the rule set to reduce false positives, allow documented exceptions via inline suppressions with justification, and track findings over time.

Likely follow-ups

- Why scan the rendered plan as well as the source code?
- How do you handle justified exceptions to a policy?

Static analysis catches misconfigurations (open security groups, unencrypted buckets, public access) before they are applied. Run scanners in CI as a required gate.

certs.eknathalabs.com

```
tfsec .                    # security-focused static analysis
```

```
checkov -d .               # policy-as-code scanning (CIS, etc.)
```

```
terrascan scan             # compliance scanning
```

```
trivy config .             # misconfig + vuln scanning
```

```
# Policy-as-code enforcement:
```

```
# - Sentinel (Terraform Cloud/Enterprise)
```

```
# - OPA / Conftest against the plan JSON: terraform show -json tfplan
```

Scanners like **tfsec**, **Checkov**, and **Terrascan** flag insecure patterns (e.g. `0.0.0.0/0` ingress, unencrypted volumes, public S3) directly from HCL. For enforceable guardrails, use **policy-as-code**: Sentinel (in Terraform Cloud) or OPA/Conftest evaluated against the plan JSON (`terraform show -json`) so you can block, for example, any plan that creates a publicly accessible database.

> **Interview tip:** distinguish *scanning* (tfsec/Checkov find issues) from *policy enforcement* (Sentinel/OPA block merges/applies that violate policy). Mentioning evaluating policy against the `-json` plan output shows you know how to make guardrails truly preventive, not just advisory.

> 📚 **Official docs**
>
> - Open Policy Agent documentation
> - HCP Terraform documentation

certs.eknathalabs.com

Q75 How do you manage least-privilege IAM for the Terraform execution role itself?

Security

🎙 Spoken answer · ~72 s

The Terraform execution role is a high-value target because it often has broad permissions to create and change infrastructure, so I scope it tightly. First, authentication: use short-lived federated credentials, such as GitHub Actions or GitLab OIDC assuming an AWS role, instead of static access keys, with a trust policy limited to a specific repository, branch, and environment. Second, separate roles for plan and apply: plan gets read-only access, apply gets the write permissions, and apply runs only from protected branches after approval. Third, per environment and per state: a production runner cannot touch development and vice versa, ideally in separate accounts. Fourth, scope permissions to what the configuration manages, built iteratively from AccessDenied errors or from CloudTrail with Access Analyzer. Fifth, guardrails: permission boundaries to cap the maximum privilege the role can ever grant, so it can't create an admin role for itself, and service control policies at the organisation level. Also deny dangerous actions such as disabling logging or modifying the pipeline's own role, and monitor and alert on its activity.

Likely follow-ups

- How does a permission boundary stop privilege escalation?
- How do you scope an OIDC trust policy to a single repository?

The Terraform runner often has broad permissions — making it a high-value target. Scope it tightly and prefer short-lived, federated credentials over static keys.

```
# CI authenticates via OIDC federation (no stored cloud keys):
#   GitHub Actions → AWS IAM role via OIDC, short-lived STS creds
# Scope the role to only what the config manages:
#   - per-environment roles (prod runner cannot touch dev and vice versa)
#   - permission boundaries to cap maximum privilege
#   - deny creating IAM admin / leaving the account
```

Replace long-lived access keys with **OIDC federation** so the pipeline assumes a role and gets credentials valid for minutes. Give each environment its own execution role scoped to that environment’s resources, and apply **permission boundaries** to cap what the role can ever do (so a compromised or buggy config cannot escalate to account-admin). Separate plan (read-only) from apply (write) credentials where possible.

Blast radius: a Terraform role with `*:*` is effectively account-admin and a single leaked credential compromises everything. Scope per environment, use permission boundaries, and federate so nothing long-lived is stored in CI.

📚 Official docs

- AWS provider: authentication and configuration

certs.eknathalabs.com

Q76 **What does sensitive = true do, and what are its limits?** Security

🎙 Spoken answer · ~76 s

Setting sensitive to true on a variable or output tells Terraform to redact the value in plan output, apply output, and logs, showing sensitive value instead. It also propagates: anything derived from a sensitive value is redacted too, which helps stop accidental leakage into CI logs or pull request comments. But it is output hygiene, not encryption or secret management. The value is still stored in plain text in the state file, and in saved plan files, so anyone with access to state can read it. It also doesn't stop the value from being passed to a provider and ending up in cloud resource metadata, and a user can still see it with terraform output given a specific name in some cases, or via the state JSON. So the limits are clear: it protects the screen, not the storage. To really protect secrets I encrypt and restrict the state backend, keep secrets out of state where possible by fetching them at runtime from a secrets manager, use managed credentials, and in newer Terraform use ephemeral values and write-only arguments, which are never persisted.

Likely follow-ups

- Where can a sensitive value still be read from?
- What are ephemeral values?

`sensitive = true` marks a variable or output so Terraform **redacts it from CLI/plan output and logs**. It is an output-hygiene feature, not encryption.

```
variable "api_key" { type = string  sensitive = true }
```

```
output "db_endpoint" { value = ...  sensitive = true }
```

```
# In plan output the value shows as: (sensitive value)
```

```
# Terraform propagates sensitivity: anything derived from it is also redacted
```

It prevents accidental leakage of secrets into console output, plan diffs, and CI logs, and Terraform tracks sensitivity through expressions (a value computed from a sensitive input stays redacted). Its limits: it does **not** encrypt the value in the state file (still plaintext), and a determined user can still extract it via `terraform output -json` or by reading state. It also does not stop the value from reaching the provider/API.

Interview tip: frame it precisely — "`sensitive` protects against accidental disclosure in output/logs, not against someone with state access." That precision is what interviewers are listening for; conflating it with encryption is a common red flag.

📚 Official docs

- Terraform: Variables (sensitive values)

certs.eknathalabs.com

Q77 **How would you enforce that no resource is ever created with public internet exposure?** Security

🎙 Spoken answer · ~73 s

This is a policy-as-code problem: I codify the rule and enforce it in the pipeline so violating plans cannot be applied. The rule needs to be concrete, for example no security group ingress from zero dot zero dot zero dot zero slash zero on administrative ports, no public S3 buckets without a public access block, no database with publicly accessible set to true, no public IPs on instances where forbidden. For enforcement, I render the plan as JSON with terraform show, and evaluate it with OPA using Conftest and Rego policies, or with Sentinel policies in HCP Terraform; the pipeline fails if any rule is violated, and the apply stage never runs. Static scanners like Checkov or Trivy add broad coverage on the source. Defence in depth matters, because Terraform checks only cover Terraform-managed resources: I add cloud-side guardrails such as service control policies, AWS Config rules with auto-remediation, S3 account-level block public access, and organisation policies on GCP, so that console or other tooling can't create exposure either. Exceptions go through a documented approval process.

Likely follow-ups

- Why combine Terraform policy checks with cloud-side guardrails?
- How would you handle a legitimate exception?

🗺 Policy as code



This is a policy-as-code problem: codify the rule and enforce it in the pipeline so violating plans cannot be applied.

```
# Sentinel (Terraform Cloud/Enterprise) policy example intent:
#   deny any security_group rule with cidr_blocks containing 0.0.0.0/0 on 22/3389
#   deny S3 buckets without public-access-block
# OPA/Conftest against plan JSON:
terraform show -json tfplan > plan.json
conftest test plan.json      # rego policies evaluate the planned resources
# Make the policy check a REQUIRED status check before apply
```

Generate the machine-readable plan (`terraform show -json`) and evaluate it with **OPA/Conftest** (open-source) or **Sentinel** (Terraform Cloud). The policy inspects every planned resource — e.g. reject any security group opening `0.0.0.0/0` on SSH/RDP, any database with `publicly_accessible = true`, any bucket missing a public-access block — and fails the pipeline so the apply never runs. This is preventive, not advisory.

Interview tip: the strong answer evaluates policy against the *plan* (what will actually be created), not just the HCL source — because

certs.eknathalabs.com

dynamic values and modules can produce public exposure that static source review misses. Wire it as a blocking gate.

📚 **Official docs**

- HCP Terraform documentation
- Open Policy Agent documentation

Q78 **How do you handle state file security given it may contain plaintext secrets?** Security

🎙 **Spoken answer · ~74 s**

State can hold passwords, keys, and certificates in plain text, so I treat the backend like a secrets store. For S3, I enable encryption at rest with a KMS key whose policy is restricted, deny non-TLS access through the bucket policy, block all public access, turn on versioning for recovery, and enable access logging so I can audit reads. Access is granted by least-privilege IAM to only the pipeline role and a small admin group, and the same applies to the lock table. HCP Terraform encrypts state and controls access through team permissions, which is a managed option. Beyond the backend: keep state out of Git and out of developer laptops, avoid local state files, and make sure plan files, which also contain sensitive data, are stored securely with short retention. Reduce exposure at the source by not putting secrets into state if I can avoid it: use a secrets manager, managed passwords, and ephemeral values. Finally, monitor and alert on unusual access to the state bucket, and test restore from versions so I'm confident I can recover.

Likely follow-ups

- How would you audit who read the state file?
- Why are saved plan files also sensitive?

State is sensitive: it can hold passwords, keys, and certificates in plaintext. Treat the backend like a secrets store.

```
# S3 backend hardening:
#   - SSE-KMS encryption at rest, with a restricted KMS key
#   - bucket policy: deny non-TLS access, restrict to the TF role
#   - block all public access; enable versioning + access logging
#   - DynamoDB lock table with the same tight access
# Terraform Cloud encrypts state and controls access via teams
```

Because `sensitive = true` does not encrypt state, the defense is at the backend: encrypt at rest (SSE-KMS), enforce TLS in transit, and restrict read access to only the Terraform execution role and a minimal set of operators. Enable versioning (for recovery) and access logging (for audit). Never store production state on a developer laptop or in an unencrypted, broadly-readable bucket.

Audit reminder: anyone who can read the state bucket can read every secret Terraform touched. Review who has access, encrypt with a scoped KMS key, and log access. State leakage is a full credential compromise.

certs.eknathalabs.com

📚 **Official docs**

- Terraform: State

Q79 **How do you implement drift detection in an automated way?** CI/CD

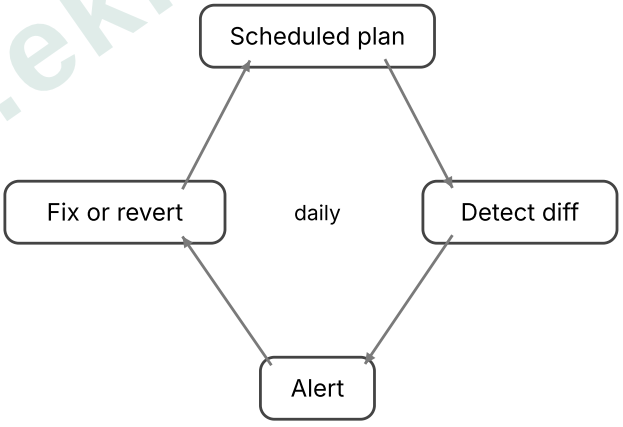
🎙 **Spoken answer · ~75 s**

Drift detection means regularly checking whether real infrastructure still matches the code and state, and alerting when it doesn't. The automated way is a scheduled pipeline, for example nightly, that runs terraform plan with the detailed-exitcode flag. Exit code zero means no changes and no drift, exit code two means there are changes, so drift or unapplied code, and exit code one means an error. The job alerts on code two, posting to Slack or opening a ticket with the plan output attached, and treats code one as a pipeline failure. HCP Terraform has built-in drift detection, health assessments, and notifications. I run it per state, per environment, and prioritise production. Because plan includes refresh, it picks up out-of-band changes. To tell drift from pending code changes, I run it against the main branch, where all merged code should already be applied. Tools like driftctl or cloud config services also find unmanaged resources that Terraform never tracked. Then there must be a process to respond: decide whether to revert the change or codify it, and prevent recurrence by tightening console access.

Likely follow-ups

- How do you distinguish drift from unapplied code changes?
- What does exit code 2 mean in plan?

🗺 **Drift detection**



Drift is when real infrastructure diverges from state (manual console changes, out-of-band tooling). Automated detection runs plan on a schedule and alerts on non-empty diffs.

```
# Scheduled CI job (e.g. nightly):
```

```
terraform plan -detailed-exitcode
```

```
#   exit 0 = no changes (no drift)
```

certs.eknathalabs.com

```
#   exit 2 = changes present (DRIFT) → alert
```

```
#   exit 1 = error
```

```
# Terraform Cloud has built-in drift detection + notifications
```

The key flag is `-detailed-exitcode`: it returns 2 when the plan is non-empty, which a scheduled job interprets as drift and routes to an alert (Slack/PagerDuty). Run it read-only (plan, never apply) so detection never mutates anything. Terraform Cloud/Enterprise offers this natively with health assessments. On detection, a human decides whether to revert the drift (apply) or absorb it into code.

Interview tip: stress that drift detection should be *read-only and scheduled*, and that the response is a decision, not an automatic apply — sometimes the manual change was a legitimate emergency fix that should be codified rather than reverted.

📚 **Official docs**

- Terraform CLI: plan
- Infrastructure drift (Terraform: plan -refresh-only)

Q80 **How do you test Terraform code? What are unit vs integration approaches?** CI/CD

🎙 **Spoken answer · ~75 s**

Terraform testing has layers, each catching a different class of error. The cheapest are static checks: fmt, validate, TFLint for provider-specific best practices, and security scanners like Checkov. Next are plan-level, unit-style tests with the native test framework, available since Terraform one point six: tftest files with run blocks and assert conditions. With command set to plan, and mocked providers in newer versions, I can verify that variables produce the expected resources, names, and tags, without creating anything, so these are fast and free. Then integration tests apply the module for real in a sandbox account, assert that the resulting infrastructure works, for example by calling an endpoint, and then destroy it. That can be the native framework with command apply, or Terratest in Go. These give the most confidence but are slower, cost money, and need robust cleanup. Policy tests with OPA or Sentinel check compliance against the plan. I run static and plan tests on every pull request, integration tests on module changes or on a schedule, and keep test environments isolated and tagged for automatic cleanup.

Likely follow-ups

- What is the difference between a plan-based and an apply-based test?
- How do you prevent leaked resources from failed integration tests?

Terraform testing spans static checks, plan assertions, and real-resource integration tests — each catching a different class of error.

LAYERS OF TESTING

```
terraform fmt -check / validate        # static: syntax, formatting
```

```
tflint                                 # lint: provider-specific best practices
```

certs.eknathalabs.com

```
terraform test                          # native test framework (.tftest.hcl), since TF 1.6
```

```
# plan-based assertions (validate values without creating resources)
```

```
# integration: Terratest (Go) — apply to a sandbox, assert, then destroy
```

```
checkov / tfsec                         # security policy tests
```

Native `terraform test` (1.6+) runs `.tftest.hcl` files that can do plan-only assertions (cheap, no resources) or full apply/destroy against a real sandbox. **Terratest** (Go) is the mature integration approach: it applies real infrastructure to a throwaway account, makes assertions (HTTP checks, API calls), and tears it down. Combine with static analysis and policy scanning for full coverage.

> **Interview tip:** mention the testing pyramid for IaC — lots of cheap static/plan checks, fewer expensive integration tests that actually provision. Native `terraform test` (1.6+) is the current, in-the-box answer; Terratest is the battle-tested integration option.

> 📚 **Official docs**
>
> - Terraform: Tests

Q81 **How do you safely roll out a change to shared modules used by many teams?** CI/CD

> 🎙 **Spoken answer · ~73 s**
>
> A shared module is a published dependency, so changing it can break every consumer. I manage it like any released software. Use semantic versioning with tags or a private registry: a breaking change bumps the major version, so consumers opt in deliberately; new compatible features are minor; fixes are patches. Consumers pin versions with constraints like tilde arrow five point zero, so they pick up compatible updates only when they choose. Before release, the module has automated tests and examples, a changelog that calls out breaking changes, and a migration guide, using moved blocks for resource renames so upgrades don't recreate resources. Roll out gradually: release a new version, upgrade a canary consumer or a low-risk environment first, review the plan for unexpected changes, and then let other teams adopt on their schedule, with a deprecation window for old versions. Tools like Renovate propose version bumps as pull requests so the plan is reviewed per consumer. I keep the interface stable, avoid removing variables suddenly, and communicate changes clearly to the teams using it.
>
> **Likely follow-ups**
>
> - How do moved blocks help module upgrades?
> - How do you handle a security fix that all consumers must adopt quickly?

A shared module is a published dependency — changing it can break every consumer. Version it semantically and roll out gradually.

```
# Semantic versioning of the module + a registry (or git tags):
```

```
module "vpc" { source = "app.terraform.io/org/vpc/aws"  version = "~> 5.0" }
```

certs.eknathalabs.com

```
# Breaking change → bump MAJOR (6.0.0); consumers opt in deliberately
```

```
# Non-breaking → MINOR/PATCH; ~> constraint lets consumers pick it up
```

Treat the module like any released library: **semantic versioning** (major = breaking, minor = feature, patch = fix), published to a registry or pinned by git tag. Consumers pin versions, so a breaking change does not auto-propagate — they upgrade on their own schedule, testing in dev first. Provide a changelog and migration notes for major bumps. Avoid `source` pointing at a mutable branch (e.g. `main`), which silently breaks consumers.

Anti-pattern: sourcing a shared module from a moving git branch. A merge to that branch instantly changes every consumer's infrastructure plan with no opt-in. Always pin to immutable versions/tags.

📚 **Official docs**

- Terraform: Publishing modules (registry & versioning)
- HCP Terraform: Recommended practices

Q82 **How do you manage Terraform Cloud / Enterprise workspaces and RBAC at scale?** Security

🎙 **Spoken answer · ~75 s**

At scale, HCP Terraform or Enterprise needs structure, so I manage it as code and by team. Workspaces are typically one per service and environment, grouped into projects that mirror teams or business units. Access is through teams mapped to the identity provider via SSO, with permissions per workspace or project: read, plan, write, and admin; developers can usually plan and request applies, while production applies need approval. Variable sets share common variables and credentials across many workspaces centrally, so they're defined once, and cloud credentials use dynamic provider credentials via OIDC instead of stored keys. Policy sets with Sentinel or OPA are enforced organisation-wide, for example no public databases and required tags. Run tasks integrate scanning and cost estimation. To avoid click-ops, I manage the platform itself with the tfe provider: workspaces, teams, variable sets, and policies are all defined in code, reviewed, and versioned. Other practices: VCS-driven runs with sensible triggers, no-code modules or a private registry for self-service, notifications for failures and drift, and regular access reviews. Cost is also a factor, since licences scale with managed resources.

Likely follow-ups

- Why manage the Terraform platform with the tfe provider?
- What are dynamic provider credentials?

At scale, you manage many workspaces with team-based access, variable sets, and policy guardrails — ideally itself defined as code.

```
# Workspace-per-(service x environment); group by project
```

```
# Teams → workspace permissions (read / plan / write / admin)
```

```
# Variable sets: share common vars/creds across workspaces centrally
```

certs.eknathalabs.com

```
# Sentinel policy sets enforced org-wide (e.g. no public DBs, required tags)
```

```
# Manage TFC itself with the tfe provider (workspaces/teams as code)
```

Structure workspaces by service × environment, assign access via **teams** (least privilege: most users get plan, fewer get apply, admin is rare), and use **variable sets** to distribute shared credentials/config without copy-paste. Enforce **Sentinel policy sets** org-wide for guardrails. Crucially, manage TFC configuration itself with the `tfe` Terraform provider so workspace/team/policy setup is reviewable code, not console clicks.

Interview tip: "manage the platform with the platform" — defining TFC workspaces, teams, and policies via the `tfe` provider is the staff-level move. It makes access and guardrails auditable and prevents console drift.

📚 **Official docs**

- Kubernetes: RBAC
- HCP Terraform documentation

Q83 **A long-running apply is stuck holding the state lock. How do you handle it safely?** CI/CD

🎙 **Spoken answer · ~78 s**

A stuck lock blocks every other run, but the key rule is to confirm whether an apply is genuinely still running before force-unlocking, because unlocking a live apply allows concurrent writes and can corrupt state. When I hit the lock error, it shows the lock ID, who holds it, the operation, and when it was created. I investigate: is there a CI job or a person actually running an apply right now? Check the pipeline run status and logs, and ask the person named. A long-running apply, say a database creation, can legitimately hold the lock for a long time, so age alone doesn't mean it's stale. If the process is still running, I wait, or cancel it properly through CI if it must stop, letting Terraform release the lock and write state on interrupt. Only when I have confirmed that nothing is running, for example the runner crashed, do I use terraform force-unlock with the lock ID. After that I run a plan to check the state is consistent, and look for partially created resources. To prevent it, I set sensible job timeouts and handle cancellation gracefully.

Likely follow-ups

- How do you cancel an apply without leaving a stale lock?
- What would you check after force-unlocking?

A stuck lock blocks all other runs. The safe path is to first confirm whether an apply is genuinely still running before force-unlocking — force-unlocking a live apply corrupts state.

```
terraform plan              # will report: Error acquiring state lock + lock ID + who/when
```

```
# Investigate: is a CI job / person actually still applying?
```

```
#   check CI run status, the lock's 'Who' and 'Created' fields
```

```
# Only if you've CONFIRMED no apply is running:
```

certs.eknathalabs.com

```
terraform force-unlock <LOCK_ID>
```

The lock error shows the lock ID, who acquired it, and when — use that to investigate. If a CI job died mid-apply (crash, cancelled runner), the lock can be orphaned and `force-unlock` is appropriate. But if an apply is truly still in progress, force-unlocking lets a second apply run concurrently against the same state → corruption. So confirm first, unlock second.

Never force-unlock blindly. Two concurrent applies on one state file can corrupt it irreparably. Verify no apply is actually running (CI status, lock metadata) before `force-unlock`. Set reasonable CI timeouts so dead jobs release locks.

📚 **Official docs**

- Terraform: State
- Terraform: State locking

Q84 **How do you handle credentials and provider authentication securely in Terraform?** Security

🎙 **Spoken answer · ~72 s**

Provider authentication should use short-lived credentials supplied by the environment, never keys hardcoded in HCL, which end up in Git and in state. So the provider block contains only things like the region, and the credentials come from outside. For local development, I use SSO logins or tools such as aws-vault that issue short-lived tokens. In CI, I use OIDC federation: the pipeline gets a token from its identity provider, exchanges it for temporary credentials by assuming a role, and the role's trust policy restricts which repository, branch, and environment may assume it. On cloud-hosted runners or Kubernetes, instance profiles, IRSA, or workload identity provide credentials with nothing stored. I separate roles for plan, which is read-only, and apply, scope them per environment, and use assume_role to reach target accounts. For third-party providers that need API tokens, I take them from a secrets manager or the CI secret store, passed as environment variables and marked sensitive. I also make sure logs don't print credentials, rotate anything long-lived, and audit role usage with CloudTrail.

Likely follow-ups

- How does an OIDC trust policy limit which pipeline can assume the role?
- Why use separate roles for plan and apply?

Provider auth should use short-lived, environment-provided credentials — never hardcoded keys in HCL, which end up in Git and state.

```
# DON'T:
```

```
provider "aws" { access_key = "AKIA..."  secret_key = "..." }  # leaked to Git
```

```
# DO — let the environment provide creds:
```

```
provider "aws" { region = "us-east-1" }   # uses env / instance role / OIDC
```

```
# Local dev: SSO / aws-vault / short-lived STS
```

certs.eknathalabs.com

```
# CI: OIDC federation → assume-role → short-lived STS creds (nothing stored)
```

Hardcoding credentials in the provider block commits them to Git and risks them landing in state — a classic leak. Instead let the provider pick up credentials from the environment: an instance/pod role, AWS SSO + `aws-vault` locally, and OIDC federation in CI so the pipeline assumes a role for minutes-long STS credentials. This keeps nothing long-lived anywhere in code or config.

Interview tip: the modern best-practice answer is OIDC federation for CI (no stored secrets) plus instance/pod roles or SSO for humans. Hardcoded keys in a provider block is an instant red flag in a security-minded interview.

📚 **Official docs**

- OAuth 2.0 / OIDC overview (OpenID)

Q85 **Terraform plan wants to destroy and recreate a production database. How do you prevent disaster?** Production

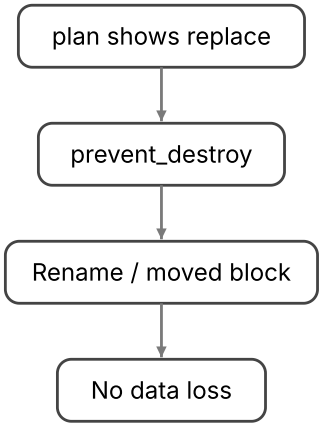
🎙 **Spoken answer · ~75 s**

A destroy and recreate on a stateful resource means data loss and downtime, so first I understand why Terraform wants it, then I protect against it. In the plan, the line marked forces replacement points at the attribute: it's often an immutable attribute such as a name, availability zone, or engine setting, or a provider that treats a change as ForceNew. The plan JSON shows the same. I never apply that plan. Options: revert the change that triggered it; make the change in a way that doesn't force replacement, such as a supported in-place modification or a different attribute; if I'm renaming or refactoring, use moved blocks so the address changes without recreation; or if replacement is truly intended, plan a migration with a snapshot, restore into the new instance, and cut over, and only then remove the old. Protective measures: lifecycle prevent_destroy on critical resources makes the plan fail instead, deletion protection at the cloud level, automated backups and snapshots, final snapshot settings, and a policy check in CI that blocks plans destroying protected resource types without explicit approval.

Likely follow-ups

- What does 'forces replacement' in the plan mean?
- How does prevent_destroy differ from cloud-side deletion protection?

🗺 **Safe replace of a database**



certs.eknathalabs.com

A destroy/recreate on a stateful resource means data loss and downtime. First understand *why* Terraform wants to replace it, then protect against it.

DIAGNOSE THE FORCED REPLACEMENT

```
terraform plan          # look for '# forces replacement' next to an attribute
```

```
# common causes: changing an immutable attribute (engine_version path,
```

```
#   availability_zone, name), provider treating a change as ForceNew
```

```
terraform plan -json | jq '.resource_changes[] | select(.change.actions[]=="delete")'
```

PROTECT IT

```
lifecycle { prevent_destroy = true }   # hard stop: apply errors instead of destroying
```

```
lifecycle { ignore_changes = [<attr>] } # if the change is spurious/managed elsewhere
```

```
terraform state mv ...                  # if it's an addressing change, not a real replace
```

Read the plan to find the attribute that "forces replacement" — often an immutable field changed inadvertently, or a provider quirk. Add `prevent_destroy = true` to critical stateful resources as a safety net so any plan that would destroy them fails loudly. If the change is spurious, `ignore_changes` or fixing the config avoids the replacement. If it is merely a rename/move, `terraform state mv` updates addressing without recreating.

Always run `prevent_destroy` on production databases, stateful volumes, and anything holding data. It converts a catastrophic silent destroy into a loud apply failure that forces a human decision.

📚 **Official docs**

- Terraform: lifecycle meta-argument
- Terraform CLI: plan

certs.eknathalabs.com

Q86 **How do you import existing (manually-created) infrastructure into Terraform management?**

Production

🎙 Spoken answer · ~75 s

Importing is how I bring already-running, manually created resources under Terraform without recreating them. The modern way, since Terraform one point five, is the declarative import block: I specify the target address and the real resource ID, run plan with generate-config-out to have Terraform scaffold the HCL for it, review and clean up the generated code, then apply, which adopts the resource into state. It's reviewable and repeatable in a pull request. The classic approach is the CLI command terraform import with the address and ID, after which I hand-write the matching configuration, and use plan until it shows no changes. The key point in both cases: import only updates state, not code, so the configuration must match reality or the next plan will show changes, possibly destructive ones. I work in small batches, import the resource and its dependencies, and check the plan is clean before moving on. For large estates, tools like Terraformer or provider-specific exporters can generate configuration in bulk, but the output needs cleanup. I take a state backup first, and import in a lower-risk order.

Likely follow-ups

- Why must the code match reality after an import?
- How do you import a large existing estate?

Brownfield adoption: bring already-running resources under Terraform without recreating them. Modern Terraform supports both CLI import and declarative import blocks.

DECLARATIVE IMPORT (TF 1.5+, PREFERRED)

```
import {
  to = aws_instance.web
  id = "i-0abc123"
}
terraform plan -generate-config-out=generated.tf  # scaffold the HCL
terraform apply                                    # adopt into state
```

CLI IMPORT (CLASSIC)

```
terraform import aws_instance.web i-0abc123
# then hand-write matching HCL until plan shows no changes
```

Import blocks (1.5+) are declarative and can even **generate** starter configuration with `-generate-config-out`, which dramatically speeds up adopting large existing estates. The classic `terraform import` imports one resource into state, after which you write HCL that matches reality until `plan` is clean. The goal in both cases: a green plan (no changes) proving config matches the real resource.

certs.eknathalabs.com

Interview tip: mention import blocks + `-generate-config-out` as the current approach — it shows you are on modern Terraform. The success criterion is always "plan shows no changes," meaning your code now faithfully represents the imported resource.

📚 **Official docs**

- Terraform: Import

Q87 **Two resources in state need to swap names / move into a module. How do you do it without recreation?** Production

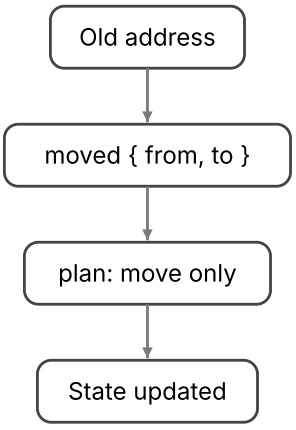
🎙 **Spoken answer · ~78 s**

Renaming a resource or moving it into a module changes its address, and if I don't tell Terraform, it sees the old address as something to destroy and the new one as something to create. The solution is to record the move. The preferred way is a moved block in the code: moved with from equal to the old address and to equal to the new address, for example moving aws_instance.web into module.compute.aws_instance.web. On the next plan, Terraform shows a move rather than a destroy and create, and every teammate and CI run applies it automatically, which makes it reviewable and repeatable. The imperative alternative is terraform state mv, which edits state directly; it works, but it's a manual step that must be done on each state, isn't reviewed, and is easy to forget. For swapping the names of two resources, I use moved blocks for both, via a temporary name if needed so addresses don't collide. I always run plan afterwards to confirm it shows only moves and no replacements, and I keep moved blocks for a while so other consumers of a shared module upgrade cleanly.

Likely follow-ups

- Why are moved blocks better than state mv for teams?
- How do you swap the names of two resources?

🗺 **moved block**



Renaming a resource or moving it into a module changes its *address*. Without telling Terraform, it sees the old address as "destroy" and the new as "create." `moved` blocks and `state mv` avoid that.

DECLARATIVE MOVED BLOCKS (PREFERRED)

```
moved {
```

certs.eknathalabs.com

```
  from = aws_instance.web
```

```
  to   = module.compute.aws_instance.web
```

```
}
```

```
# committed in code; plan shows a move, not destroy+create
```

IMPERATIVE STATE MV

```
terraform state mv aws_instance.web module.compute.aws_instance.web
```

```
terraform state mv aws_instance.a aws_instance.tmp   # for swaps, via a temp address
```

```
terraform state mv aws_instance.b aws_instance.a
```

```
terraform state mv aws_instance.tmp aws_instance.b
```

`moved` blocks (1.1+) record the address change declaratively in code, so every collaborator and CI run applies the move consistently and reviewably — the plan shows a move with no resource churn. `terraform state mv` is the imperative equivalent for one-off surgery; swapping two names requires a temporary address to avoid a collision.

Interview tip: prefer `moved` blocks over `state mv` — they are versioned, reviewed, and self-documenting, whereas manual `state mv` is an out-of-band operation others cannot see. Both avoid the destroy/recreate that a naive rename would trigger.

📚 **Official docs**

- Terraform: lifecycle meta-argument

certs.eknathalabs.com

Q88 **A resource was deleted outside Terraform (in the console). What happens on next plan?** Production

🎙 Spoken answer · ~77 s

Terraform compares state with reality when it plans. A resource deleted in the console is still in state but gone in the cloud, and the refresh step discovers that it no longer exists. Terraform treats it as drift and plans to recreate it, so the plan shows a create, and apply brings it back to match the configuration. That's usually what I want if the deletion was accidental. If the deletion was intentional, then I should reflect that in code instead: remove the resource from the configuration and, if it's still in state, either apply so Terraform tries to destroy it, which may harmlessly confirm it's gone, or run terraform state rm to drop it from state. Watch for side effects: the recreated resource may get new IDs or addresses, and dependent resources can change, and for stateful resources such as databases the data is gone, so recreation gives an empty resource, and I'd need to restore from backup. This is another reason to restrict console write access in production, use deletion protection, and run scheduled drift detection so such changes are found quickly.

Likely follow-ups

- What happens to dependent resources when the deleted one is recreated?
- When would you use state rm instead of letting Terraform recreate?

Terraform compares state against reality. A resource deleted out-of-band is still in state but gone in the cloud — Terraform detects this as drift and plans to recreate it.

```
terraform plan          # shows the resource will be CREATED (it's missing in reality)
```

```
terraform apply         # recreates it to match config
```

```
# if you intended the deletion, remove it from config + state instead:
```

```
terraform state rm <addr>   # drop from state (stop managing) ...
```

```
# ... and delete the resource block from HCL so plan stays clean
```

On refresh, Terraform notices the real resource is absent and reconciles by planning to **recreate** it (config still declares it). If the deletion was a mistake, apply restores it. If the deletion was intentional, the correct action is to remove the resource from both the HCL and the state (`state rm`) so Terraform stops expecting it — otherwise it will keep trying to recreate it on every run.

Interview tip: the key insight is that state + config define desired state; reality is reconciled toward them. "Deleted in console" → Terraform recreates, because as far as it knows you still want it. This is also why drift detection matters.

📚 Official docs

- Terraform: lifecycle meta-argument

certs.eknathalabs.com

Q89 **How do you perform a zero-downtime change to a load-balanced fleet with Terraform?** Production

🎙 Spoken answer · ~75 s

For a zero-downtime change to a load-balanced fleet, the principle is to have new capacity healthy and serving before the old capacity is removed. In Terraform that means using create_before_destroy in the lifecycle block of the replaced resources, such as a launch template or instance, so the replacement is created first. With an Auto Scaling Group, I use the instance refresh feature or a rolling update strategy: new instances launch from the updated launch template, pass load balancer health checks, register with the target group, and only then are old instances drained and terminated, with minimum healthy percentage controlling how much capacity stays in service. Connection draining through the deregistration delay lets in-flight requests finish. Health checks must be accurate, otherwise instances join the pool before they're ready. For bigger changes, blue-green is another option: create a second ASG or target group, shift traffic with weighted rules, and then retire the old. I also make sure names don't conflict, using name prefixes, keep capacity headroom for the overlap, and test the process in staging, watching error rates during rollout.

Likely follow-ups

- What does minimum healthy percentage control?
- Why do resource names need prefixes with create_before_destroy?

Use `create_before_destroy` lifecycle plus health-checked target registration so new instances are serving before old ones are removed.

```
resource "aws_launch_template" "app" { ... }   # change here triggers rollout
```

```
lifecycle { create_before_destroy = true }      # new resource up before old torn down
```

```
# With ASG + LB: instance refresh / rolling update strategy
```

```
#   - new instances launch, pass health checks, register to LB
```

```
#   - only then are old instances drained and terminated
```

The `create_before_destroy` lifecycle rule inverts Terraform's default (destroy-then-create), so the replacement exists and is healthy before the original is removed — essential for anything behind a load balancer. For autoscaling fleets, pair it with the platform's rolling/instance-refresh mechanism and connection draining so traffic shifts only to instances that pass health checks, achieving zero downtime.

Gotcha: `create_before_destroy` requires that the new and old can coexist — unique names, no hard capacity/quota limit of 1, no conflicting fixed identifiers (e.g. a static EIP or a name that must be unique). Plan for two copies existing momentarily.

📚 Official docs

- Cloudflare Learning: Load balancing
- Argo CD: ApplicationSet

certs.eknathalabs.com

Q90 **How do you structure Terraform to limit blast radius across a large infrastructure estate?** Production

🎙 Spoken answer · ~72 s

Blast radius is bounded by how state is partitioned, so I split the estate into many small, independent states instead of one large one. A typical partition is by environment, then by layer, then by service. Networking holds the VPC and subnets: it changes rarely but has high impact. Data holds databases, isolated and protected with prevent_destroy. Compute and application stacks change often and have low individual impact. Each state has its own backend key, its own pipeline, and its own permissions, so a bad apply or a corrupted state affects only that slice, and teams can apply in parallel without lock contention. Layers connect through outputs read via remote state data sources, or better through data lookups or a parameter store, to keep coupling loose. Additional controls reduce the radius further: separate cloud accounts per environment, least-privilege roles per state, policy checks, and approval gates for the high-impact layers. The trade-off is more states to manage, so I standardise structure and automate orchestration, using Terragrunt or pipelines that apply layers in order.

Likely follow-ups

- Why separate the database layer from compute?
- What is the cost of having many small states?

Blast radius is bounded by how state is partitioned. Many small, independent state files limit the damage any single apply (or corrupted state) can do.

PARTITIONING STRATEGY

```
# Separate state per: environment x layer x service
#   network/ (VPC, subnets)    — changes rarely, high blast radius
#   data/ (databases)          — isolated, prevent_destroy
#   compute/ (apps)            — changes often, low blast radius
# Connect layers via remote state data sources or published outputs:
data "terraform_remote_state" "network" { backend = "s3" config = {...} }
```

Rather than one monolithic state for everything, split by environment, layer (network/data/compute), and service. Each gets its own state file, so a mistake or corruption in the compute layer cannot affect the network or data layers, and applies are smaller and faster. Layers reference each other's outputs via `terraform_remote_state` data sources. This also enables independent change cadences — stable layers (network) change rarely; volatile layers (apps) change often.

Interview tip: "one giant state file" is the anti-pattern to call out — slow plans, huge blast radius, lock contention across teams. The senior answer is partitioned state by environment/layer/service with remote-state references between them.

certs.eknathalabs.com

📚 **Official docs**

- AWS: Disaster recovery & availability concepts (Fault isolation boundaries)
- AWS Builders' Library: Workload isolation using shuffle sharding

---

Q91 **A terraform apply is taking 30+ minutes and timing out. How do you diagnose and speed it up?**

Production

🎙 **Spoken answer · ~75 s**

When an apply takes thirty minutes or more and times out, I diagnose before changing anything. Slow applies usually come from a state that's too large, so every operation refreshes thousands of resources; excessive provider API calls or throttling; serialised dependencies that prevent parallelism; or genuinely slow resources like databases or clusters. I use debug logging to see where the time goes and look at the plan's resource count and dependency graph. Tuning options: skip refresh with refresh equals false for the plan when drift isn't a concern at that moment; raise the parallelism flag from ten, watching for API rate limits; and use target only for triage. The structural fix is to partition the configuration into smaller states so each run is quick, and to remove unnecessary depends_on that serialise work. For slow resources I check the timeouts block, since some resources need longer than the default timeouts, and for pipeline timeouts I raise the job limit if the work is legitimately long. Also check provider throttling, which shows as retries in the logs, and reduce data source calls.

Likely follow-ups

- How do you find the slowest resources in an apply?
- When is a longer resource timeout the right fix?

Slow applies usually come from too-large state, excessive provider API calls, or serialized dependencies. Diagnose, then partition or tune parallelism.

```
terraform plan -refresh=false        # skip refresh if drift detection isn't needed now
```

```
terraform apply -parallelism=20      # raise from default 10 (watch API rate limits)
```

```
terraform apply -target=module.x     # apply a subset (use sparingly, for triage)
```

```
TF_LOG=DEBUG terraform apply         # see where time goes (slow provider calls?)
```

First, the structural fix: a 30-minute apply usually means the state is too big — split it into smaller, independent states (see blast-radius partitioning). Tactically, `-refresh=false` skips the per-resource refresh round-trips when you do not need fresh drift data, raising `-parallelism` speeds independent resource operations (bounded by provider API rate limits), and `TF_LOG=DEBUG` reveals whether a specific slow provider/data source is the bottleneck.

Interview tip: lead with the root cause (state too large → partition it) and present `-refresh=false`, `-parallelism`, and targeted applies as tactical aids. `-target` is a triage tool, not a normal workflow — say so, since over-using it hides real dependency problems.

certs.eknathalabs.com

📚 **Official docs**

- Terraform CLI: apply

Q92 **How do you safely deprecate and destroy an environment without leaving orphaned resources or breaking dependents?** Production

🎙 **Spoken answer · ~75 s**

Decommissioning an environment should be planned like a controlled change, not a single destroy command. First I check dependencies: other states that read this one via remote state, other teams using shared resources, DNS records, peering connections, and anything outside Terraform that references it. I preview with plan using the destroy flag to see exactly what will go. Protected resources are an intentional barrier: prevent_destroy and deletion protection must be lifted deliberately for things I truly intend to remove, and I confirm I have final backups or snapshots of any data I might need, and agree on retention. Then run the destroy in order, usually in the reverse of the layer order, apps first, then data, then networking, each through the pipeline with approval. After destroy I verify there are no orphans in the cloud: leftover volumes, snapshots, load balancers, network interfaces, or log groups that Terraform didn't manage, using tags and cost reports, and I remove the state and backend entries, pipeline configuration, and access. Finally, I communicate with stakeholders and keep a record of what was removed and why.

Likely follow-ups

- Why destroy in reverse layer order?
- How do you find orphaned resources after a destroy?

Teardown must account for cross-state dependencies and protected resources. Plan it like a controlled decommission, not a single `destroy`.

```
terraform plan -destroy           # preview everything that will be destroyed
```

```
# check for dependents in OTHER states (remote_state references)
```

```
# temporarily lift prevent_destroy on resources you truly intend to remove
```

```
terraform destroy                 # tear down this state's resources
```

```
# verify no orphans: cloud-side resources not in any state (cost scanners help)
```

Run `plan -destroy` first to see the full set of resources targeted. Critically, check whether other state files reference this environment (via `terraform_remote_state`) — destroying a shared VPC that another state depends on breaks it. Resources with `prevent_destroy` will block destroy until you deliberately lift it. After teardown, reconcile against the cloud bill / a resource scanner to catch **orphans** — things created out-of-band or left by partial failures that Terraform never tracked.

Two failure modes: (1) breaking another environment that depended on a shared resource — check remote-state references first; (2) orphaned resources still costing money because they were never in state — reconcile against the cloud after destroy.

certs.eknathalabs.com

📚 **Start here (official docs)**

- Terraform documentation

Q93 **Terraform state has become corrupted or out of sync with reality. How do you recover?** Production

🎙 **Spoken answer · ~77 s**

State corruption, such as a truncated file, a bad manual edit, or conflicting writes, is serious but recoverable if I've prepared. First stop others from running by locking or pausing pipelines. Back up the current state with state pull so I preserve evidence. Then the recovery options. The first choice is restoring a known-good version from backend history: with S3 versioning I list the object versions and retrieve the last good one, and HCP Terraform has state versions. After restoring, I validate it with state list and a plan, expecting differences only where infrastructure changed since the backup. Resources created after the backup but missing from state show as new creations that would fail as already existing, so I import them; resources deleted since then show as needing recreation, which I examine. If the state is only partly wrong, I repair surgically: state rm for stale entries, import for untracked resources, state mv for address issues. Throughout, I avoid applying until the plan looks right. Afterwards I find the cause, such as concurrent writes or manual edits, and strengthen versioning, locking, and access controls.

Likely follow-ups

- How do you validate a restored state before applying?
- What would the plan show if the restored state is older than reality?

State corruption (truncated file, bad manual edit, conflicting writes) is serious but recoverable via backend versioning and surgical state commands.

```
# 1. Restore a known-good version from backend history:
```

```
aws s3api list-object-versions --bucket <b> --prefix <key>
```

```
aws s3api get-object --version-id <v> ... terraform.tfstate
```

```
# 2. Or repair surgically:
```

```
terraform state pull > backup.tfstate     # always back up first
```

```
terraform state list                       # see what's tracked
```

```
terraform state rm <bad-addr>              # remove a broken entry
```

```
terraform import <addr> <id>               # re-add a missing/real resource
```

```
terraform state push <fixed.tfstate>       # push a corrected state (last resort)
```

certs.eknathalabs.com

If you enabled backend versioning (you should have), the fastest recovery is restoring the last good state version. For targeted issues, always `state pull` a backup first, then use `state rm` / `import` to bring state back in line with reality, verifying with a clean `plan`. `state push` of a hand-corrected file is the last resort and must be done with locking and care.

Prevention is the real answer: S3/GCS versioning + locking makes corruption a quick rollback instead of a crisis. Always `state pull` a backup before any manual state surgery, and never hand-edit state JSON directly.

📚 **Official docs**

- Terraform: State
- Argo CD documentation

Q94 **How do you handle a provider upgrade that introduces breaking changes across many resources?** Production

🎙 **Spoken answer · ~79 s**

Major provider upgrades can change resource schemas, defaults, or force replacements, so I upgrade deliberately. I start in a dedicated pull request that bumps the version constraint, for example from tilde arrow five to tilde arrow six, and run init with upgrade. Before that I read the provider's upgrade guide and changelog for breaking changes, deprecated arguments, and changed defaults. Then I run plan against each state and review every change carefully, especially anything marked as forcing replacement, since a stateful resource being replaced is a red flag. Fixes may include updating arguments, adding explicit values to preserve old defaults, or adding moved blocks. I roll it out environment by environment: apply to a development or sandbox first, then staging, then production, with the same reviewed process. For a huge estate with many states, I upgrade states in stages by team or layer rather than all at once, and the lock file is updated and committed. I keep the ability to pin back to the previous version and make sure the state is backed up before the first apply, since some upgrades make state changes that older versions can't read.

Likely follow-ups

- Why can't you always downgrade a provider after applying?
- How do you preserve old defaults after an upgrade?

Major provider upgrades can change resource schemas, defaults, or force replacements. Upgrade deliberately in a controlled, reviewed, environment-by-environment rollout.

```
# Bump the constraint in a dedicated PR:
```

```
required_providers { aws = { version = "~> 6.0" } }   # was ~> 5.0
```

```
terraform init -upgrade
```

```
terraform plan          # carefully review EVERY forced replacement / change
```

```
# read the provider's upgrade guide for the breaking changes
```

certs.eknathalabs.com

```
# apply to dev → staging → prod, validating at each step
```

Treat it like a dependency major bump: isolate it in its own PR, run `init -upgrade`, and **scrutinize the plan** for forced replacements (a schema change can make Terraform want to recreate resources — dangerous for stateful ones). Consult the provider's official upgrade guide for migration steps, apply `moved`/`ignore_changes` where the change is cosmetic, and protect stateful resources with `prevent_destroy`. Roll through dev → staging → prod, never straight to prod.

Interview tip: the danger signal is a major-version provider bump silently planning to replace databases or volumes. The senior workflow: dedicated PR, read the upgrade guide, review the plan line by line, `prevent_destroy` on stateful resources, and a staged rollout.

📚 **Official docs**

- Google API Design Guide

Q95 **How do you manage Terraform for disaster recovery across regions?** Production

🎙 **Spoken answer · ~77 s**

Multi-region disaster recovery with Terraform means making region a parameter, replicating the stack, and being explicit about state and data. I write the infrastructure as a module that takes region as an input and deploy the same module in the primary and the DR region, using provider aliases when both live in one configuration, or separate states per region, which I prefer for isolation, so a problem in one region's state doesn't block the other. The state backend itself must survive a regional outage: I replicate the state bucket cross-region or host it outside the failure domain, and keep the pipeline able to run when the primary region is down. Terraform builds the infrastructure, but not the data, so I pair it with data replication: database cross-region replicas, S3 replication, and backups copied to the DR region. Depending on RTO and RPO, the DR stack can be backup and restore, pilot light, warm standby, or active-active, and Terraform makes each reproducible. Traffic failover uses Route 53 health checks or global load balancing. And I test it with regular DR drills, because untested DR plans fail.

Likely follow-ups

- Why must the state backend itself be resilient to a regional outage?
- How does the DR strategy chosen affect what Terraform keeps running?

Multi-region DR with Terraform means parameterizing region and replicating the stack, with state and data-replication concerns handled explicitly.

```
# Region as a variable; same root module deployed per region:
```

```
module "stack" { source = "./modules/stack"  region = var.region }
```

```
# Provider aliases for multi-region in one config:
```

```
provider "aws" { alias = "primary"  region = "us-east-1" }
```

```
provider "aws" { alias = "dr"       region = "us-west-2" }
```

certs.eknathalabs.com

```
# Separate state per region; replicate data via provider features (e.g. cross-region
replication)
```

Make region a variable and deploy the same module to each region (separate state per region for isolation), or use **provider aliases** to manage primary and DR regions from one configuration when they must be coordinated. Terraform provisions the infrastructure shape in both regions; **data** replication (DB read replicas, S3 cross-region replication, backups) is configured as resources but the actual failover/promotion is usually an operational runbook, not a Terraform apply.

Interview tip: separate the concerns — Terraform reproduces the *infrastructure* consistently across regions (provider aliases or per-region state), but failover orchestration and data promotion are operational procedures. Mentioning that distinction shows you understand DR is more than "apply in another region."

📚 **Official docs**

- Terraform: Provider configuration (aliases)

Q96

A count-based resource list changed in the middle, and Terraform wants to destroy/recreate many resources. Why, and how do you fix it?

Production

🎙 **Spoken answer · ~76 s**

Count indexes resources by position. If I remove an item from the middle of a list, every item after it shifts down one index, so Terraform sees address zero, one, two, and three now holding different values, and concludes those resources changed or must be destroyed and recreated. A small list edit can cause mass churn, possibly of production resources. The fix is to use for_each, which keys resources by a stable identifier, such as a name, so removing one item affects only that one. To migrate an existing count-based resource without recreation, I switch to for_each and add moved blocks mapping each old indexed address, like x[0], to the new keyed address, like x["web"], or I use terraform state mv, then check that plan shows moves and no replacement. Going forward, I use count only for simple zero-or-one toggles, and for_each for collections of distinct things. One for_each constraint to remember is that the keys must be known at plan time, and values need to be a map or set, so lists get converted with toset or by building a map.

Likely follow-ups

- How do you migrate from count to for_each without recreating resources?
- What constraint do for_each keys have?

`count` indexes resources by position. Removing an item in the middle shifts every subsequent index, so Terraform thinks each shifted resource is a different one — mass destroy/recreate. `for_each` avoids this.

```
# Problem: count indexes by position
```

```
resource "aws_instance" "x" { count = length(var.names) ... }
```

```
# remove names[1] → indices 2,3,4 shift to 1,2,3 → churn
```

```
# Fix: for_each keys by a stable identifier
```

certs.eknathalabs.com

```
resource "aws_instance" "x" {
  for_each = toset(var.names)   # keyed by name, not position
  ...
}
# migrate existing count resources without recreation:
terraform state mv 'aws_instance.x[1]' 'aws_instance.x["web2"]'
```

With `count`, the address is `x[0]`, `x[1]`, ... — purely positional. Deleting a middle element renumbers everything after it, and Terraform plans to destroy/recreate each shifted resource. `for_each` keys resources by a stable value (a name/ID), so adding or removing one item only affects that one. To migrate an existing count-based resource to for_each without recreation, `state mv` each from its index to its new key.

Interview tip: "use `for_each` over `count` for sets of distinct, identifiable resources; use `count` only for truly homogeneous, order-independent copies or simple on/off (`count = var.enabled ? 1 : 0`)." Knowing the positional-index pitfall is a classic senior-level distinguisher.

📚 **Official docs**

- Terraform: lifecycle meta-argument
- Terraform: for_each

Q97 **How do you roll back a bad Terraform change that has already been applied to production?** Production

🎙 **Spoken answer · ~75 s**

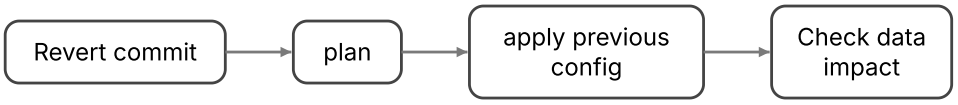
Terraform has no native rollback command, so I roll forward to the previous desired state. The mechanism is to revert the commit that introduced the bad change, open a pull request or use an emergency process, run plan to confirm it reverses the change, and apply. That makes the rollback reviewable and recorded. The backend's state versioning is the safety net: if the state itself is wrong, I can restore a prior version, but that's a careful, rare operation, since state must match reality. The important caveat is that some changes can't be rolled back: destroyed stateful resources lose data, so recovery means restoring from backup; and applied changes with side effects, like rotated credentials, don't revert themselves. So the real strategy is prevention and speed: small, reviewed changes, plan review, prevent_destroy on critical resources, snapshots before risky changes, staged rollout through environments, and policy checks. For immutable infrastructure I keep the old version available, with blue-green or create-before-destroy, so I can switch back traffic quickly. And if the previous configuration no longer works because dependencies changed, forward-fix is the answer.

Likely follow-ups

- Why is rolling back state files dangerous?
- When is forward-fix better than revert?

certs.eknathalabs.com

🗺 **Rolling back**



Terraform has no native "rollback" — you roll forward to the previous desired state. The mechanism is reverting the code and re-applying, with state versioning as the safety net.

```
git revert <bad-commit>      # restore the previous configuration
```

```
terraform plan               # confirm it reverses the bad change
```

```
terraform apply              # roll FORWARD to the prior known-good state
```

```
# if state itself is wrong, restore a prior state version from the backend, carefully
```

The clean path is Git: revert the commit that introduced the bad change and re-apply, so Terraform reconciles back toward the previous configuration. Because infrastructure changes are not always perfectly reversible (a destroyed resource may not come back identical, data may be lost), review the revert plan carefully — it shows exactly what reversing will do. Backend state versioning lets you also recover a prior state file if state, not just config, was damaged.

Key caveat: "rollback" of infrastructure is roll-forward, and some changes are irreversible (deleted data, recreated resources with new IDs). This is why `prevent_destroy`, plan review, and staged rollouts matter — preventing the bad apply beats reversing it.

📚 **Start here (official docs)**

- Terraform documentation

certs.eknathalabs.com

Q98 **How do you enforce tagging, naming, and cost-allocation standards across all resources?** Production

🎙 Spoken answer · ~70 s

Consistency comes from combining automation, shared code, and policy, not from relying on people to remember. First, provider-level default tags: in the AWS provider's default_tags block I set owner, environment, and cost centre once, and every taggable resource gets them automatically, with resource-level tags merging on top. Second, naming conventions implemented in a shared module or locals, such as environment, service, and component, so names are generated, not typed. Wrapping resources in company modules that bake in required tags and naming makes the compliant path the easy path. Third, enforcement: policy as code, using Sentinel or OPA against the plan, blocks any resource missing required tags or with a non-compliant name; static scanners can check tag presence too. Fourth, cloud-side enforcement for things created outside Terraform: tag policies in AWS Organizations, AWS Config rules, or Azure Policy, with reporting on untagged resources. For cost allocation, I activate those tags in the billing tool and review reports regularly. Variable validation can check allowed values for tags such as environment.

Likely follow-ups

- Why use default_tags instead of tagging each resource?
- How do you catch untagged resources not created by Terraform?

Consistency comes from a combination of provider default tags, shared modules, and policy enforcement — not from hoping engineers remember.

```
# 1. Provider-level default tags (applied to everything):
```

```
provider "aws" { default_tags { tags = { Owner = var.team  Env = var.env  CostCenter = var.cc } } }
```

```
# 2. Naming via a shared module / locals convention:
```

```
locals { name = "${var.env}-${var.service}-${var.component}" }
```

```
# 3. Policy enforcement (block untagged resources):
```

```
#    Sentinel / OPA-Conftest against plan JSON: require Owner+Env+CostCenter
```

Use the provider's `default_tags` to stamp common tags (owner, environment, cost-center) on every resource automatically. Standardize naming with a shared `locals`/module convention so names are predictable and parseable. Then make it **enforceable** with policy-as-code that rejects any plan creating resources missing required tags — turning standards from documentation into a blocking gate that also powers accurate cost allocation.

Interview tip: three layers — `default_tags` for automatic application, a module/convention for consistency, and policy-as-code for enforcement. Tie it to **cost allocation** (CostCenter/Owner tags drive showback/chargeback), which connects infra hygiene to FinOps — a strong signal for platform roles.

certs.eknathalabs.com

📚 **Official docs**

- AWS provider: default_tags

Q99 **How do you handle resources that must be managed partly outside Terraform (e.g. autoscaling-adjusted values)?** Production

🎙 **Spoken answer · ~74 s**

Some attributes are legitimately changed by other systems after creation: an autoscaler adjusts desired capacity, a deployment tool changes the container image, or another process adds tags. If Terraform keeps trying to reset them, it fights those systems and every plan shows noise. The tool for this is the ignore_changes setting inside the lifecycle block, listing the attributes Terraform should stop tracking after creation. For example, on an Auto Scaling Group I set desired_capacity as an initial value and ignore it afterwards, so the autoscaler owns it at runtime. I can ignore specific tags or map keys too. I should be careful and precise, ignoring only attributes that are truly owned elsewhere, because ignoring too much hides real drift and makes Terraform less useful; the keyword all is almost always too broad. I document why each ignore exists. A related design choice is to split ownership more cleanly: for instance, let Terraform manage the service and let the deployment pipeline manage the image tag, or use a placeholder image with ignore_changes. And I review ignored attributes periodically.

Likely follow-ups

- Why is ignore_changes = all usually a bad idea?
- How would you let a deploy tool own the container image while Terraform owns the service?

Some attributes are legitimately changed by other systems (autoscaling adjusts desired_count; another tool manages a tag). `ignore_changes` tells Terraform to stop fighting those changes.

```
resource "aws_autoscaling_group" "app" {
  desired_capacity = 3   # initial value only
  lifecycle {
    ignore_changes = [desired_capacity]   # autoscaler owns this at runtime
  }
}
# ignore everything that drifts but you don't manage:
#   ignore_changes = [tags["LastScaled"]]
```

Without `ignore_changes`, Terraform would revert the autoscaler's runtime adjustments on every apply (resetting `desired_capacity` back to 3), fighting the system that legitimately owns that value. `ignore_changes` on the specific attribute(s) cedes ownership of those fields to the external system while Terraform still manages the rest of the resource. Scope it

certs.eknathalabs.com

tightly to the exact attributes shared with another owner.

Interview tip: the principle is "single owner per attribute." `ignore_changes` resolves the tug-of-war when an attribute is genuinely owned at runtime by another system (autoscaling, external automation). Over-using it to paper over real drift is the anti-pattern to avoid — scope it to specific attributes, not whole resources.

📚 **Official docs**

- Terraform: lifecycle meta-argument

Q100 **How do you safely make changes when multiple teams share a single large Terraform repository?** Production

🎙 **Spoken answer · ~74 s**

With many teams in one repository, I need ownership boundaries, state partitioning, and gated automation so one team's change can't break another's infrastructure. CODEOWNERS routes review to the team that owns each directory, so changes to networking need the network team's approval. State is partitioned per team and service, so blast radius is isolated and applies run in parallel without lock contention. Automation runs plan on pull request, scoped to the directories changed, via Atlantis, HCP Terraform, or a pipeline with path filters, and apply is gated on the owning team's approval, with locking per state. Shared modules are versioned with semantic versions and consumed by pinned version, so a module change doesn't silently roll out to everyone. Permissions follow ownership: each team's pipeline role can only modify its own resources, which limits the damage of a mistake or a compromised pipeline. Standard conventions, formatting, and policy checks apply to everyone, and clear documentation explains the process. A monorepo has benefits like discoverability and atomic changes, but needs this discipline; otherwise multiple repositories per team are simpler.

Likely follow-ups

- What does CODEOWNERS give you in a shared repository?
- How do you scope CI to only the states a pull request touched?

Shared-repo collaboration needs ownership boundaries, state partitioning, and gated automation so one team’s change cannot accidentally break another’s infrastructure.

```
# CODEOWNERS routes review to the owning team per directory
```

```
# State partitioned per team/service → isolated blast radius + parallel applies
```

```
# Atlantis / TF Cloud: plan-on-PR, apply gated on owning-team approval
```

```
# Shared modules versioned so a module change doesn't auto-hit all consumers
```

Partition state by team/service so applies are independent and a mistake is contained. Use **CODEOWNERS** so changes to a directory require the owning team’s review. Run plan-on-PR with the diff posted for review, and gate apply on the right approvals. Version shared modules so updates are opt-in rather than instantly propagating. This combination lets many teams move in parallel without stepping on each other’s infrastructure.

certs.eknathalabs.com

Interview tip: the themes are *isolation* (partitioned state), *ownership* (CODEOWNERS + per-team state), and *gating* (plan-on-PR, approvals, versioned modules). This is essentially platform-engineering of the Terraform workflow — exactly what a Platform/SRE interviewer wants to hear.

📚 **Official docs**

- HCP Terraform: Recommended practices

Q101 **A terraform destroy was run accidentally against production. Walk through your response.** Production

🎙 **Spoken answer · ~76 s**

An accidental destroy in production is a severity one incident, so I follow stop, assess, recover, and harden. Immediately: stop the damage. Cancel the run if it's still in progress, though cancelling mid-run can leave partial state, and lock the state and pause pipelines so nothing else proceeds. Declare the incident, get the right people involved, and communicate. Assess what was actually destroyed: pull the state, look at the run logs, and check the cloud console or CloudTrail, because the destroy may have been partial. Recover in priority order. Stateful data comes first: restore databases and volumes from snapshots, backups, or point-in-time recovery, which is where final-snapshot settings and deletion protection matter. Then reprovision the rest by running a normal apply from the known-good code, which rebuilds stateless infrastructure, and fix any data and DNS dependencies, restore secrets, and validate service health. Throughout, keep a timeline. Afterwards, a blameless postmortem and hardening: prevent_destroy and deletion protection on critical resources, restrict who can run destroy, separate plan and apply roles, require approvals for production, policy checks that block destroy plans, and tested backups.

Likely follow-ups

- Which resources would you restore first and why?
- What guardrails would stop a destroy command reaching production?

This is a sev-1 incident. Response is: stop further damage, assess what was lost, recover from backups/state versioning, and then do a blameless post-incident hardening.

IMMEDIATE RESPONSE

```
# 1. STOP: cancel the run if still in progress; lock the state
```

```
# 2. Assess: terraform state pull / cloud console — what was actually destroyed?
```

```
# 3. Recover stateful data from backups/snapshots (RDS snapshots, EBS snapshots)
```

```
# 4. Re-provision: terraform apply from the known-good config to rebuild resources
```

```
# 5. Re-import any resources restored out-of-band so state matches reality
```

First halt the operation and prevent concurrent applies. Then assess scope — what was destroyed, and crucially what held **data** (those need snapshot/backup restores, not just re-apply). Re-apply the known-good configuration to rebuild the infrastructure shape, restore data from backups, and reconcile state with reality via imports. Finally, run a blameless post-incident review

certs.eknathalabs.com

focused on the guardrails that were missing.

The hardening that prevents recurrence: `prevent_destroy` on stateful resources, requiring approval for destroy operations in CI, restricting who/what can run destroy in prod, and backups/snapshots with tested restores. The lesson interviewers want: a destroy should be hard to do by accident, and data should always be recoverable.

📚 **Official docs**

- Terraform CLI: destroy

Q102 **How do you manage Terraform at scale with hundreds of state files and many teams (platform perspective)?** Production

🎙 **Spoken answer · ~72 s**

At scale, the challenge moves from writing HCL to operating Terraform as a platform: consistent structure, automated workflow, guardrails, and self-service. I standardise repository and module structure, with a versioned module registry offering golden modules that bake in security and tagging. State is per service and environment, with consistent backend key conventions, and a state per layer. Workflow runs through an orchestration platform: HCP Terraform, Atlantis, or Spacelift, providing plan on pull request, gated applies, locking, role-based access, and audit logs. Credentials come from OIDC and are scoped per environment. Guardrails are organisation-wide policy as code, with Sentinel or OPA, enforcing required tags, no public exposure, allowed regions, and cost limits, plus scheduled drift detection. Self-service lets teams create what they need through templates or no-code modules without waiting on the platform team. I also invest in observability of the platform itself: run success rates, durations, drift reports, and policy violations. Documentation, onboarding, and support matter, because adoption is the measure of success. And I manage the platform's own configuration as code.

Likely follow-ups

- How do you measure whether the Terraform platform is healthy?
- How do teams get new infrastructure without waiting on the platform team?

At scale, the challenge shifts from writing HCL to operating Terraform as a platform: consistent structure, automated workflow, guardrails, and self-service.

```
# Standardized repo/module structure + a shared module registry
```

```
# Per-(service x env) state with consistent backend key conventions
```

```
# Orchestration: Terraform Cloud / Atlantis / Spacelift for plan-apply + RBAC
```

```
# Guardrails: org-wide policy-as-code (Sentinel/OPA), required tags, no public exposure
```

```
# Golden modules: opinionated, versioned modules teams compose (paved road)
```

```
# Drift detection + cost estimation (Infracost) wired into PRs
```

certs.eknathalabs.com

The platform answer: provide a **paved road** — opinionated, versioned "golden" modules teams compose, a standard repo/state structure, and an orchestration layer (TFC/Atlantis/Spacelift) that handles plan-on-PR, gated apply, RBAC, and state management for them. Enforce org-wide guardrails with policy-as-code and required tags, surface cost (Infracost) and drift in PRs, and manage the platform configuration itself as code. The goal is self-service infrastructure where the safe, compliant path is the easy one.

Interview tip: this is the staff/principal framing — you are not just using Terraform, you are building the *platform* around it: golden modules, orchestration, policy guardrails, cost/drift visibility, self-service. Mentioning "paved road" and "the safe path is the easy path" signals platform-engineering maturity.

📚 **Official docs**

- Terraform: State

Q103 **Explain dynamic blocks and when you should (and should not) use them.** Modules

🎙 **Spoken answer · ~78 s**

A dynamic block generates repeated nested blocks from a collection instead of writing each one by hand, for example the ingress rules in a security group. I give it a for_each over a variable such as a list of rule objects, and a content block that uses the iterator's value for the port, protocol, and CIDRs. It's the right tool when the number or content of nested blocks is data-driven, such as rules supplied by the caller or settings that vary between environments. I avoid it when the blocks are few and fixed, because writing them explicitly is clearer. Overuse makes configuration harder to read, since the reader has to mentally expand the loop, and plan output becomes harder to map back to the code. Nested dynamic blocks are especially hard to follow, and a very generic abstraction can hide the real resource arguments and make later changes difficult. Dynamic only generates nested blocks, not top-level arguments, and the for_each value must be known at plan time. My rule is to use dynamic blocks when they remove real duplication, and keep them shallow and well commented.

Likely follow-ups

- When is writing the blocks out explicitly better?
- What can't a dynamic block generate?

A `dynamic` block generates repeatable nested blocks (like multiple ingress rules) from a collection, instead of writing each by hand.

certs.eknathlabs.com

```
resource "aws_security_group" "x" {
  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port = ingress.value.from
      to_port   = ingress.value.to
      protocol  = ingress.value.proto
      cidr_blocks = ingress.value.cidrs
    }
  }
}
```

Dynamic blocks shine when the number of nested blocks is data-driven (rules from a variable, settings from a map). But they reduce readability and can hide intent, so do not reach for them when you have a small, fixed set of blocks — explicit blocks are clearer. Overusing dynamic blocks produces "write-only" Terraform that is hard to review.

> **Interview tip:** "use dynamic blocks for genuinely variable, data-driven nested blocks; prefer explicit blocks when the set is small and static." Mentioning the readability cost shows judgment, not just knowledge of the feature.

> 📚 **Official docs**
>
> - Kubernetes: Ingress

certs.eknathalabs.com

Q104 **How do you design a reusable module with a clean interface?** Modules

🎙 Spoken answer · ~77 s

I design a module as a black box with a deliberate interface. Inputs are well-named variables with types, descriptions, sensible and safe defaults, and validation blocks, so misuse fails early with a clear message. Outputs expose only what consumers need, such as IDs and ARNs, and keep internals hidden so I can refactor without breaking callers. The module has a single responsibility, one cohesive thing like a VPC or a service, no hardcoded provider configuration, which comes from the root module, and no hidden assumptions about account, region, or naming; anything environment-specific is an input. I keep the variable count manageable and group related settings in typed objects with optional attributes, and avoid the kitchen-sink design with dozens of toggles. Supporting practices: documentation with usage examples, a README generated by terraform-docs, tests with the native framework, semantic versioning with a changelog, and moved blocks to preserve state when I rename internals. I also think about how it will be composed: modules shouldn't reach into each other, but communicate through variables and outputs, and it should be easy to use correctly and hard to use incorrectly.

Likely follow-ups

- Why keep provider configuration out of reusable modules?
- How do you avoid a module becoming a kitchen sink?

A good module is a black box with a deliberate interface: well-named input variables (with types, defaults, validation), useful outputs, and no hidden assumptions.

```
variable "instance_type" {
  type        = string
  default     = "t3.micro"
  description = "EC2 instance type"
  validation { condition = can(regex("^t3\.", var.instance_type))  error_message = "..." }
}
output "instance_id" { value = aws_instance.this.id }
# expose only what consumers need; keep internals private
```

Design the interface first: typed input variables with sensible defaults, descriptions, and `validation` blocks that fail fast on bad input; outputs that expose exactly what consumers need (IDs, ARNs, endpoints) and nothing more. Keep the module focused on one logical component, document required vs optional inputs, and avoid leaking provider/region assumptions — pass those in. A module should be composable and testable in isolation.

Interview tip: emphasize *interface design* — typed/validated variables, minimal meaningful outputs, single responsibility, no hidden assumptions. Variable `validation` blocks (fail fast with clear messages) are a detail that signals real module-authoring experience.

certs.eknathalabs.com

📚 **Official docs**

- Terraform: Modules

Q105 **What is the difference between a root module, child modules, and how is composition done well?**

Modules

🎙 **Spoken answer · ~74 s**

The root module is the directory where I run Terraform; it holds the backend, provider configuration, and calls to child modules. Child modules are reusable units called with a module block. Good composition keeps the root thin and pushes reusable logic into focused children. In the root, I wire environment-specific values into module inputs and connect modules through their outputs: for instance the app module takes subnet_ids equal to module dot network dot private_subnet_ids, and Terraform infers the ordering from that reference. The root can re-export outputs it wants visible. Child modules stay independent: they don't reach into each other or read each other's state; they communicate only through variables and outputs, which makes them testable and reusable. I keep the hierarchy shallow, preferring a flat set of modules called from the root over deep nesting, since data flow through many levels is hard to follow. Providers are configured in the root and passed down explicitly when needed. Environment directories each have their own root module and backend and call the same shared modules with different inputs.

Likely follow-ups

- Why avoid deeply nested modules?
- Where should provider configuration live?

The **root module** is the directory you run Terraform in; it calls **child modules**. Good composition keeps the root thin and pushes reusable logic into focused children.

```
# root (environments/prod/main.tf):
```

```
module "network" { source = "../../modules/network"  cidr = "10.0.0.0/16" }
```

```
module "app" {
```

```
  source     = "../../modules/app"
```

```
  subnet_ids = module.network.private_subnet_ids   # wire outputs to inputs
```

```
}
```

The root module wires environment-specific values and connects child modules by passing one module’s outputs as another’s inputs (e.g. network subnets into the app module). Well-composed Terraform has small, single-purpose child modules (network, data, compute) and a thin root that just composes and parameterizes them per environment. Avoid deep module nesting (modules calling modules calling modules) — it obscures data flow and complicates debugging.

Interview tip: "thin root, focused children, wire via outputs→inputs, avoid deep nesting." Mentioning that excessive nesting hurts

certs.eknathalabs.com

readability/debuggability shows you have maintained real module hierarchies, not just authored toy examples.

📚 **Official docs**

- Terraform: Module composition

Q106 **How does terraform_remote_state work and what are its alternatives for sharing data between configs?** State

🎙 **Spoken answer · ~75 s**

The terraform_remote_state data source reads the outputs of another configuration's state file, so one layer can consume values from another, for example a compute stack reading the VPC ID and subnet IDs from the network stack. I configure it with the backend type and the bucket, key, and region of the other state, then reference its outputs. It's simple, but it has downsides: it couples me to the other configuration's backend and output names, it needs read access to the entire state file, which may contain secrets, and the producer must expose outputs deliberately. Alternatives reduce that coupling. A provider data source looks up the actual resource from the cloud API, for example finding a VPC by tag, so it doesn't depend on another team's Terraform at all. A parameter store such as SSM Parameter Store or Secrets Manager lets the producer publish values and consumers read them, with fine-grained access control, and it works for non-Terraform consumers too. I use remote state for simple cases within one team, and data sources or a parameter store across team boundaries.

Likely follow-ups

- Why is read access to remote state a security concern?
- What are the failure modes of data source lookups by tag?

`terraform_remote_state` reads another configuration's outputs from its state file, letting one layer consume another's values (e.g. compute reads the network's VPC ID).

```
data "terraform_remote_state" "network" {
  backend = "s3"
  config  = { bucket = "tf-state"  key = "network/prod.tfstate"  region = "us-east-1" }
}
# use: subnet_ids = data.terraform_remote_state.network.outputs.private_subnet_ids
# alternatives:
data "aws_vpc" "main" { tags = { Name = "prod" } }   # query the cloud directly
# or pass values via SSM Parameter Store / variables
```

certs.eknathalabs.com

Remote state coupling is convenient but creates a hard dependency: the consumer needs read access to the producer's state, and you are coupling to internal outputs. Alternatives that reduce coupling: **data sources** that query the cloud directly (e.g. look up the VPC by tag) so you depend on the real resource not its state, or a shared store like **SSM Parameter Store** where the producer publishes values and consumers read them — decoupling the two configs from each other's state.

Interview tip: note the trade-off — `terraform_remote_state` is simple but couples configs to each other's state and requires state read access. Data sources (query reality) or a published parameter store reduce coupling and are often the more robust choice at scale.

📚 **Official docs**

- Terraform: The terraform_remote_state data source

Q107 **Explain terraform refresh, the -refresh-only mode, and why a bare refresh can be risky.** State

🎙 **Spoken answer · ~73 s**

Refresh updates state to match the real world without changing infrastructure. The old standalone terraform refresh command did that immediately and silently, which was risky: if credentials were wrong, a provider had a bug, or a resource was temporarily unavailable, state could be overwritten with misleading data, for example treating resources as gone. The modern approach is plan with refresh-only, which shows how state would change to match reality for review, and apply with refresh-only, which updates state after I approve, again without touching infrastructure. Normal plan and apply refresh as part of their work. Setting refresh equals false skips the refresh and uses the cached state: it's much faster on large configurations, but it may miss drift, so I use it deliberately for quick iteration and rely on scheduled full plans for drift. So the guidance is: never run a bare refresh blindly; use refresh-only to review drift, accept legitimate out-of-band changes into state, and then decide whether to reflect them in code. The standalone refresh command is deprecated in favour of this reviewable flow.

Likely follow-ups

- When would you accept drift into state rather than revert it?
- What are the risks of refresh=false?

Refresh updates state to match the real world without changing infrastructure. Modern Terraform uses `plan/apply -refresh-only` for a reviewable version of this.

```
terraform plan -refresh-only     # show how state would change to match reality (review)
```

```
terraform apply -refresh-only    # update state to match reality, no infra changes
```

```
terraform plan -refresh=false    # skip refresh (use cached state) — faster, but may miss drift
```

```
# legacy: terraform refresh (deprecated in favor of -refresh-only)
```

Refresh reconciles state with actual resource attributes (picking up out-of-band changes). The standalone `terraform refresh` is deprecated because it silently mutated state with no review; `-refresh-only` lets you *see* the proposed state changes (plan) and approve them (apply) without any infrastructure modification — useful for absorbing legitimate drift into state. -

certs.eknathalabs.com

`refresh=false` skips refresh entirely for speed when you trust state is current.

> **Interview tip:** "`-refresh-only` reconciles state with reality reviewably; standalone `refresh` is deprecated for being a silent state mutation." Knowing the deprecation and the reviewable replacement signals current Terraform fluency.

> 📚 **Official docs**
>
> - Terraform CLI: plan

Q108 **How do you migrate Terraform state between backends (e.g. local to S3, or S3 to Terraform Cloud)?** Backend

> 🎙 **Spoken answer · ~70 s**
>
> Backend migration is built into terraform init: you change the backend configuration, re-initialise, and Terraform offers to copy the state to the new backend. My process: first back up the current state with state pull to a file, and make sure nobody else is running Terraform, by pausing pipelines or holding the lock. Then I update the backend block, for example adding the S3 bucket, key, region, encryption, and locking table, or the cloud block for HCP Terraform. I run init with migrate-state; Terraform detects the change and asks whether to copy the existing state, and I confirm. Afterwards I verify: state list shows all the resources, and plan shows no changes, which proves the migrated state matches reality. I keep the old state until I'm confident, then archive it and remove stray local copies. The new backend should have versioning, encryption, locking, and restricted access configured before migration. Then I update CI and tell the team to re-initialise. There's no infrastructure downtime, because only the state file moves.
>
> **Likely follow-ups**
>
> - How do you confirm the migration worked?
> - What should be configured on the new backend first?

Backend migration is built into `terraform init`: change the backend config, re-init, and Terraform offers to copy state to the new backend.

```
# 1. Back up current state first
```

```
terraform state pull > backup.tfstate
```

```
# 2. Update the backend block (e.g. add the s3 backend config)
```

```
terraform { backend "s3" { bucket="tf-state" key="prod.tfstate" dynamodb_table="tf-lock" }
}
```

```
# 3. Re-init; Terraform detects the change and prompts to migrate:
```

```
terraform init -migrate-state
```

```
# 4. Verify:
```

certs.eknathalabs.com

```
terraform plan   # should show NO changes
```

Update the `backend` block to the new target, then run `terraform init -migrate-state` — Terraform recognizes the backend changed and offers to copy the existing state to the new backend. Always `state pull` a backup beforehand, and verify with a clean `plan` afterward (no changes = migration succeeded and config still matches reality). Ensure locking is in place on the new backend before others use it.

Coordinate the cutover: migrate when no one else is applying, and make sure everyone re-inits to the new backend afterward. Two people on different backends pointing at the same resources is a recipe for divergent state.

📚 **Official docs**

- Terraform: State
- Terraform: Backends
- Terraform: S3 backend

Q109 **What are locals, and how do they differ from variables and outputs?** Modules

🎙 **Spoken answer · ~77 s**

Locals are named values computed inside a module for internal reuse. They are neither inputs nor outputs. A variable is an input, set by the caller through the command line, a tfvars file, environment variables, or defaults, and it's the module's public interface. A local is derived: for example a name prefix built from the environment and application name, or a merged map of common tags, and it exists to avoid repeating expressions and to give meaningful names to complex ones. An output exposes a value from the module to its caller, or from the root module to the CLI and other configurations. So the flow is: variables come in, locals compute things in the middle, and outputs go out. The common mistake is making something a variable when it should be derived, which lets callers set inconsistent values, or using locals for things that callers legitimately need to customise. I keep locals readable and minimal, resist hiding too much logic in them, and treat outputs as a stable contract. Locals can reference variables, resources, and other locals, and cannot be overridden from outside.

Likely follow-ups

- When should a derived value be a local rather than a variable?
- Can a caller override a local?

`locals` are named values computed within a module for internal reuse — they are not inputs (variables) or exposed values (outputs).

```
variable "env" { type = string }              # INPUT from the caller
```

```
locals {
```

```
  name_prefix = "${var.env}-myapp"            # INTERNAL computed value
```

```
  common_tags = { Env = var.env  ManagedBy = "terraform" }
```

certs.eknathalabs.com

```
}
```

```
output "name" { value = local.name_prefix }   # EXPOSED to the caller
```

Variables are the module's inputs (set by the caller). Outputs are what the module exposes to its caller. `locals` sit in between: intermediate expressions computed once and reused throughout the module (a name prefix, a merged tag map, a derived flag) to avoid repetition and centralize logic. Unlike variables they cannot be overridden from outside; unlike outputs they are not visible externally.

Interview tip: crisp framing — "variables = inputs, outputs = exposed results, locals = internal computed reuse." A common use is a `common_tags` local merged into every resource. Knowing locals are not overridable (unlike variables) shows you understand the data-flow model.

📚 **Official docs**

- Terraform: Locals
- Terraform: Output values

Q110 **Explain the difference between count, for_each, and when each is appropriate.** Modules

🎙 **Spoken answer · ~77 s**

Both create multiple instances of a resource or module, but they index differently, which determines stability when the collection changes. Count uses integer positions, so addresses are index zero, one, two. It's appropriate for a fixed number of identical copies and, most commonly, for a simple conditional toggle where count is one or zero. The danger is lists: removing or reordering an element shifts all later indexes, so Terraform plans to modify or destroy and recreate resources that didn't really change. For_each uses a map or a set of strings, and addresses are keyed by the map key or set value, such as x with the key web. Because keys are stable, adding or removing one entry affects only that entry. It's the right choice for collections of distinct things, such as users, buckets, or subnets by name. Its constraints: keys must be known at plan time, so I can't derive them from attributes only known after apply, and the input must be a map or set, so lists get converted with toset. Renaming a key means destroy and create, so use moved blocks.

Likely follow-ups

- Why does for_each avoid the shifting-index problem?
- How do you convert a list for use with for_each?

Both create multiple instances of a resource/module, but they index differently — which determines stability when the collection changes.

```
# count: positional index [0], [1], [2] ...
```

```
resource "aws_instance" "x" { count = 3 }           # 3 identical copies
```

```
resource "aws_instance" "x" { count = var.enabled ? 1 : 0 }  # conditional create
```

certs.eknathalabs.com

```
# for_each: keyed by map key or set value
```

```
resource "aws_instance" "x" { for_each = toset(["web","api"]) }  # x["web"], x["api"]
```

```
resource "aws_instance" "x" { for_each = var.instances }         # map of configs
```

Use **count** for a number of truly identical, order-independent copies, or for a simple conditional (`count = var.enabled ? 1 : 0`). Use **for_each** for a set/map of distinct, individually-identifiable resources — because it keys by a stable identifier, adding or removing one element only affects that element (no positional churn). For_each is the safer default whenever the items have meaningful identities.

Interview tip: the discriminator is "are the instances interchangeable copies (count) or distinct identifiable things (for_each)?" Pairing this with the count-middle-deletion churn problem (positional reindexing) demonstrates real production scar tissue.

📚 **Official docs**

- Terraform: for_each

Q111 **What is the purpose of terraform output, and how is it consumed downstream?** State

🎙 **Spoken answer · ~78 s**

Outputs expose values from a configuration for three audiences. For humans, terraform output prints the values after an apply, such as a load balancer DNS name. For scripts and CI, terraform output with the json flag gives machine-readable output, and the raw flag prints a single value unquoted, which is handy in shell scripts, for example to pass a cluster name to a deploy step. For other Terraform configurations, outputs from the root module are available through the terraform_remote_state data source, so one layer can consume another's values. Within a module, outputs are how a child module exposes values to its caller, and the caller must re-export them if they should be visible at the root. I mark outputs holding secrets as sensitive, which hides them in the CLI display, but they remain in the state in plain text, and terraform output with an explicit name or the json flag can still reveal them, so I protect state access. Good practice is to expose only what consumers need, document each output, and treat outputs as a stable interface, because downstream configurations and scripts depend on them.

Likely follow-ups

- How does a script consume Terraform outputs in CI?
- Does marking an output sensitive remove it from state?

Outputs expose values from a configuration — for humans, for other configs (via remote state), and for scripts/CI that need resource attributes after apply.

```
output "lb_dns" { value = aws_lb.app.dns_name }
```

```
output "db_endpoint" { value = aws_db_instance.main.address  sensitive = true }
```

```
terraform output                # human-readable
```

certs.eknathalabs.com

```
terraform output -json          # machine-readable for scripts/CI
```

```
terraform output -raw lb_dns    # single value, unquoted (for shell use)
```

Outputs surface computed attributes (endpoints, ARNs, IDs) that you cannot know until after apply. They are consumed three ways: by people inspecting results, by *other* Terraform configs through `terraform_remote_state`, and by CI/scripts via `-json` or `-raw` (e.g. feeding a DB endpoint into a deployment step). Mark outputs `sensitive` to redact them from CLI display.

Interview tip: note the three consumers (humans, other configs, automation) and the formats (`-json` for scripts, `-raw` for a single shell value). Outputs as the contract a module exposes ties back to clean module interface design.

📚 **Official docs**

- Terraform: Output values

Q112 **How does state locking work, and what happens when locking is misconfigured or absent?** Backend

🎙 **Spoken answer · ~76 s**

State locking serialises operations that can write state, so only one apply touches a state file at a time. With the S3 backend and DynamoDB, Terraform acquires a lock by writing an item with a conditional write, which is atomic, so only one writer wins, and deletes it afterwards. Newer versions also support locking with a lock file in the S3 bucket. If locking is absent or misconfigured, two concurrent applies can both read the same state, make different changes, and write back, so one overwrites the other. The result can be lost resources, orphaned infrastructure, duplicated resources, or a corrupted state file. Common misconfigurations: no lock table specified, a table with the wrong key schema, IAM permissions missing for the lock table so locking fails or is skipped, or using a backend that doesn't support locking. A stuck lock from a crashed run is cleared with force-unlock, but only after confirming nothing is running. I protect against all this by enabling locking everywhere, testing it, restricting who can write state, and running applies through a pipeline that serialises runs per state.

Likely follow-ups

- What can happen if two applies run without locking?
- What IAM permissions does the lock table need?

State locking serializes mutating operations so only one apply touches a state file at a time. Without it, concurrent applies race and corrupt state.

```
# S3 + DynamoDB locking:
```

```
backend "s3" { bucket="tf-state" key="prod.tfstate" dynamodb_table="tf-lock" }
```

```
# DynamoDB provides atomic lock acquisition (conditional write)
```

```
# (modern Terraform/OpenTofu also support S3-native lockfile locking)
```

certs.eknathalabs.com

```
terraform force-unlock <LOCK_ID>   # clear a stuck lock (only if no apply is running)
```

When an apply starts, Terraform acquires a lock (historically a DynamoDB item for S3 backends); others attempting to apply get "Error acquiring state lock" and wait/fail. This prevents two applies from writing the state file simultaneously — which would interleave updates and corrupt it. If locking is absent (e.g. local state, or S3 without a lock mechanism), concurrent runs can silently corrupt state or create duplicate resources.

Always enable locking for shared state. Its absence is invisible until two applies collide and corrupt state. A stuck lock from a crashed run is cleared with `force-unlock` — but only after confirming no apply is genuinely in progress, or you risk the very corruption locking prevents.

📚 **Official docs**

- Terraform: State
- Terraform: State locking

Q113 **How do you write conditional logic and handle optional resources/attributes in Terraform?**

Modules

🎙 **Spoken answer · ~74 s**

HCL is declarative, so conditional logic is expressed through a few constructs rather than if statements. For optional resources, I use count or for_each as a gate: count equals one if the variable is true and zero otherwise, so a bastion host exists only when requested. When something may not exist, references must handle that, so I use a ternary expression such as var.create_bastion ? aws_instance.bastion[0].id : null, or the one function in newer styles. For optional values and fallbacks, coalesce picks the first non-null value, try returns a fallback if an expression errors, lookup with a default handles missing map keys, and optional attributes in object types with defaults let callers omit fields. Dynamic blocks handle optional nested blocks. I keep conditions based on values known at plan time, not on attributes only known after apply. And I avoid piling up toggles: when a module has many booleans controlling large chunks of behaviour, I take that as a sign to split it into smaller modules or separate compositions. Validation and preconditions guard invalid combinations of options.

Likely follow-ups

- Why can't a condition use a value only known after apply?
- What is the difference between try and coalesce?

HCL is declarative, so "conditionals" are expressed via `count`/`for_each` gating, the ternary operator, and `try`/`coalesce` for optional values.

```
# conditional resource creation:
```

```
resource "aws_instance" "bastion" { count = var.create_bastion ? 1 : 0 }
```

```
# reference it safely (it may be 0):
```

```
value = var.create_bastion ? aws_instance.bastion[0].id : null
```

certs.eknathalabs.com

```
# optional attribute / default fallback:
```

```
tags = coalesce(var.tags, {})
```

```
x = try(var.config.optional_field, "default")
```

```
# optional object attributes (TF 1.3+):
```

```
variable "cfg" { type = object({ size = optional(string, "small") }) }
```

Conditional creation uses `count = condition ? 1 : 0` (or `for_each` over a possibly-empty collection). Reference such resources guardedly since the index may not exist. For optional inputs, `coalesce`/`try` provide fallbacks, and `optional()` object attributes (1.3+) let you define typed inputs with defaults. The mindset shift: you do not write imperative if/else — you declaratively gate resource existence and choose values via expressions.

Interview tip: show the declarative idioms — `count`-gating for optional resources, ternaries for values, `try`/`coalesce` for optional attributes, and `optional()` in object types. Mentioning the safe-reference pattern (`resource[0]` may not exist) reflects real-world burns.

📚 **Official docs**

- Terraform: Conditional expressions

Q114 **When and why would you use -target, and why is it considered a smell if overused?** Performance

🎙 **Spoken answer · ~77 s**

The target option limits an operation to specific resources and their dependencies. It is a legitimate tool for triage and recovery: re-running just the resource that failed mid-apply, unsticking a tricky dependency, or applying an urgent fix to one resource during an incident, and for iterating faster during development on a huge configuration. But it's a smell when it becomes routine, because it bypasses the whole-graph view. Other changes remain unapplied, so infrastructure can drift from the configuration; dependencies may only be partly handled, which can produce surprising results; and a clean targeted plan says nothing about the rest of the infrastructure. Terraform even prints a warning that target is for exceptional use. If I find myself using it all the time, the real problem is usually structural: the configuration is too large and slow, so split it into smaller states, or there's a dependency problem I should fix. After any targeted apply, I run a full plan to confirm that everything is consistent, and I don't build pipelines around target. In an interview I'd say: it's a scalpel for emergencies, not a daily tool.

Likely follow-ups

- What should you run after a targeted apply?
- What structural problem does heavy -target use usually point to?

`-target` limits an operation to specific resources. It is a legitimate triage/recovery tool but a dangerous habit if it becomes routine.

```
terraform apply -target=aws_instance.web    # apply only this resource + its deps
```

certs.eknathalabs.com

```
terraform plan  -target=module.network      # scope to one module
```

```
# legitimate uses: recovering from a partial failure, unsticking a dependency cycle,
```

```
#   applying an urgent fix to one resource during an incident
```

`-target` tells Terraform to operate on only the named resources (and their dependencies), skipping the rest. It is appropriate for **recovery** — finishing a partially-failed apply, or surgically fixing one resource in an incident. But routine use is a smell: it means you are applying a partial view of desired state, which can leave config and reality inconsistent, hide dependency problems, and produce surprising drift. If you *need* targeting regularly, the real fix is usually to split the state.

Anti-pattern: using `-target` as a normal workflow to "only apply my change." This bypasses the holistic plan and can mask broken dependencies. Reserve it for triage; if it is a habit, partition your state instead.

📚 **Official docs**

- Terraform CLI: apply

Q115 **How do you handle very large outputs or data that does not belong in state (e.g. generated files, secrets)?** State

🎙 **Spoken answer · ~75 s**

State should track resource metadata, not bulk data or secrets. Putting large blobs or secrets into outputs or state bloats it, slows every operation because the file is read and written each time, and risks exposure. So I keep them out. For secrets, store them in a secrets manager and reference them, rather than embedding the values, and have applications fetch them at runtime; use ephemeral values where supported so they aren't persisted. For large artifacts such as generated files, scripts, or templates, put them in object storage like S3 or GCS and keep only the reference, such as the bucket and key, in Terraform. Use local_file only when a file is genuinely needed on disk as part of the run. For big data blocks, avoid reading huge files with the file function or storing long rendered templates as resource attributes. Where Terraform must manage a large number of items, split into multiple states. And watch for provider attributes that echo large content back into state. I check state size periodically, since a state that grows unexpectedly is a signal.

Likely follow-ups

- Why does a large state slow down every run?
- How do you keep a rendered template out of state?

State should track resource metadata, not bulk data or secrets. Large/inappropriate data in state bloats it, slows operations, and risks exposure.

```
# Don't stuff big blobs or secrets into outputs/state. Instead:
```

```
# - write generated files via local_file only when truly needed (they go to disk, not
infra)
```

certs.eknathalabs.com

```
# - store secrets in a secrets manager; reference, don't embed
```

```
# - put large artifacts in object storage (S3/GCS), keep only the reference in state
```

```
# - mark unavoidable sensitive outputs as sensitive (redaction, not encryption)
```

Keep state lean: it is an index of resources, not a data store. Large outputs slow every plan/apply (state is read/written each time) and big sensitive values increase exposure if state leaks. Put bulk data in object storage and keep only references (URLs/ARNs) in state; source secrets from a secrets manager rather than passing them through Terraform; generate files only when a workflow genuinely requires them. The less sensitive/bulky data in state, the safer and faster Terraform is.

Interview tip: "state is metadata, not a database." The senior instinct is to keep references in state and store the actual bulk/secret data in purpose-built systems (object storage, secrets manager). This keeps state small, fast, and lower-risk.

📚 **Start here (official docs)**

- Terraform documentation

Q116 **What is partial backend configuration and why is it useful in CI/CD?** Backend

🎙 **Spoken answer · ~73 s**

Partial backend configuration leaves some backend settings out of the HCL and supplies them at init time. This is needed because backend blocks are evaluated before variables, so they can't use variables and would otherwise have to be hardcoded or duplicated per environment. In the code I keep a minimal block, for example just backend s3 with empty braces, and in the pipeline I pass the rest with backend-config options or a backend config file per environment: bucket, key, region, and the lock table. The same code then initialises against different backends depending on the environment, with no edits. It keeps environment-specific or sensitive details such as account-specific bucket names or credentials out of the repository, and it reduces the risk of pointing at the wrong backend, because the pipeline selects it explicitly. In CI/CD, each environment's job passes its own file or flags, combined with its own credentials. I keep non-secret backend files versioned for transparency. If the backend settings change, I need to re-run init, and use migrate-state if the state must move.

Likely follow-ups

- Why can't backend blocks use variables?
- How does partial config reduce the risk of using the wrong backend?

Partial backend config leaves some backend settings out of the HCL and supplies them at init time — useful so the same code initializes different backends per environment without hardcoding.

```
# code: backend block with minimal/empty config
```

```
terraform { backend "s3" {} }
```

```
# supply the rest at init (per environment, in CI):
```

certs.eknathalabs.com

```
terraform init \
  -backend-config="bucket=tf-state-prod" \
  -backend-config="key=app/prod.tfstate" \
  -backend-config="region=us-east-1"
# or: terraform init -backend-config=prod.backend.hcl
```

Instead of hardcoding bucket/key/region (which would differ per environment), you leave them out and pass them via `-backend-config` flags or a per-env `.hcl` file at `init`. This lets one codebase target dev/staging/prod backends by varying the init parameters in the pipeline, keeping environment-specific backend details out of the shared HCL.

Interview tip: partial backend config is the clean way to reuse one configuration across environments with different state locations — supply the backend key/bucket per env at init. It pairs naturally with directory-per-env or a pipeline that injects the right backend file.

📚 **Official docs**

- Terraform: Backends

Q117 **How do you debug a confusing Terraform plan or unexpected behavior?** Modules

🎙 **Spoken answer · ~77 s**

Terraform offers several inspection tools, and I use them from the simplest outward. First I read the plan carefully, attribute by attribute, because most confusion is just a misread diff. Then, for details, terraform plan with the json flag, or terraform show with the json flag on a saved plan, gives structured output I can query with jq, showing exactly which attributes change and why, including whether it's an update or a replacement. Terraform console evaluates expressions interactively against the current state and variables, so I can test a for expression or a lookup without applying anything. Terraform state show and state list show what state holds. Terraform graph shows dependencies when ordering is the question. If the behaviour still isn't clear, TF_LOG at debug level shows internal decisions and provider calls, and trace is even more verbose, though I'm careful because logs may contain sensitive data. I also compare provider and Terraform versions, check for variable precedence surprises and workspace mix-ups, and reduce the problem by testing a minimal configuration. And I check the provider documentation for attribute behaviour, like computed or force-new.

Likely follow-ups

- How can you test an expression without applying anything?
- What does the plan JSON tell you that the text output doesn't?

Terraform gives several inspection tools: verbose logs, the machine-readable plan, the dependency graph, and console expression evaluation.

```
TF_LOG=DEBUG terraform plan       # verbose internals (provider calls, decisions)
```

cons.eknathalabs.com

```
TF_LOG=TRACE terraform plan        # even more (rarely needed)
```

```
terraform plan -json | jq ...      # structured plan: inspect exact changes/reasons
```

```
terraform show -json tfplan        # decode a saved plan
```

```
terraform console                  # evaluate expressions interactively
```

```
terraform graph | dot -Tpng > g.png  # visualize the dependency graph
```

```
terraform validate                 # catch config errors early
```

Start with `terraform console` to test expressions/interpolations in isolation, and the **JSON plan** (`-json` or `show -json`) to see precisely what is changing and which attribute "forces replacement." `TF_LOG=DEBUG` exposes provider API calls and Terraform's decisions when behavior is mysterious. `terraform graph` visualizes dependencies when ordering or cycles confuse you. `validate` catches structural errors before plan.

Interview tip: name the toolset — `terraform console` for expressions, JSON plan for "why is it changing this," `TF_LOG=DEBUG` for provider-level mysteries, `graph` for dependency issues. Knowing the JSON plan reveals the forced-replacement attribute is a particularly practical detail.

📚 **Official docs**

- Terraform CLI: plan

Q118 **How do you reduce Terraform plan/apply time and provider API pressure on large configs?**

Performance

🎙 **Spoken answer · ~76 s**

Performance depends on state size, refresh round-trips, and parallelism, and structure matters most. The best fix is structural: split a large configuration into smaller independent states so each plan refreshes fewer resources and each run is faster, with a smaller blast radius and less lock contention. Then tactical options: skip refresh with refresh equals false when fresh drift data isn't needed, such as during development, and keep scheduled full plans to catch drift; raise parallelism above the default ten if the provider's API can take it, or lower it if throttling occurs; use data sources sparingly because each one is an API call on every plan, and cache stable values in locals or parameters; remove unnecessary depends_on that serialise the graph; use target only for triage. For provider API pressure, configure retries with backoff, stagger pipelines so many don't run against the same account simultaneously, and request quota increases when needed. In CI, cache the provider plugins to avoid downloading them each run. And I measure with debug timing before and after changes, so I'm fixing the real bottleneck rather than guessing.

Likely follow-ups

- Which data sources are most expensive per plan?
- How do you measure where plan time goes?

certs.eknathalabs.com

Performance comes from smaller state, fewer/refresh round-trips, and controlled parallelism — plus structural partitioning when a single config grows too big.

```
# structural (best): split into smaller independent states
```

```
terraform plan -refresh=false       # skip refresh when fresh drift data isn't needed
```

```
terraform apply -parallelism=20     # more concurrency (mind provider rate limits)
```

```
# use data sources sparingly — each can be an API call every plan
```

```
# provider plugin cache to avoid re-downloading providers:
```

```
export TF_PLUGIN_CACHE_DIR=~/.terraform.d/plugin-cache
```

The dominant factor is state size: a huge monolithic state refreshes and diffs slowly, so **partition it**. Tactically, `-refresh=false` avoids per-resource refresh API calls when you do not need current drift data, tuning `-parallelism` trades speed against provider rate limits, minimizing heavy data sources cuts per-plan API calls, and a plugin cache speeds `init`. Profile with `TF_LOG` to find the slow operations.

> **Interview tip:** lead with the structural fix (partition state), then list the tactical levers (`-refresh=false`, parallelism, fewer data sources, plugin cache). Noting that data sources cost an API call per plan, and parallelism is bounded by provider rate limits, shows depth.

> 📚 **Official docs**
>
> - Terraform CLI: plan

certs.eknathalabs.com

Q119 How do you ensure supply-chain integrity of providers and modules?

Security

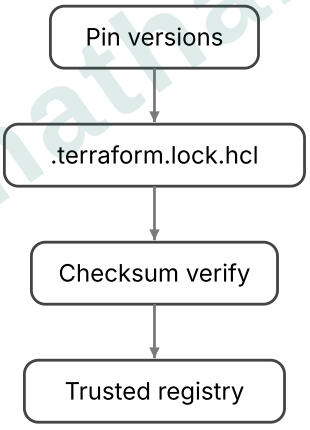
🎙 Spoken answer · ~78 s

Providers and modules are external code that runs with my cloud credentials, so I verify their authenticity and pin them like any other software supply chain. For providers, the dependency lock file records the exact version and cryptographic checksums, so Terraform verifies downloaded binaries against them, and I commit it, generating hashes for all platforms I use. Providers from the official registry are also signed, and Terraform checks signatures. I use trusted sources, prefer HashiCorp or verified partner providers, and review community ones. For modules, I pin to exact versions or immutable tags and never a moving branch, and for critical modules I might vendor a copy or use a private registry mirror, so upstream changes or takedowns don't affect me. I review module code before adopting it, particularly for anything that creates IAM or network access. In CI, I restrict network egress to the registries I trust, run dependency update tools like Renovate so upgrades are reviewed as pull requests, and run security scanners on module code. Provider and module updates are reviewed with plan diffs. And pipeline credentials are least-privilege, limiting what malicious code could do.

Likely follow-ups

- What does the lock file's checksum protect against?
- Why vendor critical modules or use a private mirror?

🗺 Provider integrity



Providers and modules are external dependencies that execute with your cloud credentials — verify their authenticity and pin them, just like any software supply chain.

```
# .terraform.lock.hcl pins provider versions AND checksums — COMMIT it
```

```
terraform providers lock -platform=linux_amd64 -platform=darwin_arm64  # multi-platform hashes
```

```
# pin modules to exact versions/immutable tags (never a moving branch)
```

```
module "vpc" { source="..."  version="5.8.1" }
```

```
# prefer trusted/official providers; consider an internal mirror/registry
```

certs.eknathalabs.com

```
# scan IaC with tfsec/checkov; review third-party module source before adoption
```

The `.terraform.lock.hcl` file is central: it records exact provider versions and cryptographic checksums, so every init verifies integrity and reproducibility — commit it and use `terraform providers lock` to capture hashes for all platforms your team/CI use. Pin modules to immutable versions/tags (never a mutable branch). Prefer official/verified providers, optionally mirror them in an internal registry to control what enters your environment, and review third-party module source before adopting it.

Why it matters: a provider or module runs with your infrastructure credentials — a malicious or tampered one is a serious supply-chain risk. The lock file (versions + checksums), exact module pinning, trusted sources, and scanning are your defenses. An uncommitted lock file or a module sourced from a moving branch are red flags.

📚 **Official docs**

- SLSA specification

Q120 **Capstone: design Terraform foundations for a multi-account, multi-team organization from scratch.** Production

🎙 **Spoken answer · ~73 s**

I'd design the foundations around a few decisions. Accounts: separate cloud accounts per environment and per business unit, created by an automated account factory with baseline guardrails, giving hard isolation. State: one state per account, layer, and service, stored in an encrypted, versioned, locked backend, with consistent key conventions, and layers connected through outputs published to a parameter store or data sources. Layers go from organisation and identity, to networking, to shared platform, to application stacks, with the slowest-changing, highest-impact layers most tightly controlled. Modules: a versioned registry of golden modules that embed security, tagging, and naming standards, with semantic versioning and tests. Workflow: pull request driven; plan on pull request, gated apply from a pipeline on merge, using OIDC and scoped roles per environment, with approvals for production. Guardrails: policy as code on plans, service control policies, scheduled drift detection, and cost visibility. Ownership: CODEOWNERS and team-scoped permissions, with self-service templates so teams move quickly. Finally operations: runbooks for lock and state recovery, documentation, onboarding, and measurement of the platform's reliability and adoption.

Likely follow-ups

- How would you roll this out to existing teams incrementally?
- What would you build first in the first month?

Bring together repo structure, state strategy, module strategy, workflow, security, and guardrails into a coherent platform foundation.

FOUNDATIONS

```
# Accounts: separate AWS accounts per env (and per business unit) — hard isolation
```

```
# State: per (account x layer x service); S3+KMS+DynamoDB (or TF Cloud); versioned+locked
```

certs.eknathalabs.com

```
# Layers: network → data → platform → apps, connected via remote-state/data sources
```

```
# Modules: versioned 'golden' modules in a registry (the paved road)
```

WORKFLOW & GUARDRAILS

```
# CI: plan-on-PR (diff comment), gated apply-on-merge with saved plan, CODEOWNERS
```

```
# Auth: OIDC federation, per-env scoped roles, permission boundaries (no static keys)
```

```
# Policy-as-code: Sentinel/OPA org-wide (no public exposure, required tags, encryption)
```

```
# Visibility: Infracost (cost on PRs), scheduled drift detection, audit via Git + state
```

Start with **account-level isolation** (separate accounts per environment) for the strongest blast-radius boundary, then partition state per account × layer × service with an encrypted, versioned, locked backend. Provide **golden modules** in a registry so teams compose a paved road rather than reinventing. Standardize the **workflow** (plan-on-PR, gated apply of saved plans, CODEOWNERS) and **auth** (OIDC, scoped per-env roles, permission boundaries). Enforce **guardrails** with org-wide policy-as-code, and add **visibility** with cost estimation, drift detection, and Git+state audit trails. The objective: self-service infrastructure where the secure, compliant, cost-aware path is the easy default.

Interview tip: structure the answer as isolation (accounts) → state strategy → module/paved-road → workflow → security/auth → policy guardrails → cost/drift visibility. Emphasizing "the safe path is the easy path" and managing the platform itself as code is exactly the staff/principal-level framing for a Platform Engineering interview.

📚 **Official docs**

- Terraform: Provider configuration (aliases)
- HCP Terraform: Recommended practices

Q121 **What do terraform state pull and push do, and why is terraform state push dangerous?** State

🎙 **Spoken answer · ~65 s**

State pull downloads the current state from the configured backend and prints it, which is useful for backups, diffing before and after a risky operation, and for scripts that need to inspect it. State push uploads a local file to replace the remote state. Terraform checks the lineage, which identifies the state's history, and the serial number, which increases with every change, and refuses to push an older or unrelated state unless forced. Pushing is dangerous because the state is the only record that maps configuration to real infrastructure. If I overwrite it with a stale or edited copy, Terraform may forget resources, causing duplicates or orphaned infrastructure, or try to recreate things that already exist. I use push only as a last resort for recovery, always with a fresh pull backup, a state lock held, and a review of the diff. For routine changes I use the dedicated commands, or moved and import blocks.

Likely follow-ups

- What are lineage and serial?
- What should you do before state surgery?

certs.eknathalabs.com

pull prints the current remote state as JSON, which is safe for inspection and backups. push overwrites the remote state with a local file, bypassing normal checks except a serial and lineage comparison. A wrong push can erase knowledge of real resources.

```
terraform state pull > backup-$(date +%F).tfstate
```

```
terraform state list | wc -l
```

```
terraform state push -force fixed.tfstate
```

Interview tip: Take a pull backup before any state surgery, and prefer state mv, rm and import over hand-editing and pushing.

📚 **Official docs**

- Terraform: State

Q122 **How do the removed block and terraform state rm differ for taking a resource out of Terraform without destroying it?** State

🎙 **Spoken answer · ~60 s**

Sometimes I want Terraform to stop managing an object without deleting the real resource, for example when ownership is moving to another team or another state. The older way is the state rm command, which edits the state directly. It works but happens outside review, depends on one person running it against the right workspace, and leaves no history in Git. The newer approach is a removed block in the configuration that names the resource address, with the lifecycle setting that says do not destroy it. When the plan runs, Terraform shows that the resource will be removed from state only, and applying updates the state in a controlled, reviewed way, including in CI and remote runs. I then delete the block after the change is applied everywhere. The same philosophy applies to moved and import blocks, which turn state surgery into code.

Likely follow-ups

- What does destroy = false do?
- When is state rm still acceptable?

state rm is an imperative command that forgets a resource immediately. The removed block with lifecycle destroy = false is declarative, reviewable in a pull request, and applied by the normal workflow, so everyone's state is updated consistently.

```
terraform state rm aws_s3_bucket.legacy
```

```
removed { from = aws_s3_bucket.legacy lifecycle { destroy = false } }
```

```
terraform plan
```

Interview tip: Declarative blocks leave a code trail and work with remote execution, which is why they beat one-off commands on shared infrastructure.

certs.eknathalabs.com

📚 **Official docs**

- Terraform: State
- Terraform: Refactoring (moved blocks)
- Terraform CLI: state mv

Q123 **How do you use terraform plan -generate-config-out to import existing resources, and what review does the generated code need?** State

🎙 **Spoken answer · ~60 s**

Importing used to mean writing the resource block by hand, then running import and iterating on plan until the diff disappeared. Now I add an import block with the target address and the real ID, run plan with the generate-config-out option, and Terraform produces a file with a resource block populated from the live object. That saves time, but the output is only a starting point. It often contains every attribute, including read-only ones and defaults, hardcoded values I want as variables, and sometimes conflicting arguments that fail validation. I clean it up, move it into the right module, replace literals with references, and apply to bring the object into state. Then I run another plan to confirm it shows no changes, which proves the configuration matches reality. For bulk imports, I script discovery, generate blocks programmatically, and review in a pull request.

Likely follow-ups

- Why must the plan be clean after import?
- What attributes should you strip from generated config?

With an import block that has no matching resource block, plan -generate-config-out writes starter HCL based on the real object's attributes. It is a draft: you must remove computed and default noise, parameterise values, and verify the next plan shows no changes.

```
import { to = aws_instance.web  id = "i-0abc123" }
```

```
terraform plan -generate-config-out=generated.tf
```

```
terraform apply
```

```
terraform plan   # expect: No changes
```

Interview tip: The success criterion for any import is a clean plan afterwards, with the config tidy enough to maintain.

📚 **Official docs**

- Terraform: Import
- Terraform CLI: plan
- Terraform CLI: plan (saved plan files)

certs.eknathalabs.com

Q124 **What is the difference between terraform taint and the -replace option, and why is -replace preferred?** State

🎙 Spoken answer · ~62 s

Both approaches are for forcing a healthy-looking resource to be destroyed and recreated, for example a VM with corrupted configuration. The taint command writes a tainted flag into the state, so any later apply, possibly by someone else or by CI, will replace the resource, even though nobody reviewed that in a plan. It is easy to forget a tainted resource and then be surprised. The replace flag is passed to plan or apply for a single run, so the plan clearly shows the resource marked for replacement, the team can review the impact, including dependent resources that may also be recreated, and nothing is stored in state beforehand. If the apply is cancelled, state is not left in an altered condition. That is why the replace option is recommended and taint is deprecated. In an automated pipeline, replacement can also be triggered declaratively with the replace_triggered_by lifecycle argument.

Likely follow-ups

- What does replace_triggered_by do?
- Why can replacement affect dependents?

taint marks a resource in state as tainted so the next apply recreates it, but modifies state without a plan review. -replace=ADDRESS on plan or apply forces replacement for that run, is visible in the plan, and doesn't change state until applied.

```
terraform plan -replace=aws_instance.web
```

```
terraform apply -replace=aws_instance.web
```

```
terraform untaint aws_instance.web
```

Interview tip: taint is deprecated in favour of -replace because it changes state before anyone has reviewed the consequences.

📚 Official docs

- Kubernetes: Taints and tolerations

certs.eknathalabs.com

Q125 **How can large Terraform state files hurt, and what are the signs that it is time to split state?** State

🎙 Spoken answer · ~63 s

The state is read and refreshed on every plan, so thousands of resources mean thousands of API calls and slow feedback. A single lock also means only one person or pipeline can change anything at once, and every apply carries the risk of touching unrelated infrastructure, so a mistake in one area can damage another. I look at warning signs: plan times measured in many minutes, frequent lock contention, engineers afraid to run apply, API rate limit errors, and unrelated teams colliding in one repository. The remedy is to split state along natural boundaries, such as foundational network and identity layers that change rarely, shared platforms, data stores, and individual application stacks that change daily. The layers share information through outputs, remote state or data sources, and I move resources using moved blocks and state migration carefully. Each state should have a clear owner, a permission boundary and a manageable size.

Likely follow-ups

- How do layers share data?
- What is blast radius?

Large state means slow plans and refreshes, long lock times, a big blast radius, noisy diffs, and merge conflicts in review. Signs: plans over several minutes, many unrelated teams editing one root module, and fear of applying. Split by lifecycle, ownership and risk.

```
terraform state list | wc -l
```

```
terraform state list | cut -d. -f1-2 | sort | uniq -c | sort -nr | head
```

```
time terraform plan -refresh=false
```

Interview tip: Split along boundaries that change at different rates, such as network, data, platform and applications.

📚 Official docs

- Terraform: State

certs.eknathalabs.com

Q126 **What is state encryption in OpenTofu, and how does it differ from backend encryption?** State

🎙 Spoken answer · ~62 s

State files can contain secrets in plaintext, such as database passwords and private keys. Backend server-side encryption, like S3 with KMS, encrypts the object at rest, but anyone allowed to read the object through the service gets decrypted data, so it protects mainly against theft of the underlying disks. OpenTofu added client-side state and plan encryption: Tofu encrypts the data before sending it to the backend, using a key provider such as a cloud KMS or a passphrase and an algorithm like AES-GCM, so the stored object is ciphertext and reading it requires access to the key as well as the bucket. This provides defence in depth and separates duties. Key management becomes critical, since losing the key means losing the state. With Terraform itself, I rely on backend encryption, tightly scoped IAM, versioning and audit logging, and keep secrets out of state where possible using ephemeral values.

Likely follow-ups

- What happens if you lose the encryption key?
- Why does SSE not stop an authorised reader?

OpenTofu can encrypt state and plan files client-side using a key provider such as AWS KMS, GCP KMS or a passphrase, so even someone with read access to the bucket sees ciphertext. Backend encryption (SSE) protects data at rest only from outside the storage service.

```
terraform { encryption { key_provider "aws_kms" "main" { kms_key_id = "alias/tf" region = "eu-west-1" } method "aes_gcm" "default" { keys = key_provider.aws_kms.main } state { method = method.aes_gcm.default } } }
```

```
tofu init && tofu plan
```

Interview tip: Be balanced: mention it is an OpenTofu feature, and that with HashiCorp Terraform you rely on backend encryption and strict access control.

📚 Official docs

- Terraform: Backends
- OpenTofu documentation

certs.eknathalabs.com

Q127 **How does S3 native state locking (use_lockfile) compare with the DynamoDB lock table?** Backend

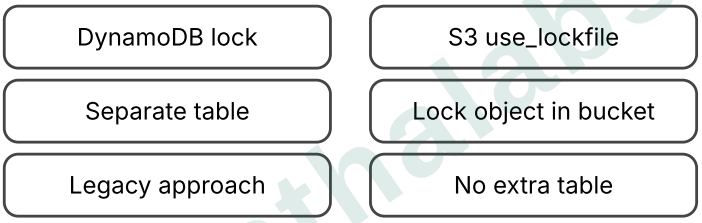
🎙 Spoken answer · ~60 s

Historically the S3 backend stored state in the bucket and used a DynamoDB table with a lock item to prevent concurrent runs. Recent versions support a lock file stored next to the state object in S3, created with a conditional write so that only one client can succeed. That removes a separate service to provision, secure and pay for, and it reduces the permissions required for the pipeline role. The things to check are the minimum tool versions across every engineer and CI runner, because an older client that only knows DynamoDB would not honour the new lock, which is why a transition period using both can be sensible. Also ensure bucket versioning stays on, the role can write and delete the lock object, and that stuck locks are cleaned up deliberately. I remove the DynamoDB table once every consumer is upgraded and verified.

Likely follow-ups

- Why keep both locks during migration?
- How do you clear a stuck S3 lock?

🗺 Lock mechanisms



Newer Terraform and OpenTofu versions can lock S3 state using a .tflock object created with conditional writes, removing the need for a DynamoDB table. It simplifies setup and permissions, but you must migrate carefully and make sure all users run supported versions.

```
terraform { backend "s3" { bucket = "tf-state" key = "prod/network.tfstate" region = "eu-west-1" use_lockfile = true } }
```

```
aws s3 ls s3://tf-state/prod/ | grep tflock
```

```
terraform init -migrate-state
```

Interview tip: During migration you can enable both mechanisms for a while so that older clients still respect the lock.

📚 Official docs

- Terraform: State
- Terraform: S3 backend
- Terraform: State locking

certs.eknathalabs.com

Q128 **How do remote state backends differ on Google Cloud and Azure, and how is locking handled?**

Backend

🎙 Spoken answer · ~58 s

Both clouds have first-class object storage backends. On Google Cloud, the gcs backend writes state objects into a bucket, using a prefix to separate states, and handles locking with a lock object that is created with a precondition so concurrent creators fail. On Azure, the azurerm backend stores state as a blob in a container and takes a lease on the blob while an operation runs. Both integrate with native identity, so pipelines authenticate with workload identity federation or managed identities rather than static access keys. For safety I enable versioning or soft delete so earlier state versions can be recovered, restrict the bucket or storage account with IAM to the pipeline identities, turn on encryption with customer-managed keys where policy demands it, enable logging, and block public access. Separate buckets or containers per environment limit the blast radius.

Likely follow-ups

- How do you recover an older state version?
- Why prefer OIDC over keys here?

The gcs backend stores state in a bucket and uses object generation preconditions for locking. The azurerm backend stores state in a blob container and uses blob leases. Both support encryption, versioning and identity-based auth without keys.

```
terraform { backend "gcs" { bucket = "tf-state-prod" prefix = "network" } }
```

```
terraform { backend "azurerm" { resource_group_name = "tf-rg" storage_account_name = "tfstate123" container_name = "state" key = "prod.tfstate" use_oidc = true } }
```

Interview tip: Turn on object versioning and soft delete so a bad write can be rolled back.

📚 Official docs

- Terraform: State
- Terraform: Backends

certs.eknathalabs.com

Q129 **What can go wrong if different team members use different Terraform versions or backend settings, and how do you standardise them?**

Backend

🎙 Spoken answer · ~63 s

Terraform records its version in the state and may change the state format when a newer version writes to it. After that, older versions refuse to read it, so one developer who runs a local apply with a newer binary can break CI and teammates. Provider versions matter too, since a different provider can plan different changes. To avoid this, I set required_version and required_providers constraints, commit the dependency lock file so every machine installs identical provider versions and hashes, store a tool version file read by a version manager such as tfenv or asdf, and run all applies through CI using a container image with a pinned version. Backend configuration is kept in code or in partial config files supplied by the pipeline, not in individual shell history. Upgrades are planned: upgrade the pinned version in a pull request, run plans for every state, then apply during a controlled window.

Likely follow-ups

- What does the lock file contain?
- How do you upgrade Terraform safely across many states?

A newer version can upgrade the state format so older versions can no longer read it, providers behave differently, and backend arguments drift. Pin required_version, commit the lock file, use a version manager and run through CI to keep one tool version.

```
terraform { required_version = "~> 1.9.0" }
```

```
tfenv install && tfenv use  # reads .terraform-version
```

```
terraform version
```

```
terraform providers lock -platform=linux_amd64 -platform=darwin_arm64
```

Interview tip: A state written by a newer Terraform may be unreadable by an older one, so one engineer's local apply can block everyone else.

📚 Official docs

- Terraform: Backends
- Terraform: Remote state & state storage (team workflows)

certs.eknathalabs.com

Q130 **What is Terraform Cloud remote execution versus local execution, and how does it change who needs cloud credentials?**

Backend

🎙 **Spoken answer · ~57 s**

In remote execution mode, the CLI uploads the configuration to the platform, and the plan and apply run on managed workers. Cloud credentials and sensitive variables live in the workspace, or are obtained dynamically through OIDC federation, so individual engineers do not need production credentials on their laptops. Every run is logged, policy checks run before apply, approvals can be required, and runs are queued so only one runs per workspace at a time. In local execution mode, the workspace is used only as a state backend, and the plan runs wherever the CLI is, using local credentials, which is flexible for debugging but loses centralised control. For production I prefer remote or CI-based execution with short-lived credentials, role-based access on workspaces, and run triggers to chain dependent workspaces, keeping developers on speculative plans for pull requests.

Likely follow-ups

- What are speculative plans?
- What are run triggers?

With remote execution, plan and apply run on HCP Terraform workers, using credentials and variables stored in the workspace. With local execution the CLI runs on your machine and only state is stored remotely. Remote runs centralise secrets, logs, policy checks and approvals.

```
terraform { cloud { organization = "acme" workspaces { name = "prod-network" } } }
```

```
terraform login
```

```
terraform plan   # runs remotely and streams logs
```

Interview tip: Dynamic provider credentials with OIDC let remote runs assume cloud roles without any stored keys.

📚 **Official docs**

- Terraform: Locals
- HCP Terraform documentation

certs.eknathalabs.com

Q131 **How do you protect Terraform state buckets from deletion, tampering and unauthorised reading?** Backend

🎙 Spoken answer · ~59 s

State holds the map of everything Terraform manages and often secrets, so losing or altering it is serious. I protect it in layers. Versioning keeps previous revisions so I can roll back after a bad write or a deletion. Object lock or MFA delete stops even privileged users from permanently removing versions. A bucket policy denies unencrypted uploads and non-TLS access, and denies delete actions to everyone except a controlled break-glass role. Public access blocks are enabled, and the bucket is encrypted with a customer-managed key with a narrow key policy. Read and write permissions are granted per state path to the specific pipeline roles that need them, so a team cannot read another team's state. Access logging and CloudTrail data events provide an audit trail, replication to another region gives disaster recovery, and the bucket lives in a dedicated, hardened account.

Likely follow-ups

- Why use a dedicated account for state?
- How do you recover from an accidental deletion?

Enable versioning, MFA delete or object lock, bucket policies denying delete except from a break-glass role, block public access, encrypt with KMS, restrict read to pipeline roles, log access, and replicate the bucket for disaster recovery.

```
aws s3api put-bucket-versioning --bucket tf-state --versioning-configuration Status=Enabled
```

```
aws s3api put-public-access-block --bucket tf-state --public-access-block-configuration
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=tru
e
```

```
aws s3api put-bucket-policy --bucket tf-state --policy file://deny-unencrypted-and-
delete.json
```

Interview tip: Treat the state bucket as tier-zero infrastructure, with its own account and tightly limited administrators.

📚 Official docs

- Terraform: State

certs.eknathalabs.com

Q132 **How do policy-as-code tools such as OPA Conftest, Sentinel and Checkov fit into a Terraform pipeline?**

Security

🎙 Spoken answer · ~63 s

I use a layered approach. Static analysis tools such as Checkov and tfsec read the code and flag known problems, for example an open security group, unencrypted storage or missing logging, and give developers fast feedback in pre-commit hooks and pull requests. But variables, modules and computed values mean the source does not always show what will really be created. So I also evaluate the rendered plan: convert it to JSON and run Open Policy Agent policies with Conftest, or Sentinel in HCP Terraform, to enforce organisation-specific rules such as required tags, allowed regions, instance types and no public buckets. The policies run after plan and before apply, and a failure blocks the pipeline. Rules have severity levels, with soft-fail overrides requiring approval for exceptions. Policies are versioned and tested like code, and they are complemented by cloud-side guardrails such as service control policies as the last line of defence.

Likely follow-ups

- Why scan the plan as well as the source?
- What are soft-mandatory policies?

Static scanners like Checkov and tfsec inspect HCL for known misconfigurations. Conftest with OPA evaluates the JSON plan against custom Rego rules. Sentinel does the same inside HCP Terraform. Run them between plan and apply to fail unsafe changes.

```
terraform plan -out=tfplan && terraform show -json tfplan > plan.json
```

```
conftest test plan.json -p policy/
```

```
checkov -f plan.json --framework terraform_plan
```

```
tfsec .
```

Interview tip: Scanning the plan JSON catches values only known at plan time, which pure source scanning misses.

📚 Official docs

- Terraform: Tests
- Conftest
- Checkov

certs.eknathalabs.com

Q133 **What are ephemeral resources and write-only attributes in recent Terraform, and how do they keep secrets out of state?** Security

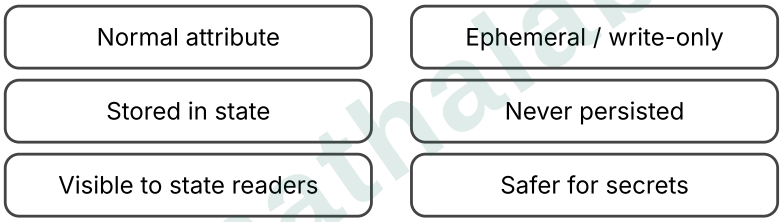
🎙 Spoken answer · ~64 s

For years, any secret passed through Terraform ended up in the state file in plaintext, whether generated by the random provider, read from a data source, or supplied as a variable. Newer Terraform versions introduce ephemeral values: variables, outputs and resources that exist only while a run is in progress and are not written to plan or state. Providers can also expose write-only arguments, which accept a value during apply but never save it. A pattern is to read a secret from a vault with an ephemeral resource and pass it to a write-only password argument on a database, with a version attribute to trigger updates when the secret rotates. That prevents the secret from appearing in state, plan output or logs. It depends on provider support, and the secret still has to be managed and rotated in a proper secrets manager, so I combine it with workload identity and runtime secret retrieval.

Likely follow-ups

- How do you trigger rotation with a write-only attribute?
- What is still stored in state?

🗺 Secrets in state



Ephemeral values exist only during a run and are never stored in plan or state. Write-only attributes accept a value to send to the provider but don't persist it. Together they let you fetch a secret from a vault and set a password without it appearing in state.

```
ephemeral "random_password" "db" { length = 24 }
```

```
resource "aws_db_instance" "main" { password_wo = ephemeral.random_password.db.result
password_wo_version = 1 }
```

```
terraform state show aws_db_instance.main | grep -i password
```

Interview tip: Say it clearly: this reduces secrets in state, but the secret still needs a proper secret manager as its source of truth.

📚 Official docs

- Terraform: State
- Terraform: Sensitive data in state

certs.eknathalabs.com

Q134 **How do you authenticate Terraform in CI to AWS, Azure or GCP without long-lived keys, and what do you restrict?** Security

🎙 Spoken answer · ~59 s

Static access keys in CI secrets are a standing risk, so I use OpenID Connect federation. The CI platform issues a signed token for each job, identifying the repository, branch and workflow. In the cloud I configure an identity provider that trusts that issuer, and a role, service account or app registration whose trust policy accepts tokens only with specific claims, such as the exact repository and the main branch or a protected environment. The job exchanges the token for temporary credentials that expire within an hour. I split roles by purpose: a plan role that can read state and describe resources, used for pull requests, and an apply role with write permissions that only the protected branch or an approved environment can assume. Permissions are least-privilege and scoped to the resources that stack manages, and assumption events are logged and monitored.

Likely follow-ups

- Why separate plan and apply roles?
- What claims would you trust?

Use OIDC federation: the CI system presents a short-lived signed token, and the cloud exchanges it for temporary credentials on a role whose trust policy limits repository, branch and environment. Use separate roles for plan (read-mostly) and apply.

```
aws iam create-open-id-connect-provider --url https://token.actions.githubusercontent.com --client-id-list sts.amazonaws.com
```

```
az ad app federated-credential create --id <app> --parameters @fed.json
```

```
gcloud iam workload-identity-pools create ci-pool --location=global
```

Interview tip: Condition the trust on the exact repo and branch or environment. A loose subject claim lets any repo in the organisation assume your role.

📚 Official docs

- GitHub Actions: OpenID Connect

certs.eknathalabs.com

Q135 **How do you use data sources to read secrets from Vault, Secrets Manager or Key Vault, and what are the risks?** Security

🎙 Spoken answer · ~60 s

A data source for a secret version reads the value from the secret manager during a run. This is convenient, for example to configure a database or a provider, but Terraform stores data source results in state, so the secret ends up in plaintext there, and also may appear in plan output unless marked sensitive. Anyone who can read the state can read the secret. The safer approach is to avoid passing values through Terraform at all. I give the workload an identity and a permission to read the secret, and pass only the secret's ARN or name, for example through the ECS or Kubernetes secret reference mechanism, so the platform injects it at runtime. When a value is unavoidable, I use ephemeral resources or write-only arguments if supported, mark variables and outputs sensitive, lock down state access, and rotate the secret regularly.

Likely follow-ups

- Why is sensitive = true not enough?
- How does ECS inject secrets without Terraform seeing them?

Data sources read secret values at plan or apply time, but the values are stored in state in plaintext. Safer patterns reference secret ARNs or names and let the application fetch them at runtime, or use ephemeral resources. Never output secrets.

```
data "aws_secretsmanager_secret_version" "db" { secret_id = "prod/db" }
```

```
resource "aws_ecs_task_definition" "app" { container_definitions = jsonencode([{ secrets =
[{ name = "DB_PASSWORD", valueFrom = aws_secretsmanager_secret.db.arn }] }]) }
```

Interview tip: Pass the secret's reference to the workload, not its value. Then neither state nor plan holds the secret.

📚 Official docs

- AWS Secrets Manager
- Azure Key Vault
- HashiCorp Vault documentation

certs.eknathalabs.com

Q136 **How do you verify the supply chain of Terraform providers and modules, including signatures and checksums?**

Security

🎙 Spoken answer · ~61 s

Providers are executables that run with your cloud credentials, and modules are code that creates infrastructure, so both are supply chain risks. Terraform verifies provider signatures from the registry and records the checksums of the exact packages in the lock file, so subsequent installs on any machine must match, and I commit that file and generate hashes for every platform used in CI and laptops. I pin exact or tightly bounded versions and update through reviewed pull requests with automation such as Renovate. For modules I pin to immutable versions or commit hashes rather than a branch, prefer modules from the internal private registry that have been reviewed, and read the source of third-party ones for provisioners, external data sources or unexpected resources. Mirroring providers and modules internally gives control over what is available and protects against deletion, and dependency scanning tools check for known vulnerabilities.

Likely follow-ups

- Why pin modules to commit hashes?
- What is a provider mirror?

The registry provides signed provider releases and the lock file records hashes for each platform. Pin versions, commit the lock, use a private registry or mirror for modules and providers, review module source, and scan dependencies for known issues.

```
terraform init -upgrade
```

```
cat .terraform.lock.hcl
```

```
terraform providers mirror ./mirror
```

```
terraform { required_providers { aws = { source = "hashicorp/aws", version = "~> 5.60" } }
}
```

Interview tip: A filesystem or network mirror also protects builds from registry outages and tag changes.

📚 Start here (official docs)

- Terraform documentation

certs.eknathalabs.com

Q137 **How do you restrict what Terraform itself can do, using IAM permission boundaries and separate roles per stack?** Security

🎙 Spoken answer · ~61 s

Terraform needs broad permissions to create infrastructure, so the role it uses is a prime target. I avoid one all-powerful role for everything. Each stack or environment has its own role limited to the services it manages and, where possible, to resource name prefixes or tags. A permission boundary caps the maximum permissions of any role Terraform may create, which prevents privilege escalation by creating an admin role. Explicit denies protect critical assets such as audit logs, the state bucket and security tooling. Organization-level service control policies set the outer limits. To size the permissions, I start with generous access in a sandbox, use last accessed data and policy generation to see what is actually used, and tighten. Plan jobs run with a read-only role, and apply jobs run only from the protected branch with approvals. All role usage is logged and alerts fire on unusual actions.

Likely follow-ups

- What does a permission boundary do?
- Why use a read-only role for plan?

Give each state its own role with only the services and resource prefixes it manages, add a permission boundary so it cannot create more powerful roles, deny destructive actions on protected resources, and use SCPs as the outer guardrail.

```
aws iam put-role-permissions-boundary --role-name tf-app-prod --permissions-boundary
arn:aws:iam::123456789012:policy/TerraformBoundary
```

```
aws iam simulate-principal-policy --policy-source-arn <role-arn> --action-names
iam:CreateUser s3:DeleteBucket
```

```
aws iam generate-service-last-accessed-details --arn <role-arn>
```

Interview tip: A Terraform role with AdministratorAccess is effectively a root key stored in CI. Start from last-accessed data and trim.

📚 Official docs

- AWS IAM: Permissions boundaries
- AWS provider: authentication and configuration

certs.eknathalabs.com

Q138 **What are good module composition patterns, and why prefer small focused modules wired together by a root module?**

Modules

🎙 Spoken answer · ~60 s

The best modules represent a meaningful building block, such as a network, a database with its subnet group and parameter group, or a service with its load balancer, and expose a small, stable interface of variables and outputs. The root module wires them together, passing the outputs of one as inputs to another, which makes dependencies explicit and the graph easy to understand. I avoid very deep nesting because it hides where resources come from and makes debugging hard, and avoid passing entire objects or configuration blobs through many layers. Modules should not configure providers internally, they should receive them, and should validate inputs and document outputs. I resist wrapping a single resource just to rename arguments, because that adds indirection with no value. Versioning modules and testing them separately lets teams consume them safely, and composition lets different environments reuse the same blocks.

Likely follow-ups

- Why shouldn't modules configure providers?
- What makes a module interface stable?

Keep modules focused on one capability with a clear input and output contract, compose them in root modules, avoid deep nesting, and don't wrap a single resource without adding value. Pass dependencies explicitly via variables and outputs.

```
module "network" { source = "./modules/network" cidr = var.cidr }
```

```
module "app" { source = "./modules/app" subnet_ids = module.network.private_subnet_ids }
```

```
terraform graph | dot -Tsvg > graph.svg
```

Interview tip: A thin wrapper around one resource is usually a smell. A module should encapsulate a decision or a pattern, not just rename arguments.

📚 Official docs

- Terraform: Modules
- Terraform: Module composition

certs.eknathalabs.com

Q139 **How do you use for_each and count on modules, and what limitations apply?** Modules

🎙 Spoken answer · ~65 s

Since Terraform version 0.13, modules accept count and for_each, so I can instantiate the same module for each item in a map or set, for example one per microservice or per environment. Each instance is addressed in state by its key, such as module dot service with the key in brackets, so adding or removing one item affects only that instance. With count, the address is the numeric index, so removing an element in the middle shifts all later indexes and causes unwanted destroy and recreate actions, which is why I prefer for_each with stable keys. The keys must be known at plan time, they cannot depend on values only known after apply, which sometimes forces a two-stage apply or a redesign. Renaming a key looks like a delete and a create, so I add a moved block to preserve the resource. Also, remember that providers can't be passed dynamically per instance in old versions.

Likely follow-ups

- Why is count risky for lists that change?
- What does 'known after apply' break?

Modules support count and for_each, creating multiple instances addressed by index or key. Each instance has its own state addresses. Keys must be known at plan time, and changing a key destroys and recreates that instance unless you use moved blocks.

```
module "service" { for_each = var.services  source = "./modules/service"  name = each.key
config = each.value }
```

```
terraform state list | grep 'module.service'
```

```
moved { from = module.service["old"] to = module.service["new"] }
```

Interview tip: Use stable keys such as names, not list indexes, so inserting an item does not shift everything.

📚 Official docs

- Terraform: for_each

certs.eknathalabs.com

Q140 **How does terraform test work, and what can you assert with it?** Modules

🎙 Spoken answer · ~63 s

Terraform has a built-in test framework. Test files with the tftest extension contain run blocks. Each run block executes either a plan or an apply of the module under test with specific variable values, and then evaluates assert blocks that check conditions on outputs, resource attributes or the number of resources, failing with a clear message. Because tests that apply create real infrastructure and are cleaned up afterwards, I keep them for important integration scenarios in a sandbox account. For quick feedback, I use plan-only runs and mock providers, which simulate provider responses without touching a cloud, to check logic like conditionals, naming and count calculations. Expected failures can be tested with expect_failures for variable validations. I run tests in CI on every pull request for shared modules, and complement them with static analysis and policy checks, while Terratest or Kitchen-style tests remain useful for end-to-end validation of live behaviour.

Likely follow-ups

- What are mock providers?
- How do you test a validation rule?

terraform test runs .tftest.hcl files containing run blocks that apply or plan a configuration with given variables and then check assertions on outputs and resources. It can use mock providers for fast unit tests, and real applies for integration tests.

```
terraform test
```

```
terraform test -filter=tests/network.tftest.hcl
```

```
run "creates_private_subnets" { command = plan variables { cidr = "10.0.0.0/16" } assert { condition = length(aws_subnet.private) == 3 error_message = "expected 3 private subnets" } }
```

Interview tip: Use command = plan with mock providers for fast, free checks, and reserve real applies for a few key integration paths.

📚 Official docs

- Terraform: Tests

certs.eknathalabs.com

Q141 **When would you use terraform_data or the null_resource replacement, and what are the alternatives to provisioners?** Modules

🎙 Spoken answer · ~60 s

terraform_data is a built-in resource that needs no provider and can hold a value, expose it as output, and trigger replacement when the value in triggers_replace changes. It replaces the old null_resource from the null provider, with simpler behaviour. It is typically used to attach a provisioner or to create a dependency or change-trigger that has no real cloud object behind it. Provisioners such as local-exec and remote-exec run scripts, but they are imperative, not tracked in state beyond success or failure, and make runs fragile and hard to reproduce. Alternatives are better: cloud-init or user data for instance bootstrap, pre-baked images from Packer, configuration management or Kubernetes operators for software setup, and native provider resources or data sources for the actual capability. When I do need a script, I keep it idempotent, make triggers explicit and treat it as technical debt to remove.

Likely follow-ups

- Why are provisioners a last resort?
- What does triggers_replace do?

terraform_data is the built-in replacement for null_resource, storing input and triggering replacement on changes with triggers_replace. Provisioners are a last resort; prefer cloud-init or user data, configuration management, images, or the provider's native resources.

```
resource "terraform_data" "bootstrap" { triggers_replace = [aws_instance.web.id, filesha256("scripts/init.sh")] provisioner "local-exec" { command = "./scripts/init.sh ${aws_instance.web.private_ip}" } }
```

```
terraform state list | grep terraform_data
```

Interview tip: HashiCorp documents provisioners as a last resort because they are not declarative and failures leave resources tainted.

📚 Official docs

- Terraform: Provisioners (last resort)

certs.eknathalabs.com

Q142 **How do you design module outputs and variable types to keep interfaces stable as modules evolve?**

Modules

🎙 Spoken answer · ~57 s

A module's variables and outputs are its public API, so I design them as carefully as any library interface. For inputs, I use precise types, with objects and optional attributes with defaults for grouped settings, add validation blocks that fail early with helpful messages, and write descriptions. I keep the number of required inputs small and provide sensible secure defaults. For outputs, I return only what consumers need, such as identifiers, endpoints and ARNs, and avoid dumping the whole resource object, which couples callers to provider internals. Changes follow semantic versioning: adding optional inputs and new outputs is non-breaking, while renaming, removing or changing a type is a major version with a migration guide and, where resource addresses change, moved blocks. Automated tests and examples in the module repository act as documentation and a safety net.

Likely follow-ups

- What makes a change breaking?
- Why avoid returning entire resource objects?

Expose only what consumers need, use typed objects with optional attributes and defaults, validate inputs, avoid leaking provider-specific structures, and treat outputs as a public API. Deprecate and version rather than rename or remove abruptly.

```
variable "db" { type = object({ engine = string, size = optional(string, "db.t3.medium"), multi_az = optional(bool, true) }) }
```

```
variable "env" { type = string validation { condition = contains(["dev","stage","prod"], var.env) error_message = "env must be dev, stage or prod" } }
```

```
output "endpoint" { value = aws_db_instance.main.endpoint description = "Writer endpoint" }
```

Interview tip: Adding an optional input or a new output is backward compatible. Renaming or removing one is a breaking change that deserves a major version.

📚 Official docs

- Terraform: Output values

certs.eknathalabs.com

Q143 What does the provider plugin cache do, and how does it speed up CI?

Performance

🎙 Spoken answer · ~61 s

The AWS and other large providers can be hundreds of megabytes, and each time init runs in a fresh directory it downloads them again. In a repository with dozens of Terraform roots, or on CI runners that start clean for every job, this wastes a lot of time and bandwidth and can hit registry rate limits. The plugin cache directory solves this: when it is configured, init stores each provider version once and links to it from the working directory. I set the environment variable in the CI image, persist the directory using the CI system's cache keyed by the hash of the lock file, and make sure the lock files exist so that the checksums are verified. For fully offline or restricted environments, a provider mirror gives the same benefit with more control. This is a simple change that often cuts minutes from every pipeline run.

Likely follow-ups

- How does a provider mirror differ?
- What should the CI cache key be?

By default every working directory downloads its own copies of providers during init. Setting plugin_cache_dir (or TF_PLUGIN_CACHE_DIR) shares downloaded providers across directories and builds, saving bandwidth and minutes, especially in monorepos with many states.

```
export TF_PLUGIN_CACHE_DIR=$HOME/.terraform.d/plugin-cache
```

```
mkdir -p $TF_PLUGIN_CACHE_DIR
```

```
terraform init   # second run reuses cached providers
```

```
du -sh .terraform/providers
```

Interview tip: Cache the directory between CI runs, keyed on the lock file hash, to avoid redownloading large providers.

📚 Start here (official docs)

- Terraform documentation

certs.eknathalabs.com

Q144 **When is -refresh=false safe to use, and what are the risks?** Performance

🎙 Spoken answer · ~62 s

During a normal plan, Terraform calls the provider to read the current state of each resource and updates its in-memory view before computing differences. For large states this is the slowest part, so skipping it with the refresh flag set to false gives much faster plans. The risk is that Terraform then compares configuration to the stored state, not to reality, so if something changed outside Terraform it will not be noticed, and the plan may be wrong: it might try to create something that exists or miss a deletion. I use it when iterating on code in a development environment, or immediately after an apply when I know state is current. For production applies, and for drift checks, I always run a full plan, and I schedule separate refresh-only plans to detect drift. A faster alternative for big configurations is splitting state so full refreshes are smaller.

Likely follow-ups

- What does refresh-only do?
- Why not use refresh=false for production?

-refresh=false skips reading real infrastructure and trusts state, making plans much faster. It is safe when state is known to be fresh, for example in a quick re-plan after a recent apply, but dangerous for detecting drift or in production decisions.

```
terraform plan -refresh=false
```

```
terraform plan -refresh-only
```

```
terraform apply -refresh=false -target=module.app
```

Interview tip: Use it for iteration while developing, but run a full refreshing plan before the final apply.

📚 Official docs

- Terraform CLI: plan

certs.eknathalabs.com

Q145 **How can many for_each resources and large data lookups slow Terraform, and how do you design around it?** Performance

🎙 Spoken answer · ~61 s

Terraform builds a graph with a node per resource instance and refreshes each one. A for_each over a thousand security group rules or DNS records can mean thousands of API reads on every plan, and data sources inside the loop add more, so runtimes grow and rate limits are hit. Ways to design around it are to use resources that manage many items in a single object where the provider supports it, such as a single policy document with many statements, to fetch shared lookups once into a local or a data source outside the loop and index into it, to avoid per-item data sources, and to split very large sets across multiple states or modules by ownership. I can tune parallelism carefully, but the providers' rate limits matter more. Measuring the actual API calls with debug logging helps me find the dominant cost, rather than guessing.

Likely follow-ups

- What does -parallelism change?
- How do provider rate limits show up?

Each instance is a separate graph node and API call set. Thousands of instances, or data sources that run for each, make plans slow and hit API limits. Batch via resources that accept lists, move to separate states, cache lookups with locals, and avoid per-item data sources.

```
terraform graph | grep -c '\[label'
```

```
TF_LOG=DEBUG terraform plan 2>&1 | grep -c 'GET\|POST'
```

```
terraform plan -parallelism=20
```

Interview tip: Count API calls with debug logs. The number of requests, not Terraform CPU, is usually the cost driver.

📚 Official docs

- Terraform: for_each

certs.eknathalabs.com

Q146 **What are the pros and cons of Terragrunt for reducing duplication, and when is it worth adopting?**

Performance

🎙 Spoken answer · ~61 s

Terragrunt is a thin wrapper that generates backend and provider blocks from a shared parent configuration, so each stack directory only needs its inputs, and it understands dependencies between directories so run-all commands apply things in the right order and can pass outputs along. This removes lots of copy-pasted backend settings and makes environments like dev, stage and prod, each with dozens of small states, manageable. The costs are an extra tool and language layer for new team members to learn, harder debugging because errors come through the wrapper, version compatibility to track, and some overlap with newer native features. It pays off when there are many stacks that follow the same pattern across accounts and regions. For small setups, plain Terraform with modules, a clean directory layout and a CI pipeline is simpler. I decide based on the number of states and the team's skills.

Likely follow-ups

- What does run-all do?
- What native features overlap with Terragrunt?

Terragrunt wraps Terraform to keep backend and provider configuration DRY, orchestrate dependencies between stacks and run commands across many directories. It adds another tool and abstraction layer, so adopt it when you have many similar stacks and environments.

```
terragrunt run-all plan
```

```
terragrunt run-all apply --terragrunt-non-interactive
```

```
include "root" { path = find_in_parent_folders() }
```

```
dependency "vpc" { config_path = "../vpc" }
```

Interview tip: Alternatives: Terraform workspaces plus modules, Terraform Cloud stacks, or plain CI orchestration. Choose the simplest that fits.

📚 Official docs

- Terragrunt documentation

certs.eknathalabs.com

Q147 **How do preconditions, postconditions and check blocks improve safety, and how do they differ?**

Production

🎙 **Spoken answer · ~60 s**

These features move validation closer to where assumptions are made. A precondition in a resource or data source lifecycle block is evaluated before creating or updating it, so I can verify that a selected AMI has the right architecture or that a subnet belongs to the expected VPC, and fail with a clear message before touching anything. A postcondition is evaluated after, checking that the result meets expectations, for example that a looked-up resource has a required tag, protecting downstream consumers. Both stop the run on failure. A check block is different: it contains assertions and optional scoped data sources, evaluated at the end of plan and apply, and failures appear as warnings without blocking, which is suited to verifying that a deployed endpoint responds or a certificate is not close to expiry. Variable validation covers simple input rules, and together these make configurations self-checking.

Likely follow-ups

- Variable validation versus preconditions?
- Why not block on a check failure?

Preconditions and postconditions inside lifecycle blocks fail the run when assumptions about inputs or results are violated. check blocks run assertions, including on data sources, that only warn rather than stop, and are good for ongoing health validation.

```
resource "aws_instance" "web" { ami = var.ami lifecycle { precondition { condition = data.aws_ami.selected.architecture == "x86_64" error_message = "AMI must be x86_64" } } }
```

```
check "site_up" { data "http" "home" { url = "https://example.com" } assert { condition = data.http.home.status_code == 200 error_message = "home page down" } }
```

Interview tip: Use conditions for assumptions that must hold before or after change, and check blocks for monitoring-style assertions that should not block applies.

📚 **Official docs**

- Terraform: Custom conditions & variable validation

certs.eknathalabs.com

Q148 **How do prevent_destroy and ignore_changes work, and what are their pitfalls?** Production

🎙 Spoken answer · ~57 s

The lifecycle block lets me adjust Terraform's behaviour for a resource. Setting prevent_destroy to true makes Terraform error out if a plan would destroy that resource, including a replacement, which protects databases and state buckets from accidental removal. The pitfalls are that it only applies while the code is present, since deleting the resource block also deletes the protection, and it can obstruct intentional teardown, so people sometimes remove it casually. It should be backed by deletion protection on the cloud resource itself and by IAM. ignore_changes tells Terraform not to react to differences in specific attributes. That is appropriate for fields managed elsewhere, such as an autoscaler adjusting desired capacity or a security tool adding tags. Used too broadly it hides drift that matters, so I list exact attributes, document why, and review them periodically.

Likely follow-ups

- Why also enable deletion protection on the resource?
- What happens if you delete the resource block?

prevent_destroy makes any plan that would destroy the resource fail, a guard against accidents, but it blocks legitimate destroys until removed. ignore_changes tells Terraform to disregard drift in listed attributes, useful for fields changed by other systems, but it can hide real problems.

```
lifecycle { prevent_destroy = true }
```

```
lifecycle { ignore_changes = [tags["LastScanned"], desired_capacity] }
```

```
terraform plan
```

Interview tip: Use ignore_changes narrowly on named attributes, never on whole blocks, and leave a comment explaining who else changes them.

📚 Official docs

- Terraform: lifecycle meta-argument

certs.eknathalabs.com

Q149 **What does create_before_destroy do, and what problems can it cause with unique names?**

Production

🎙 Spoken answer · ~60 s

By default, when a change forces replacement, Terraform destroys the old object first and then creates the new one, which causes downtime if something depends on it. With create_before_destroy the order is reversed: the new object is created, dependants are updated to point to it, and then the old one is removed. That suits certificates, launch templates, security groups and instances behind load balancers. The common problem is name collisions: if the resource must have a unique name, the replacement cannot be created while the old exists, so I use name_prefix to let the provider generate unique names, or add a random suffix that changes with the replacement trigger. Another subtlety is that the flag is inherited by upstream dependencies, which can lead to unexpected duplicates and quota usage, and some resources, like those with unique constraints on IP or port, cannot support it.

Likely follow-ups

- Why does name_prefix help?
- What quota issues can arise?

🗺 create_before_destroy



It creates the replacement before destroying the old resource, giving zero-downtime replacement for things like launch templates, certificates and load balancer listeners. If the resource has a unique name, both can't exist at once, so use name_prefix or random suffixes.

```
lifecycle { create_before_destroy = true }
```

```
resource "aws_launch_template" "app" { name_prefix = "app-" lifecycle {
create_before_destroy = true } }
```

```
terraform plan | grep -E 'must be replaced|create replacement'
```

Interview tip: The setting propagates to dependencies, so resources the replaced one depends on may also be forced into create-before-destroy.

📚 Official docs

- Terraform: lifecycle meta-argument

certs.eknathalabs.com

Q150 **What does replace_triggered_by do, and how do you use it to roll dependent resources?** Production

🎙 Spoken answer · ~58 s

Sometimes a resource needs to be rebuilt because something it depends on changed, but the dependency is not in an argument that triggers replacement. For example, a running instance does not change when a launch template gets a new version, or a container is not restarted when a secret version changes. The replace_triggered_by lifecycle argument lists resources or attributes, and when any of them is updated or replaced, Terraform plans a replacement of this resource too. I use it for rolling instances after a configuration change, recreating a job after a schema change, or tying replacement to a version number held in terraform_data so I can trigger it deliberately. It must be used carefully, because it can cascade into large replacements, and I check the plan to see the full set of resources affected. Combining it with create_before_destroy limits downtime.

Likely follow-ups

- What references are allowed?
- How do you limit cascading replacements?

replace_triggered_by makes a resource be replaced whenever a referenced resource or attribute changes. It is useful when something must be recreated to pick up a change that Terraform wouldn't otherwise see, like instances after a launch template or secret version change.

```
resource "aws_instance" "app" { ami = var.ami lifecycle { replace_triggered_by = [aws_launch_template.app.latest_version, terraform_data.config_version] } }
```

```
terraform plan | grep 'forces replacement'
```

Interview tip: Pair it with terraform_data holding a version value so you can force a controlled roll by bumping one variable.

📚 Official docs

- Terraform: lifecycle meta-argument

certs.eknathalabs.com

Q151 **How do you design blue-green or canary infrastructure changes with Terraform, for example a new Kubernetes node group or database version?**

Production

🎙 Spoken answer · ~59 s

Terraform is declarative, so zero-downtime strategies come from how I model the infrastructure. For blue-green, I define both environments, often as two instances of the same module with different version inputs, and put a traffic switch in front: weighted target groups on a load balancer, weighted DNS records, or a service mesh route. A variable controls the weight, so a pull request that moves ten percent, then fifty, then one hundred percent to green is small and reviewable, and rollback is changing the variable back. I validate health checks and metrics between steps. Only after an observation period do I remove the old environment in a separate change. For stateful resources such as databases, replacement is risky, so I prefer replicas, blue-green features of the managed service or migration tooling, with lifecycle protections and backups. The key is making each step reversible.

Likely follow-ups

- Why destroy the old stack in a separate change?
- How do you handle databases in blue-green?

Create the new stack alongside the old, shift traffic gradually via weighted DNS, load balancer rules or target groups, verify health, then retire the old. Control the shift with variables or separate applies, and keep the rollback path as a one-variable change.

```
resource "aws_lb_listener_rule" "canary" { action { type = "forward" forward { target_group { arn = aws_lb_target_group.blue.arn weight = 100 - var.green_weight } target_group { arn = aws_lb_target_group.green.arn weight = var.green_weight } } } }
```

```
terraform apply -var green_weight=10
```

Interview tip: Treat destroying the old environment as a separate, later change, after the observation window passes.

📚 Official docs

- Amazon EKS: Managed node groups
- Google SRE Workbook: Canarying Releases
- Argo Rollouts

certs.eknathalabs.com

Q152 **How do you handle the Terraform lifecycle when a provider defaults or API behaviour change and cause perpetual diffs?** Production

🎙 Spoken answer · ~65 s

A perpetual diff happens when each plan shows a change that applying never fixes. I run plan and read exactly which attribute differs, then compare the value in configuration with the value in state. Frequent causes are an API that normalises what I send, for example reordering or lower-casing values, JSON policies with different whitespace or key order, a provider default that is filled in when I leave an argument empty, and sets or lists whose order changes. The fix is to write the configuration in the canonical form, use jsonencode or policy document data sources for JSON, specify the defaulted value explicitly, or switch from a list to a set. If the provider has a bug, I look for an issue and a fixed version, and as a workaround use ignore_changes on that specific attribute, with a comment linking the issue. A noisy plan trains people to ignore diffs, so I treat it as a bug.

Likely follow-ups

- Why do JSON policies cause diffs?
- When is ignore_changes acceptable here?

Identify the attribute that differs between config and state, check provider changelogs and issues, set the value explicitly to match the API's normalised form, use ignore_changes as a targeted workaround, or upgrade the provider that fixes it.

```
terraform plan -no-color | grep -B2 -A6 '~ '
```

```
terraform state show aws_s3_bucket.logs
```

```
TF_LOG=DEBUG terraform plan 2> debug.log
```

Interview tip: Common culprits are normalised JSON policy documents, case differences, ordering of lists and sets, and defaults the provider fills in.

📚 Official docs

- Terraform: lifecycle meta-argument

certs.eknathalabs.com

Q153 **How does Atlantis work for pull-request automation, and what are its security considerations?** CI/CD

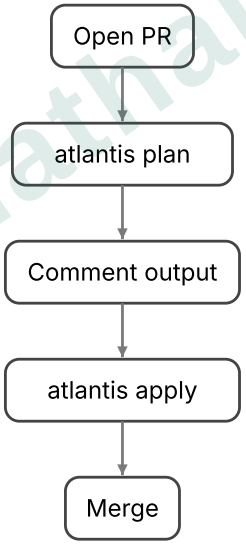
🎙 Spoken answer · ~60 s

Atlantis is a self-hosted server that connects to the version control system. When a pull request changes Terraform files, it automatically runs plan for the affected projects and comments the output on the pull request. After review, an authorised person comments to apply, and Atlantis runs the apply, locks the project to avoid conflicts and then the change is merged. The advantages are visibility, locking and a consistent execution environment. The security considerations are significant: the server runs Terraform code from pull requests with real cloud credentials, and a malicious change, for example an external data source or provisioner, can exfiltrate those credentials during plan. So I restrict which repositories and users can trigger runs, require approvals and passing checks before apply, use least-privilege credentials per project, run workers in isolated network segments with ephemeral workspaces, and protect the webhook secret and server access.

Likely follow-ups

- Why can a plan be dangerous?
- How does Atlantis locking work?

🗺 Atlantis



Atlantis listens to pull request webhooks, runs terraform plan when code changes and posts the output, then runs apply when a reviewer comments atlantis apply. It holds credentials and executes arbitrary code, so restrict who can trigger it, require approvals, and isolate workers.

```
atlantis plan -d environments/prod
```

```
atlantis apply -d environments/prod
```

```
atlantis.yaml: projects: - name: prod dir: environments/prod apply_requirements: [approved, mergeable]
```

Interview tip: Anyone who can open a pull request can make Atlantis run code with its cloud credentials, so repo access equals credential access.

certs.eknathalabs.com

📚 **Official docs**

- Atlantis documentation

Q154 **How do you add cost estimation with Infracost to a Terraform pull request workflow?** CI/CD

🎙 **Spoken answer · ~60 s**

Cost problems are cheapest to fix before deployment. Infracost runs in the pipeline, parses the Terraform plan or configuration, matches resources to public price lists, and calculates monthly cost. It compares against the base branch and comments the difference on the pull request, for example a new instance type or a bigger database adds a certain amount. Reviewers see cost next to the code diff. I add policies, such as requiring approval from finance or a platform lead when the monthly increase exceeds a threshold, and tag resources so costs map to teams after deployment. The limitations are that estimates cover only resources whose prices it knows, usage-based items like data transfer, requests and storage growth require assumptions supplied in a usage file, and reserved pricing or discounts may not be reflected. So I treat the numbers as directional and reconcile against actual billing regularly.

Likely follow-ups

- Why are usage-based costs hard to estimate?
- What threshold would you set for approvals?

Infracost reads the plan or code, prices resources with cloud pricing data, and posts a diff of monthly cost on the pull request. Teams set thresholds and policies so large increases need approval, building cost awareness into review.

```
infracost breakdown --path plan.json
```

```
infracost diff --path . --compare-to infracost-base.json
```

```
infracost comment github --path infracost.json --repo org/repo --pull-request 123 --github-token $TOKEN
```

Interview tip: Usage-based costs, such as data transfer or requests, need usage estimates in a config file, or they will be underreported.

📚 **Start here (official docs)**

- Terraform documentation

certs.eknathalabs.com

Q155 **How do you securely store, pass and verify the saved plan file between plan and apply jobs in CI?** CI/CD

🎙 Spoken answer · ~59 s

The point of a saved plan is to guarantee that apply performs exactly what was reviewed. The plan job writes the plan to a file, publishes a human-readable summary for reviewers, and uploads the file as an artifact. The apply job downloads that exact artifact, optionally verifies a checksum recorded earlier, and runs apply on it with no further prompts. If anything changed in the state in between, Terraform detects the mismatch and refuses to apply, which forces a fresh plan. Because plan files can contain sensitive values, I store them in encrypted storage with limited access, short retention, and never attach them to public logs or pull request comments. The same version of Terraform and providers must be used in both jobs, which I enforce with the same container image and lock file. Approvals sit between the jobs for protected environments.

Likely follow-ups

- Why must tool versions match between jobs?
- What happens if state changes after the plan?

Write the plan with -out, store it as a protected artifact (encrypted, short retention, access-limited), and apply exactly that file so what was reviewed is what runs. The plan can contain secrets, so treat it like state.

```
terraform plan -out=tfplan -input=false
```

```
sha256sum tfplan > tfplan.sha256
```

```
terraform show -json tfplan | jq '.resource_changes[] | select(.change.actions!=["no-op"])
| .address'
```

```
terraform apply -input=false tfplan
```

Interview tip: If the state changes between plan and apply, Terraform refuses the stale plan, which is the safety you want.

📚 Official docs

- Terraform CLI: plan (saved plan files)

certs.eknathalabs.com

Q156 **How do you detect and alert on Terraform drift on a schedule, and what do you do when drift is found?** CI/CD

🎙 Spoken answer · ~62 s

I schedule a pipeline, for example nightly, that runs a refresh-only plan for every state using a read-only role, with the detailed exit code option so a result of two means there are differences. When drift is found, the job posts the diff to the owning team's channel with a link to the run. The team then decides what is right. If the change was an unauthorised or accidental console edit, I reapply the code to restore the intended state. If the change was legitimate, such as an emergency fix, I update the code through a pull request so the code and reality match, and then the plan returns to clean. Benign fields changed by other systems are excluded with ignore_changes. I also reduce drift at the source by restricting console write access, using service control policies, and logging who made changes. Trend reports show which stacks drift most.

Likely follow-ups

- What does exit code 2 mean?
- How do you reduce drift at the source?

Run scheduled refresh-only plans with -detailed-exitcode in CI for each state. Exit code 2 means differences. Notify the owning team with the diff, then decide: revert the manual change by applying, or adopt it into code via a reviewed pull request.

```
terraform plan -refresh-only -detailed-exitcode -input=false; echo $?
```

```
terraform plan -detailed-exitcode -input=false
```

```
terraform apply -refresh-only
```

Interview tip: Classify drift: benign (autoscaler fields), risky (security groups) and malicious (IAM). Route alerts accordingly.

📚 Official docs

- Terraform CLI: plan

certs.eknathalabs.com

Q157 **How do you structure a reusable GitHub Actions or GitLab pipeline for Terraform across many repositories?** CI/CD

🎙 Spoken answer · ~56 s

Duplicating pipeline YAML in dozens of repositories leads to drift, forgotten security steps and painful upgrades. Instead, the platform team maintains a reusable workflow, or a GitLab include template, that encapsulates the standard stages: format check, validate, linting, security scans, plan with a comment on the pull request, policy checks, and an apply job gated on the main branch and environment approvals. Each repository calls it with a few inputs such as the working directory, the role to assume and variables. Credentials come from OIDC, not stored secrets. The shared workflow is versioned with tags, consumers pin a version, and updates are rolled out through automated pull requests with release notes. This gives consistent guardrails, easier audits and one place to improve caching, concurrency controls, and notifications. Exceptions are handled through inputs, not forks.

Likely follow-ups

- Why pin the reusable workflow version?
- How do you roll out a pipeline change to 50 repos?

Create a shared, versioned reusable workflow or template that does fmt, validate, scan, plan and apply with standard guards, called from each repo with a few inputs. Central control means improvements and security fixes roll out everywhere through version bumps.

```
uses: org/platform-workflows/.github/workflows/terraform.yml@v3
```

```
with: working-directory: environments/prod  role-arn: ${{ vars.TF_ROLE }}
```

```
terraform fmt -check -recursive && terraform validate && tflint && checkov -d .
```

Interview tip: Pin the shared workflow to a tag or commit and update it deliberately, because it runs with your cloud credentials.

📚 Official docs

- GitHub Actions documentation

certs.eknathalabs.com

Q158 **terraform apply fails with 'Error acquiring the state lock' and the lock info shows a run from hours ago. What do you do?**

State

🎙 Spoken answer · ~66 s

The error includes the lock ID, who holds it, the operation, the time and the version. I check whether that operation is really still running: look at the CI job, ask the person named in the lock, and check whether the process or runner still exists. Locks become stale when a job is killed, a laptop sleeps, or a network connection drops. If I am confident nothing is running, I release it with force-unlock and the lock ID, or for the S3 lock file backend by removing the lock object, and for DynamoDB by deleting the item. Then I run a plan and inspect the result carefully, because the interrupted run may have created some resources without recording them. If state is missing resources, I import them, and if there is any doubt I restore a previous state version from bucket versioning. Afterwards, I prevent cancelling apply jobs in CI and add alerts for locks held longer than normal.

Likely follow-ups

- How do you tell if a run is still active?
- Why avoid cancelling apply jobs?

First prove no apply is running (check CI, ask the owner, read lock info). If it is stale, use terraform force-unlock with the lock ID. Then run plan to verify state integrity, and handle any partially applied changes.

```
terraform force-unlock <LOCK_ID>
```

```
aws dynamodb get-item --table-name tf-locks --key '{"LockID":{"S":"tf-state/prod.tfstate"}}'
```

```
aws s3 rm s3://tf-state/prod.tfstate.tflock
```

```
terraform plan
```

Interview tip: Forcing an unlock during a live apply can corrupt state, so verification comes first.

📚 Official docs

- Terraform: State
- Terraform: State locking
- Terraform CLI: apply

certs.eknathalabs.com

Q159 **After an apply you see 'Provider produced inconsistent result after apply'. What does it mean and how do you recover?** State

🎙 Spoken answer · ~64 s

Terraform computes a plan with expected values, applies it through the provider, and then checks that the values returned match what was promised. When they do not, for example an attribute was planned as one value and the API returned another, or an unknown value resolved differently, Terraform reports that the provider produced an inconsistent result. It indicates a bug in the provider or an API that changes data asynchronously. The resource might have been created, and state may record it as tainted or partially filled. I run another plan to see what Terraform believes, look at the real object in the console, and often a refresh-only apply or a simple re-apply fixes the state, since the second run sees the real value. If it keeps happening, I search the provider's issue tracker, try a different provider version, or add a workaround such as ignore_changes. I also report the issue with reproduction steps.

Likely follow-ups

- Why does Terraform verify results after apply?
- How would you report a provider bug?

The provider returned a value different from what the plan predicted, a provider bug or eventual consistency issue. The resource may exist but state may be tainted or partial. Re-run plan, check the real object, refresh or import, and upgrade or report the provider.

```
terraform plan
```

```
terraform state show aws_xyz.name
```

```
terraform apply -refresh-only
```

```
terraform providers
```

Interview tip: This is almost always a provider bug, not user error, so check the issue tracker and try a newer or older provider version.

📚 Start here (official docs)

- Terraform documentation

certs.eknathalabs.com

Q160 **Terraform fails with 'Cycle: ...' errors. How do you find and break the dependency cycle?** Modules

🎙 Spoken answer · ~64 s

Terraform builds a directed acyclic graph from references between resources, and if A depends on B and B depends on A, directly or through a longer chain, there is no valid order and it reports a cycle with the list of involved objects. I read the error, which names the nodes, and visualise the graph around them if needed. The usual causes are two security groups that each reference the other in inline rules, a resource that uses an output of something that itself depends on it through a provisioner or a module output, or an overuse of depends_on. To break the cycle I separate the part that creates the circular reference from the creation of the object: for security groups, create both groups first and then add the rules as standalone rule resources, or use a data source or variable instead of a direct reference. I then re-validate and check the plan order.

Likely follow-ups

- Why do inline security group rules cause cycles?
- What does depends_on add to the graph?

Two or more resources reference each other, directly or through intermediaries, so no valid order exists. Read the cycle path in the error, use terraform graph to visualise, and break it by splitting a resource, using separate rule resources, or moving the shared value into a data source or variable.

```
terraform validate
```

```
terraform graph -type=plan | dot -Tsvg > g.svg
```

```
terraform graph | grep -E 'aws_security_group.a|aws_security_group.b'
```

Interview tip: The classic case is two security groups referencing each other inline. Use aws_security_group_rule resources instead.

📚 Start here (official docs)

- Terraform documentation

certs.eknathalabs.com

Q161 **for_each fails with 'The for_each value depends on resource attributes that cannot be determined until apply'. How do you fix it?**

Modules

🎙 Spoken answer · ~65 s

To plan instance addresses, Terraform must know the set of keys for for_each before anything is created. If I derive the keys from something computed during apply, such as the IDs of subnets that are being created in the same run, it cannot do that and fails. The fix is to design the keys to come from static information: variable inputs, names I choose, or values known at plan time. For example, key the map by a logical name such as the availability zone or role, and use the unknown ID only in the values, which are allowed to be unknown. If the structure truly depends on infrastructure that does not exist yet, I split the configuration into layers so the first creates the dependencies and the second reads them as data, or as a last resort do a targeted first apply, which I document and avoid in automation. Count has the same rule for its number.

Likely follow-ups

- Why can values be unknown but keys cannot?
- What is a two-layer apply?

for_each keys must be known at plan time. Build keys from static values or inputs, not from attributes of resources not yet created. Use a stable map keyed by names, split the apply with -target as a last resort, or restructure into two layers.

```
terraform plan
```

```
for_each = { for s in var.subnets : s.name => s }
```

```
for_each = toset(var.names)   # not aws_subnet.x[*].id
```

Interview tip: Keys must be known; values can be unknown. Use human-chosen names as keys and the unknown ID as the value.

📚 Official docs

- Terraform: for_each
- OpenTelemetry: Semantic conventions

certs.eknathalabs.com

Q162 **A tag or name change makes Terraform want to replace a production resource. How do you avoid the replacement?** Production

🎙 Spoken answer · ~60 s

The plan marks the attribute that forces replacement. For many resources, names and identifiers are immutable in the cloud API, so changing them means destroy and create. I first ask whether the change is intended. If someone refactored a naming convention in a shared module, the right action is often to revert the value for existing resources, adding a variable to preserve the legacy name. If the provider supports in-place rename through a different argument, I use that. Where a field is managed elsewhere, ignore_changes can suppress the diff. For stateful resources like databases, I add prevent_destroy and deletion protection so plan fails loudly. If replacement is genuinely needed, I plan a migration: create the new resource, copy data, switch consumers, and then remove the old one, possibly importing or using moved blocks. I never merge a plan that unexpectedly destroys production data.

Likely follow-ups

- Why do names often force replacement?
- How do you migrate data before replacing?

Check which attribute forces replacement in the plan. If it is only metadata you want to keep, restore the old value, use ignore_changes, rename in place if the provider supports it, or use moved and lifecycle controls. Never apply a surprise replacement of stateful resources.

```
terraform plan | grep -B3 'forces replacement'
```

```
terraform state show aws_db_instance.main | grep -E 'identifier|name'
```

```
lifecycle { ignore_changes = [identifier] }
```

Interview tip: Read the (forces replacement) marker. It tells you exactly which argument is immutable on the provider side.

📚 Official docs

- Terraform: lifecycle meta-argument

certs.eknathalabs.com

Q163 **A secret ended up in the Terraform state and in plan output in CI logs. What is your response?**

Security

🎙 Spoken answer · ~63 s

First I treat the secret as compromised and rotate it at its source, updating consumers. Cleaning up logs and state is secondary, because copies may exist in artifacts, caches and backups. Next I assess exposure: who could read the CI logs and the state bucket, including old object versions, and what access logs show about reads. I redact or delete log entries where the platform allows, and remove old state versions that contain the secret if policy permits. Then I fix the cause. I mark variables and outputs as sensitive so they are masked in output, move secrets out of Terraform by passing references to a secret manager and letting workloads fetch at runtime, use write-only and ephemeral features where available, mask values in CI, and avoid printing plans with unredacted values. I add pipeline scanning for secrets in code, tighten state access to pipeline roles, and write up lessons learned.

Likely follow-ups

- Why is rotation more important than log cleanup?
- How do you mask secrets in CI logs?

Rotate the secret immediately, remove it from logs where possible, restrict who can read state and logs, then fix the root cause by using sensitive values, ephemeral or write-only attributes, runtime secret references, and log masking. Audit access to the state and logs.

```
terraform state pull | jq '.. | objects | select(has("password"))' | head
```

```
aws s3api list-object-versions --bucket tf-state --prefix prod/
```

```
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=ResourceName,AttributeValue=tf-state
```

Interview tip: Rotation matters more than cleanup. Assume anyone who could read the logs or state has the secret.

📚 Official docs

- Terraform: State
- Terraform CLI: plan

certs.eknathalabs.com

Q164 **Apply fails halfway because of an account quota or service limit error. How do you proceed safely?**

Production

🎙 Spoken answer · ~64 s

A failure in the middle of apply does not undo earlier steps, and Terraform writes the state for everything that succeeded, so the right move is usually to fix the cause and run apply again, not destroy. I read the error to identify the limit, for example number of vCPUs, elastic IPs or load balancers per region. I request a quota increase through the service quotas API or support, or change the plan to stay within the limit, perhaps by using a different region or instance family. While waiting I run a plan to see what remains, and confirm that no resource was created in the cloud but missing from state, which can occur if the provider crashed after creating something, and import it if so. Then I apply again, which creates the remaining resources. To prevent surprises, I check quotas before large rollouts, request increases in advance, and monitor quota usage with alarms.

Likely follow-ups

- What if a resource is created but not in state?
- How do you pre-check quotas?

Terraform records what was created, so state reflects the partial progress. Request the quota increase or reduce the change, then re-run plan and apply to finish. Check for resources created but not recorded, and avoid destroying the half-built stack.

```
terraform apply 2>&1 | tee apply.log
```

```
terraform plan
```

```
aws service-quotas get-service-quota --service-code ec2 --quota-code L-1216C47A
```

```
aws service-quotas request-service-quota-increase --service-code ec2 --quota-code L-1216C47A --desired-value 256
```

Interview tip: Check quotas in the plan phase with a pre-flight script for big launches, so you do not discover limits mid-apply.

📚 Official docs

- Terraform CLI: apply

certs.eknathalabs.com

Q165 **Terraform says a resource 'already exists' on create, such as an S3 bucket, IAM role or DNS record. How do you handle it?** Production

🎙 Spoken answer · ~66 s

The cloud API is telling me that the name or identifier is already taken. Before acting, I find out who created it. If it was created by hand, or by an earlier apply that failed to save state, and this configuration is meant to own it, I import it, via an import block or the import command, and then run a plan to see any differences between the configuration and the object and decide whether to adjust the code or let Terraform change the resource. If another Terraform state already manages it, I must not import it in a second place; instead I decide which stack owns it and reference it with a data source from the other. If it is an unrelated object with a clashing name, I rename mine, especially for globally unique names such as buckets. I avoid deleting the existing object until I understand what depends on it, and update the documentation on ownership.

Likely follow-ups

- Why must one object have one owning state?
- How do you find which state manages an object?

The object exists in the cloud but not in your state, from a manual creation, another stack, a previous partial apply or a failed state write. Decide who owns it: import it into this state, rename yours, or remove the duplicate. Avoid deleting without checking ownership.

```
aws iam get-role --role-name app-role
```

```
terraform import aws_iam_role.app app-role
```

```
terraform state list | grep app-role
```

```
import { to = aws_iam_role.app id = "app-role" }
```

Interview tip: Search other states first. Two stacks managing the same object will fight forever.

📚 Official docs

- RFC 1035: DNS implementation and specification
- AWS IAM: Roles
- Amazon S3 User Guide

certs.eknathalabs.com

Q166 **A provider major upgrade breaks many plans, for example the AWS provider splitting S3 bucket arguments into separate resources. How do you migrate safely?** Modules

🎙 Spoken answer · ~63 s

I start with the provider's upgrade guide and changelog to understand breaking changes, such as arguments removed from a resource and moved into a standalone resource. I keep production pinned to the old version while working in a branch with a non-production state copy. Upgrading the lock file and running plan reveals the impact per state. My goal is no destroys and no replacements: I refactor the code to the new structure, and where new resource types need to take over management of existing settings, I import them or use import blocks so that state matches reality. Modules shared by many stacks are updated and versioned first, then consumers. I check the count of destroy actions in plan JSON as an automated gate. Rollout proceeds from development to production with applies, and I keep a rollback plan of pinning the previous provider version and restoring state backups if something goes wrong.

Likely follow-ups

- How do you automate the no-destroy gate?
- Why upgrade shared modules first?

Read the upgrade guide, pin the old version, upgrade in a branch, run plans per state to see diffs, refactor configuration to the new resource model, import or move where needed so no infrastructure is replaced, then roll out environment by environment.

```
terraform init -upgrade
```

```
terraform plan -out=upgrade.plan
```

```
terraform show -json upgrade.plan | jq '[.resource_changes[] |
select(.change.actions|index("delete"))] | length'
```

```
terraform providers lock
```

Interview tip: The success criterion is a plan with no destroys, and ideally no changes at all, before any real apply.

📚 Official docs

- AKS: Upgrade a cluster
- Amazon S3 User Guide

certs.eknathalabs.com

Q167 **Two pipelines applied to the same state at nearly the same time and one failed or overwrote the other. How do you prevent concurrent applies?**

CI/CD

🎙 Spoken answer · ~64 s

Backend locking stops two runs from writing state at the same time, but a race can still occur if a misconfigured backend has no locking, or when a second job queues behind the first, then runs a plan that was computed earlier against older state. I fix it in layers. The backend must support locking and it must be enabled, which I verify by testing. In CI, I define a concurrency group per state or environment so only one apply job runs at a time and others wait, and I set it to queue rather than cancel running jobs. Applying a saved plan means a stale plan is rejected if the state changed meanwhile. I forbid local applies to shared states by removing write credentials from individuals and permitting only the pipeline role. Atlantis or HCP Terraform provide queueing and locks natively. Finally, I make sure deploy jobs have timeouts and clear failure handling.

Likely follow-ups

- Why queue instead of cancel apply jobs?
- What does a stale plan error mean?

Locking prevents simultaneous writes if configured, but queued or retried jobs can still race on stale plans. Use CI concurrency groups per state, one apply per environment at a time, saved plans that fail when state changed, and no manual local applies.

```
concurrency: group: terraform-prod-network cancel-in-progress: false
```

```
terraform apply tfplan   # fails with 'Saved plan is stale' if state changed
```

```
terraform force-unlock <id>
```

Interview tip: Never set cancel-in-progress to true for apply jobs. Cancelling mid-apply is how stale locks and partial state happen.

📚 Start here (official docs)

- Terraform documentation

certs.eknathalabs.com

Q168 **Terraform init or plan fails with 403 or AccessDenied against the state backend after a role or bucket policy change. How do you debug it?**

Backend

🎙 Spoken answer · ~64 s

I first confirm which identity Terraform is actually using, because the wrong profile, an expired session or a changed role trust policy gives surprising results. Then I identify the failing API call from the error message or CloudTrail. The S3 backend needs permission to list the bucket, read and write the state object, and, if used, to create and delete the lock object or items in the lock table. If the bucket is encrypted with a customer-managed key, the role also needs permission to use that key for encrypt and decrypt, and the key policy must allow it, which is easy to overlook after a policy tightening. Bucket policies with explicit denies, VPC endpoint restrictions or condition keys such as source IP and organization ID may also block requests from the CI runner's network. The policy simulator and the CloudTrail event details point to the denying statement, which I then fix with least privilege.

Likely follow-ups

- Why does the KMS key policy matter?
- How can a VPC endpoint policy cause 403?

Identify which identity Terraform used and which action was denied. S3 needs list on the bucket, get and put on the object, and KMS permissions; locking needs lockfile or DynamoDB rights. Compare the policy simulator results and CloudTrail denial events.

```
aws sts get-caller-identity
```

```
aws s3api head-object --bucket tf-state --key prod/network.tfstate
```

```
aws iam simulate-principal-policy --policy-source-arn <role> --action-names s3:GetObject
s3:PutObject kms:Decrypt --resource-arns arn:aws:s3:::tf-state/prod/network.tfstate
```

```
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=EventName,AttributeValue=GetObject
```

Interview tip: KMS key policies are a frequent hidden cause. The role can read the object but cannot decrypt it.

📚 Official docs

- Terraform: State
- Terraform: Backends
- Terraform CLI: plan

certs.eknathalabs.com

Q169 **A data source returns multiple results or none, causing 'Your query returned no results' or 'multiple results' errors. How do you make lookups robust?** Modules

🎙 Spoken answer · ~67 s

Data sources that search by attributes are fragile: an extra match in the account, a deleted image, or a naming change turns into a failed plan, or worse, silently chooses a different object. For lookups that should return exactly one item, I use precise filters, such as owner account, a release tag, and a specific name pattern, and set most_recent only when that is the real intent. For critical dependencies like AMIs, I prefer passing an explicit ID or reading it from a controlled source such as an SSM parameter that the image pipeline updates, so changes are deliberate and reversible, rather than letting the next plan pick whatever is newest. I add preconditions to verify properties of the result and produce readable errors. In modules I often accept the ID as an input and keep the lookup in the root module. Where the object belongs to another Terraform state, an output or remote state reference is more reliable than searching.

Likely follow-ups

- Why is most_recent risky for AMIs?
- When use remote state over a data source?

Make filters specific and deterministic, use most_recent or unique identifiers, or avoid fuzzy lookups for critical resources by passing IDs explicitly. Add preconditions to fail clearly and pin inputs such as AMI IDs through a managed source.

```
data "aws_ami" "app" { most_recent = true owners = ["123456789012"] filter { name = "name" values = ["app-*"] } filter { name = "tag:release" values = [var.release] } }
```

```
aws ec2 describe-images --owners 123456789012 --filters Name=name,Values='app-*' --query 'Images[].ImageId'
```

Interview tip: Moving AMI selection to an SSM parameter or explicit variable makes rollbacks possible, whereas most_recent changes under you.

📚 Start here (official docs)

- Terraform documentation

certs.eknathalabs.com

Q170 **Someone applied against the wrong workspace or account and changed production while intending dev. How do you recover, and how do you prevent it?**

Production

🎙 Spoken answer · ~62 s

First I stop anyone else from applying, then work out exactly what changed using the apply log, the state versions and cloud audit logs. If resources were modified, I decide whether to revert by applying the correct configuration to the affected environment, or to restore a previous state version and reconcile. If something was destroyed, I restore from backups or recreate and import. I communicate quickly with stakeholders and run a blameless review. The deeper issue is that the design allowed one command to reach multiple environments. I prevent it by using separate state backends and separate cloud accounts per environment, with credentials that only work for one, so a developer's default login cannot touch production at all. The provider setting for allowed account IDs aborts if credentials mismatch. Workspaces alone are weak isolation. Production changes go only through the pipeline with approvals, and prompts show the environment.

Likely follow-ups

- Why are workspaces weak isolation?
- What does allowed_account_ids do?

Stop further applies, assess the damage using the plan and cloud audit logs, restore or revert via a corrective apply or state backup, communicate. Prevent by separate backends and credentials per environment, no shared default workspace, account guards, and CI-only applies.

```
terraform workspace show
```

```
terraform state pull | jq '.serial,.lineage'
```

```
aws sts get-caller-identity
```

```
provider "aws" { allowed_account_ids = ["111122223333"] }
```

Interview tip: allowed_account_ids on the provider aborts the run if the credentials point to the wrong account, a cheap and effective guard.

📚 Start here (official docs)

- Terraform documentation

certs.eknathalabs.com

Q171 **A policy check blocks your pull request for a rule you believe is a false positive. How should exceptions be handled?**

Security

🎙 Spoken answer · ~61 s

I first investigate whether the finding is truly wrong: read the rule's intent, check the resource's real configuration, and consider whether another control already covers the risk. If it is a genuine false positive or an accepted risk, I follow the exception process rather than bypassing the pipeline. In code, I add a narrowly scoped inline suppression for that specific resource and rule with a comment that gives the justification and a ticket reference, so reviewers see it in the diff. The security or policy owner approves it, and where the policy engine supports it, the exception has an expiry date so it is revisited. I never turn off the rule across the repository or make the check non-blocking to get a merge through. If the same false positive appears repeatedly, I propose improving the rule or the module default. Exceptions are reported periodically to track risk.

Likely follow-ups

- Why should exceptions expire?
- How do you improve a noisy rule?

Verify whether it is truly a false positive, then handle the exception transparently: a documented, scoped suppression in code with justification and expiry, approved by the policy owner, tracked, and re-reviewed. Don't disable the check globally or bypass the pipeline.

```
checkov -d . --check CKV_AWS_18 --skip-check CKV_AWS_18
```

```
#checkov:skip=CKV_AWS_18: access logs go to central log bucket, ticket SEC-1234
```

```
conftest test plan.json -p policy/ --no-fail
```

```
opa eval -d policy/ -i plan.json 'data.terraform.deny'
```

Interview tip: Good exceptions have an owner, a reason, a ticket and an expiry date, and are visible in code review.

📚 Start here (official docs)

- Terraform documentation

certs.eknathalabs.com

Q172 **HCP Terraform or your CI runs are stuck in 'pending' or queued for a long time. What do you check?** Production

🎙 Spoken answer · ~65 s

In HCP Terraform, each workspace processes one run at a time in order, so a run waiting for manual approval, a failed plan that has not been discarded, or a long apply blocks the runs behind it. I look at the workspace run list for the head of the queue and what it is waiting for. If runs use self-hosted agents, I check that agents are online, have capacity and match the pool, because an empty pool leaves runs pending indefinitely. Run triggers can chain workspaces, so an upstream failure or pending run delays dependents. Organisation-level concurrency limits on the plan can also queue runs. In a CI system the equivalent causes are concurrency groups, no available runners, or a stuck job holding the group. I resolve it by approving or discarding the blocking run, scaling the agents or runners, and I set auto-discard and timeouts to avoid recurrence. Alerts on queue age catch issues early.

Likely follow-ups

- Why does one run block the workspace?
- How would you scale self-hosted agents?

Another run on the same workspace holds the queue or lock, a run is waiting for confirmation, an agent pool has no available agents, run triggers chain upstream runs, or organisation concurrency limits are reached. Inspect the run queue and agent status.

```
tfe-cli runs list --workspace prod-network
```

```
kubectl get pods -n tfc-agent
```

```
gh run list --workflow terraform.yml --status queued
```

```
terraform force-unlock <id>
```

Interview tip: Check for a plan waiting for manual approval first. It blocks everything behind it in the workspace queue.

📚 Official docs

- Kubernetes: Debug Pods
- HCP Terraform documentation

certs.eknathalabs.com

Q173 **You must split a monolithic Terraform state into three smaller ones without recreating anything. Walk through the process.**

Modules

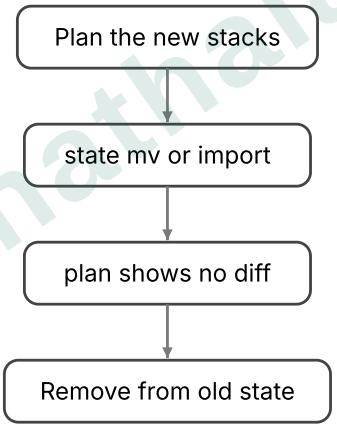
🎙 Spoken answer · ~69 s

I begin with preparation: freeze changes, take a backup of the state, and map which resources belong to which new stack based on lifecycle and ownership, such as network, data and applications. I create the new root modules with their own backends and copy the relevant configuration, replacing direct references to resources in other stacks with outputs consumed through remote state or data sources. Then I move the resource entries. Using state move with an output state file, or import blocks into the new state followed by removed blocks in the old, I transfer each group without touching real infrastructure. After every move I run a plan in both the old and the new root, expecting no changes and certainly no creates or destroys. I move the groups in dependency order, starting with the lowest layer. Once all stacks are clean, I delete the leftover configuration, update the pipelines, permissions and documentation, and keep the backup until the new setup has run successfully for a while.

Likely follow-ups

- Why move in dependency order?
- How do import blocks help with splits?

🗺 Splitting state



Back up state, create new roots and backends, move resource addresses with terraform state mv -state-out or moved/import blocks, replace cross-references with outputs and remote state or data sources, and verify each new root shows a clean plan before decommissioning the old config.

```
terraform state pull > monolith-backup.tfstate
```

```
terraform state mv -state-out=network.tfstate module.network module.network
```

```
terraform init -migrate-state
```

```
terraform plan   # in each root, expect: No changes
```

Interview tip: Freeze changes during the migration, and make the old root's plan show no destroys at each step before moving on.

certs.eknathalabs.com

📚 **Official docs**

- Terraform: State
- Terraform: lifecycle meta-argument

certs.eknathalabs.com